



CSC 404

MACHINE LEARNING



MIVA OPEN UNIVERSITY

HEADQUARTERS

PLOT 1059, O.P. FINGESI ROAD,

UTAKO, ABUJA

COURSE DEVELOPMENT TEAM

Content Editor: Augustus Isichei, MSc Financial Engineering

Language Editor: Udochi Obiukwu, MA English Language

Instructional Designer: Yakub Kayode Saheed, Ph.D.

Table of Contents

Table of Contents	4
List of Figures	8
Course Introduction	10
Course Overview	11
Course Aims	12
Course Objective	13
Study Session One	15
Introduction	16
Learning Outcomes for Study Session One	18
1.1 What is Learning?	19
1.2 Supervised Learning	19
1.3 Unsupervised Learning	20
1.4 Reinforcement Learning	21
1.5 The Learner's Output	22
1.6 A Simple Data-Generation Model	23
1.7 Measures of Success	23
Conclusion	24
Summary	25
Study Session Two	31
Introduction	32
Learning Outcomes for Study Session Two	34
2.1 Python Ecosystem for Machine Learning	36
2.2 Python Installation	38
2.3 How To Install scikit-learn	40
2.4 Course in Python and SciPy	40
2.5 Assignment	41
2.6 Flow Control	43
2.7 Data Structures	45
2.8 Functions	48
2.9 NumPy Crash Course	49
2.10 Matplotlib Package	52
2.11 Pandas Crash Course	55
Conclusion	59

Summary	59
Study Session Three	65
Introduction	66
Learning Outcomes for Study Session Three	66
3.1 How To Load Machine Learning Data	69
3.2 Pima Indians Dataset	70
3.3 Understand Your Data With Descriptive Statistics	75
3.4 Look at Your Data	75
3.5 Dimensions of Your Data	77
3.6 Data Type For Each Attribute	78
3.7 Descriptive Statistics	79
3.8 Class Distribution (Classification Only)	81
3.9 Correlations Between Attributes	82
3.10 Skew of Univariate Distributions	83
3.11 Tips To Remember	84
Conclusion	85
Summary	86
Study Session Four	91
Introduction	92
Learning Outcomes for Study Session Four	92
4.1 Visualisation	94
4.2 Univariate Plots	94
4.2.1 Histograms	94
4.2.2 Density Plots	96
4.2.3 Box and Whisker Plots	98
4.3 Multivariate Plots	100
4.3.1 Correlation Matrix Plot	101
4.3.2 Scatter Plot Matrix	104
Conclusion	107
Summary	108
Study Session Five	113
Introduction	114
Learning Outcomes for Study Session Five	114
5.1 Prepare Your Data For Machine Learning	116
5.2 Need For Data Pre-Processing	116

5.3 Rescale Data	118
5.4 Standardise Data	119
5.5 Normalise Data	121
5.6 Binarise Data (Make Binary)	122
5.7 Feature Selection For Machine Learning	123
5.7.1 Univariate Selection	124
5.7.2 Recursive Feature Elimination	125
5.7.2 Principal Component Analysis	127
5.7.3 Feature Importance	128
5.8 Evaluate Machine Learning Algorithms	130
5.9 Split into Train and Test Sets	131
5.10 K-fold Cross-Validation	133
5.11 Leave-One-Out Cross-Validation	134
5.12 Repeated Random Test-Train Splits	136
5.13 Machine Learning Algorithm Performance Metrics	138
5.14 Algorithm Evaluation Metrics	138
5.15 Classification Metrics	139
5.15.1 Classification Accuracy	139
5.15.2 Logarithmic Loss	141
5.15.3 Area Under ROC Curve	142
5.15.4 Classification Report	145
5.16 Regression Metrics	146
5.16.1 Mean Absolute Error	147
5.16.2 Mean Squared Error	148
5.16.3 R2 Metric	149
Conclusion	151
Summary	151
Study Session Six	157
Introduction	158
Learning Outcomes for Study Session Six	158
6.1 Spot-Check Classification Algorithms	160
6.2 Algorithm Spot-Checking	160
6.3 Algorithms Overview	161
6.4 Linear Machine Learning Algorithms	162
6.4.1 Logistic Regression	162

6.4.2 Linear Discriminant Analysis	163
6.5 Nonlinear Machine Learning Algorithms	164
6.5.1 k-Nearest Neighbours	165
6.5.2 Naive Bayes	166
6.5.3 Classification and Regression Trees	167
6.5.4 Support Vector Machines	168
6.6 Spot-Check Regression Algorithms	169
6.7 Algorithms Overview	170
6.8 Linear Machine Learning Algorithms	171
6.8.1 Linear Regression	171
6.8.2 Ridge Regression	173
6.8.3 LASSO Regression	174
6.8.4 ElasticNet Regression	176
6.9 Nonlinear Machine Learning Algorithms	177
6.9.1 K-Nearest Neighbours	177
6.9.2 Classification and Regression Trees	178
6.9.3 Support Vector Machines	179
6.10 Compare Machine Learning Algorithms	181
6.11 Choose The Best Machine Learning Model	181
6.12 Compare Machine Learning Algorithms Consistently	182
6.13 Automate Machine Learning	186
6.13.1 Workflows with Pipelines	186
6.13.2 Automating Machine Learning Workflows	186
6.13.3 Data Preparation and Modelling Pipeline	187
6.13.4 Feature Extraction and Modelling Pipeline	189
Conclusion	192
Summary	192
Study Session Seven	199
Introduction	200
Learning Outcomes for Study Session Seven	200
7.1 Improve Performance with Ensemble	202
7.2 Combine Models Into Ensemble Predictions	202
7.2.1 Bagging	202
7.2.2 Boosting	202
7.2.3 Voting	203

7.2.4 Bagging Algorithms	203
7.2.5 Random Forest	204
7.2.6 Extra Trees	205
7.3 Boosting Algorithms	207
7.3.1 AdaBoost	207
7.3.2 Stochastic Gradient Boosting	209
7.3.3 Voting Ensemble	210
7.4 Improve Performance with Algorithm Tuning	212
7.5 Machine Learning Algorithm Parameters	212
7.5.1 Grid Search Parameter Tuning	213
7.5.2 Random Search Parameter Tuning	213
7.6 Finalise Your Model with Pickle	215
7.7 Finalise Your Model with Joblib	217
Conclusion	220
Summary	220
Study Session Eight	226
Introduction	227
Learning Outcomes for Study Session Eight	227
8.1 Use A Structured Step-By-Step Process	228
8.2 Machine Learning Project Template in Python	228
8.2.1 Template Summary	228
8.2.2 How To Use The Project Template	229
8.3 Prepare Problem	230
8.4 Summarise Data	230
8.5 Prepare Data	231
8.6 Evaluate Algorithms	231
8.7 Improve Accuracy	232
8.8 Finalise Model	233
Conclusion	234
Summary	234
References/Further Reading	240
Glossary of Terms	242

List of Figures

Figure 1.1: Supervised learning algorithms	20
--	----

Figure 1.2 Unsupervised learning	21
Figure 2.1: Line Plot Matplotlib	53
Figure 2.2: Scatter Plot with Matplotlib	54
Figure 4.1: Histograms of each attribute	96
Figure 4.2: Density plots of each attribute	98
Figure 4.3: Box and whisker plots of each attribute	100
Figure 4.4: Correlation matrix plot	102
Figure 4.5 Generic correlation matrix plot	104
Figure 4.6: Scatter plot matrix of the data	106
Figure 6.1: Algorithm comparison	185

Course Introduction

Over the past couple of decades, Machine Learning has become one of the mainstays of information technology and, with that, a rather central, albeit usually hidden, part of our lives. With the ever-increasing amounts of data becoming available, there is good reason to believe that smart data analysis will become even more pervasive as a necessary ingredient for technological progress.

Machines have come a long way since the Industrial Revolution. They continue to fill factory floors and manufacturing plants, but now their capabilities extend beyond manual activities to cognitive tasks that, until recently, only humans were capable of performing. Judging song competitions, driving automobiles, and mopping the floor with professional chess players are three examples of the specific complex tasks machines are now capable of simulating.

Skill Requirements

Understanding of Classical Statistics: Classical statistics is at the heart of machine learning, and many of these algorithms are based on the same statistical equations you studied in high school. Indeed, statistical algorithms were conducted on paper well before machines ever took on the title of artificial intelligence.

Computer programming is another indispensable part of machine learning.

There isn't a click-and-drag or Web 2.0 solution to perform advanced machine learning.

learning in the way one can conveniently build a website nowadays with WordPress or Strikingly. Programming skills are therefore vital to manage data and design statistical models that run on machines.

Course Overview

Machine Learning has evolved over recent decades from a niche field into a cornerstone of modern information technology, becoming an essential, though often invisible, part of daily life. As the volume of available data continues to grow, smart data analysis is increasingly viewed as a vital component for technological advancement.

This course, **CSC 404**, is designed to transition students from theoretical understanding to practical mastery of machine learning. It covers the entire lifecycle of a machine learning project, starting from the fundamental concepts of how machines "learn" through experience to the deployment of complex predictive models. Students will explore various learning paradigms, including:

- **Supervised Learning:** Learning patterns from labelled datasets to predict outcomes.
- **Unsupervised Learning:** Uncovering hidden structures and patterns within unlabelled data.
- **Reinforcement Learning:** Understanding how agents learn through interactions and success measures.

The curriculum is heavily grounded in the **Python ecosystem**, specifically leveraging industry-standard libraries such as **NumPy, Matplotlib, Pandas, and scikit-learn**. Throughout the course, students will engage in hands-on sessions involving data preparation (rescaling and standardising), feature selection, and algorithm evaluation using real-world datasets, like the Pima Indians diabetes dataset. By the conclusion of the course, students will be equipped with a structured six-step template to manage end-to-end predictive modelling projects.

Course Aims

The primary objectives of the CSC 404 course are to enable students to:

1. **Grasp Fundamental Learning Concepts:** Develop a deep understanding of what "learning" means in both biological and computational contexts, and distinguish between the various categories of machine learning.
2. **Master the Python ML Ecosystem:** Attain proficiency in using Python and its core libraries (NumPy, Pandas, and Matplotlib) to load, manipulate, and visualise complex datasets for analysis.
3. **Execute Effective Data Pre-processing:** Learn to prepare raw data for machine learning by applying transformation techniques such as normalisation and binarisation, and by performing automated feature selection to improve model efficiency.
4. **Evaluate and Optimise Algorithms:** Gain the skills to "spot-check" multiple linear and non-linear algorithms, compare their performance using

metrics like accuracy and logarithmic loss, and improve results through ensemble methods and algorithm tuning.

5. **Implement End-to-End Projects:** Master a structured, step-by-step process for defining, summarising, preparing, and finalising a predictive modelling project, ensuring the ability to deliver accurate results on unseen data.

Course Objective

The purpose of this course is to provide the students with an overview of what machine learning is all about and the vast range of applications that have at their heart a machine learning problem.

Working Through the Course

To complete this course, you are required to study all the units, the recommended textbooks, and other relevant materials. Each unit contains some Hands-on Sessions and some exercises; you are required to submit

References

Notes in this course have been compiled from the following textbooks: For further study, please refer to

Machine Learning for Absolute Beginners: Second Edition by Oliver Theobald.

Understanding Machine Learning: From theory to Algorithms (Shai Shalev-Shwartz)

The Hebrew University, Jerusalem (Shai Ben-David, University of Waterloo,
Canada)

Machine Learning Mastery with Python: Jason Brownlee

STUDY SESSION ONE

BACKGROUND OF MACHINE LEARNING

Introduction

In this session, we will examine the fundamental concept of learning, seeking to unravel its essence and significance. Learning encompasses the intricate process of acquiring new understanding, knowledge, behaviours, skills, values, attitudes, and preferences. This transformative journey isn't confined to humans alone; animals and even certain machines exhibit the capacity to learn. Evidence even suggests a semblance of learning in certain plant species. One can grasp the notion of learning as the acquisition of wisdom or proficiency through various channels—be it study, firsthand experiences, or formal instruction. A succinct definition by the Oxford Dictionary characterises learning as the assimilation of knowledge or skills through study, experience, or pedagogy. Building on this foundation, Susan Ambrose and her colleagues describe learning as a catalyst for change, stemming from experiences and culminating in improved performance and a broader platform for future learning endeavours.

This session of exploration extends beyond traditional learning into the realm of machine learning, a transformative discipline that has permeated numerous fields. Within this landscape, we encounter distinct categories. Firstly, supervised learning, the initial branch, centres on discerning patterns by correlating variables with known outcomes, working primarily with labelled datasets. The process entails feeding the machine examples featuring various attributes ("X") along with corresponding correct output values ("y"). This labelled dataset empowers the algorithm to discern latent patterns and construct a model to replicate these underlying rules when introduced to novel

data. Examples of supervised learning techniques encompass regression analysis, decision trees, k-nearest neighbours, neural networks, and support vector machines.

Moving on, unsupervised learning unfolds with a different approach. Here, not all variables and data patterns are predefined; instead, the machine uncovers hidden patterns and generates labels through specialised algorithms. A prominent illustration of this is the k-means clustering algorithm, which groups data points with similar attributes. The beauty of unsupervised learning lies in its capacity to unveil latent data patterns, illuminating hitherto unnoticed insights, such as the presence of distinct customer segments. Notably, unsupervised learning serves as a stepping stone for further analyses once discrete groups have been unveiled.

The session culminates with a glimpse into reinforcement learning. Various terminologies enrich our understanding, including the learner's input, domain set (X), label set (Y), and training data (S), which consists of labelled domain points. The learner's output involves generating a prediction rule ($h: X \rightarrow Y$) to anticipate labels for new instances. A simple data-generation model outlines how training data is created—instances are generated according to a probability distribution (D) over X , while labels are assigned based on an unknown function ($f: X \rightarrow Y$). The task lies in the learner's endeavour to deduce this function. Success in this context is gauged by a classifier's error, measured as the likelihood of mispredicting labels on random instances drawn from the distribution D .

This enthralling journey into learning systems and machine learning categories lays the groundwork for a deeper exploration of this transformative field.

Learning Outcomes for Study Session One

When you have studied this session, you should be able to:

- 1.1. Explain the concept of learning
- 1.2. Identify the transformative nature of learning, according to Susan Ambrose and her colleagues.
- 1.3. Recognise the significance of machine learning across various fields for information extraction from large datasets.
- 1.4. Differentiate between supervised learning and unsupervised learning within the realm of machine learning.
- 1.5. Describe the process of supervised learning.
- 1.6. Recognise the importance of labelled datasets and the creation of models for predicting outcomes.
- 1.7. Explain unsupervised learning.
- 1.8. Define the key terminologies associated with reinforcement learning.
- 1.9. Illustrate the process of generating training data in reinforcement learning.
- 1.10. Explain the concept of measuring success in reinforcement learning.

1.1 What is Learning?

What is learning? Learning is the process of acquiring new understanding, knowledge, behaviours, skills, values, attitudes, and preferences. The ability to learn is possessed by humans, animals, and some machines; there is also evidence for some kind of learning in certain plants (~wikipedia)

The acquisition of knowledge or skills through study, experience, or being taught. (Oxford Dictionary)

“A process that leads to change, which occurs as a result of experience and increases the potential for improved performance and future learning.” From How Learning Works: Seven Research-Based Principles for Smart Teaching by Susan Ambrose, et al.

- Learning Systems
- Machine Learning

In many fields today, machine learning has become a common tool in many tasks that involve information extraction from large datasets.

- Categories of Machine Learning

1.2 Supervised Learning

As the first branch of machine learning, supervised learning concentrates on learning patterns by connecting the relationship between variables and known outcomes and working with labelled datasets. Supervised learning works by feeding the machine sample data with various features (represented as “X”) and

the correct value output of the data (represented as “y”). The fact that the output and feature values are known qualifies the dataset as “labelled”. The algorithm then deciphers patterns that exist in the data and creates a model that can reproduce the same underlying rules with new data.

Examples of supervised learning algorithms include regression analysis, decision trees, k-nearest neighbours, neural networks, and support vector machines.

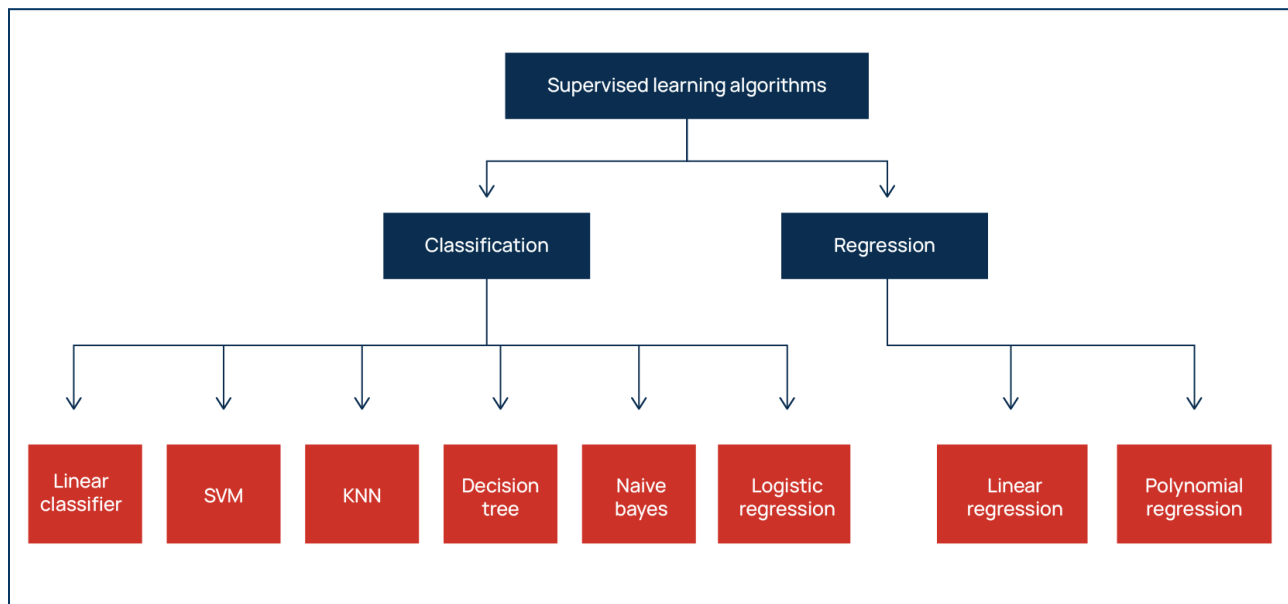


Figure 1.1: Supervised learning algorithms

1.3 Unsupervised Learning

In the case of unsupervised learning, not all variables and data patterns are classified. Instead, the machine must uncover hidden patterns and create labels through the use of unsupervised learning algorithms. The k-means clustering algorithm is a popular example of unsupervised learning. This simple algorithm groups data points that are found to possess similar features.

The advantage of unsupervised learning is that it enables you to discover patterns in the data that you were unaware existed—such as the presence of two major customer types. Clustering techniques such as k-means clustering can also provide a springboard for conducting further analysis after discrete groups have been discovered.

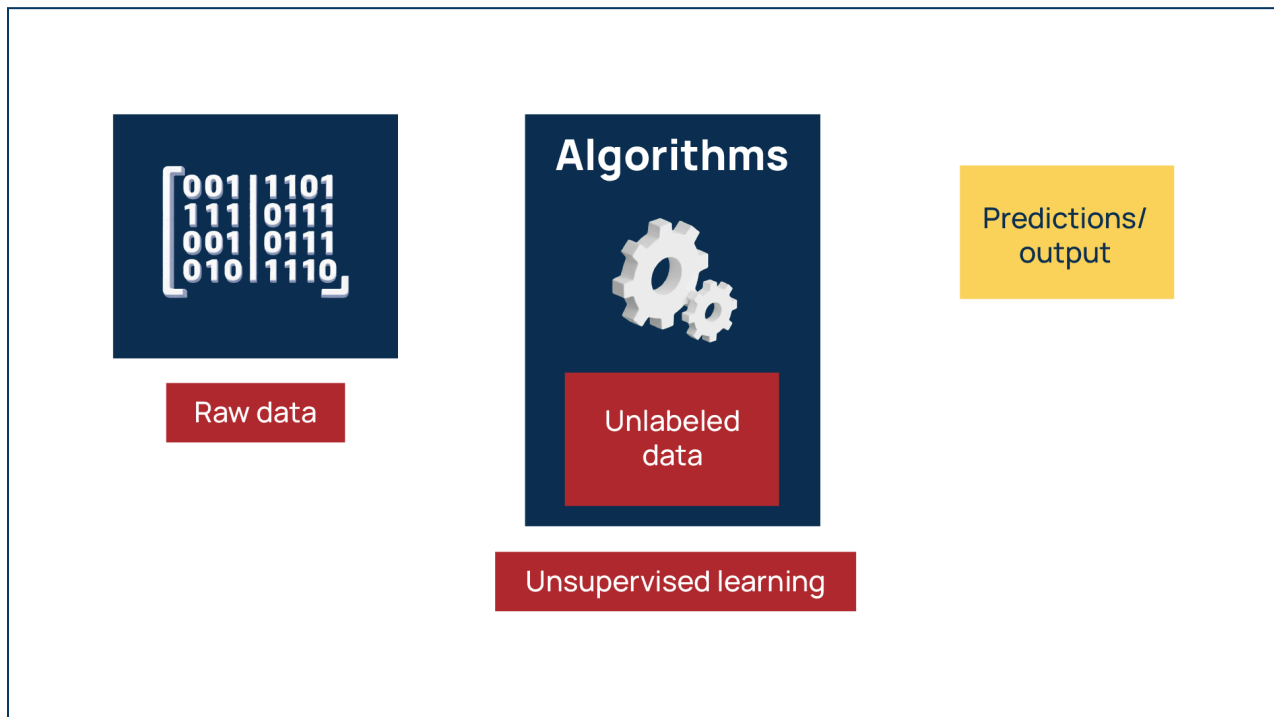


Figure 1.2 Unsupervised learning

1.4 Reinforcement Learning

Some Terminologies

1. **The Learner's Input:** In the basic statistical learning setting, the learner has access to the following:
2. **Domain Set:** An arbitrary set, X . This is the set of objects that we may wish to label. For example, in the papaya learning problem mentioned before,

the

3. The domain set will be the set of all papayas. Usually, these domain points will be represented by a vector of features (like the papaya's colour and softness). We also refer to domain points as instances and to X as instance space.
4. **Label Set:** For our current discussion, we will restrict the label set to be a
5. two-element set, usually $\{0, 1\}$ or $\{-1, +1\}$. Let Y denote our set of possible labels. For our papaya example, let Y be $\{0, 1\}$, where 1 represents being tasty and 0 stands for being not tasty.
6. Training data: $S = ((x_1, y_1), \dots, (x_m, y_m))$ is a finite sequence of pairs in $X \times Y$:

that is, a sequence of labelled domain points. This is the input that the learner has access to (like a set of papayas that have been tasted and their colour, softness, and tastiness). Such labelled examples are often called training examples. We sometimes also refer to S as a training set.

1.5 The Learner's Output

The learner is requested to output a prediction rule, $h : X \rightarrow Y$. This function is also called a 'predictor', a 'hypothesis', or a 'classifier'. The predictor can be used to predict the label of new domain points. In our papaya example, it is a rule that our learner will employ to predict whether future papayas he examines in the farmers' market are going to be tasty or not. We use the notation $A(S)$ to denote the hypothesis that a learning algorithm, A , returns upon receiving the training sequence S .

1.6 A Simple Data-Generation Model

How the training data is generated. First, we assume that the instances (the papayas we encounter) are generated by some probability distribution (in this case, representing the environment). Let us denote the probability distribution over X by D . It is important to note that we do not assume that the learner knows anything about this distribution. For the types of learning tasks we discuss, this could be any arbitrary probability distribution. As to the labels, in the current discussion we assume that there is some “correct” labelling function, $f : X \rightarrow Y$, and that $y_i = f(x_i)$ for all i . The labelling function is unknown to the learner. In fact, this is just what the learner is trying to figure out. In summary, each pair in the training data S is generated by first sampling a point x_i according to D and then labelling it by f .

1.7 Measures of Success

We define the error of a classifier as the probability that it does not predict the correct label on a random data point generated by the underlying distribution. That is, the error of h is the probability of drawing a random instance x , according to the distribution D , such that $h(x)$ does not equal $f(x)$.

Formally, given a domain subset, $A \subseteq X$, the probability distribution, D , assigns a number, $D(A)$, which determines how likely it is to observe a point $x \in A$. In many cases, we refer to A as an event and express it using a function $\pi : X \rightarrow \{0, 1\}$, namely $A = \{x \in X : \pi(x) = 1\}$. In that case, we also use the notation $P_{x \sim D}[\pi(x)]$ to express $D(A)$.

In-Text Questions and Answers (ITQs and ITAs)

ITQ 1.1: What does the concept of learning encompass?

ITA: Learning encompasses the process of acquiring understanding, knowledge, behaviours, skills, values, attitudes, and preferences through study, firsthand experiences, or formal instruction.

ITQ 1.2: How does Susan Ambrose define learning in the context of improved performance and future learning?

ITA: Susan Ambrose defines learning as a process that leads to change, resulting from experiences, and culminates in improved performance and increased potential for future learning.

Conclusion

The session explores the concept of learning, which encompasses the acquisition of knowledge and skills. It introduces machine learning's categories: supervised, which learns from labelled data; unsupervised, revealing hidden patterns; and reinforcement, learning through trial and error. Learning is portrayed as a transformative process leading to improved performance. The text underscores the interplay between traditional and machine learning, highlighting their shared essence of growth and enhancement.

Summary

In this study session, you have learnt to:

1. Define and explain the concept of learning as the process of acquiring understanding, knowledge, behaviours, skills, values, attitudes, and preferences through various channels, such as study, firsthand experiences, or formal instruction.
2. Recognise the significance of learning as a catalyst for change, improved performance, and future learning endeavours, drawing from the perspective presented by Susan Ambrose and her colleagues.
3. Differentiate between supervised learning and unsupervised learning in the context of machine learning, understanding their fundamental approaches, data requirements, and real-world applications.
4. Describe the process of supervised learning, which involves utilising labelled datasets to allow algorithms to identify patterns and construct models for predicting outcomes of new data instances. You can now provide examples of popular supervised learning techniques such as regression analysis, decision trees, k-nearest neighbours, neural networks, and support vector machines.
5. Explain the principles underlying unsupervised learning, where algorithms like k-means clustering are employed to discover hidden patterns and generate labels without pre-existing annotations. You comprehend the value of unsupervised learning in uncovering insights from data and facilitating further analysis.

6. Define key terminologies associated with reinforcement learning, including input, domain set (X), label set (Y), and training data (S). Furthermore, you understand the learner's task of generating a prediction rule ($h: X \rightarrow Y$) for new instances in the context of reinforcement learning.
7. Illustrate the data-generation model within reinforcement learning, where instances are generated following a probability distribution (D) over X , and labels are assigned based on an unknown function ($f: X \rightarrow Y$). You've gained insight into the learner's challenge of deducing this function and evaluating success through classifier error.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 1: What is the fundamental idea behind the concept of learning, and how does it differ from innate knowledge? Provide an example to illustrate your explanation.

Answer: The core concept of learning involves the acquisition of new knowledge, skills, behaviours, or understanding through experiences, study, or teaching. Learning is a dynamic process that leads to a lasting change in an individual's thoughts, actions, or perceptions. Unlike innate knowledge, which refers to inherent abilities or knowledge present from birth, learning is gained through external interactions and experiences.

SAQ 2: Discuss the key elements that Susan Ambrose and her colleagues suggest contribute to a transformative learning experience. How do these elements differ from traditional, non-transformative learning approaches?

Provide insights into how educators can facilitate such transformative experiences.

Answer: Susan Ambrose and her colleagues propose several key elements that contribute to a transformative learning experience:

- **Challenging Assumptions:** Transformative learning prompts learners to question their existing beliefs and assumptions. This differs from traditional learning, where information might be accepted passively. Educators can encourage this by presenting diverse perspectives and encouraging critical analysis.
- **Reflection:** Transformative learning involves reflection on both new information and how it aligns with one's existing beliefs. Regular opportunities for journaling, group discussions, or personal reflection can facilitate this element.
- **Dissonance:** Introducing cognitive dissonance – a discomfort between existing beliefs and new information – encourages learners to seek resolution through critical thinking. Traditional approaches may avoid such discomfort, while transformative learning embraces it.
- **Dialogue and Interaction:** Engaging in meaningful dialogue with peers and instructors fosters a deeper understanding of different viewpoints. Collaborative projects, debates, and interactive workshops can create these opportunities.
- **Application:** Applying new knowledge to real-world contexts helps

learners solidify their understanding and assess the practical implications. Hands-on experiences and case studies facilitate this element.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **Which of the following best defines learning in the context of computer science?**

- A) Acquiring new skills through study and experiences.
- B) Increasing the number of operations performed on a dataset.
- C) Learning from past failures only.
- D) Learning without data involvement.

Answer: A) Acquiring new skills through study and experiences.

Explanation: Learning in the context of machine learning is about acquiring knowledge and skills through various experiences, including study and interactions with data.

2. **What type of data does supervised learning work with?**

- A) Unlabelled data
- B) Random data
- C) Labelled data
- D) Unstructured data

Answer: C) Labelled data

Explanation: Supervised learning requires labelled data, where each data point is associated with the correct output value, helping the algorithm learn to predict the correct outcomes.

3. **Which of the following is an example of unsupervised learning?**

- A) Linear regression
- B) K-means clustering
- C) Neural networks
- D) Support vector machines

Answer: B) K-means clustering

Explanation: K-means clustering is a classic example of unsupervised learning, where the algorithm groups similar data points together without prior labels.

4. **What is the primary advantage of unsupervised learning?**

- A) It can generate labelled data
- B) It works only with simple datasets
- C) It helps uncover hidden patterns without predefined labels
- D) It uses fewer computational resources

Answer: C) It helps uncover hidden patterns without predefined labels

Explanation: Unsupervised learning is advantageous because it allows the discovery of data structures or patterns that were not previously known, without requiring labelled data.

5. **In reinforcement learning, how is the success of an algorithm typically measured?**

- A) By the number of labels predicted correctly
- B) By the amount of data processed
- C) By the error in predicting labels for new instances

D) By the number of iterations performed

Answer: C) By the error in predicting labels for new instances

Explanation: In reinforcement learning, the success is often gauged by the classifier's error, which is the probability that the learner will mispredict the label of a new data instance.

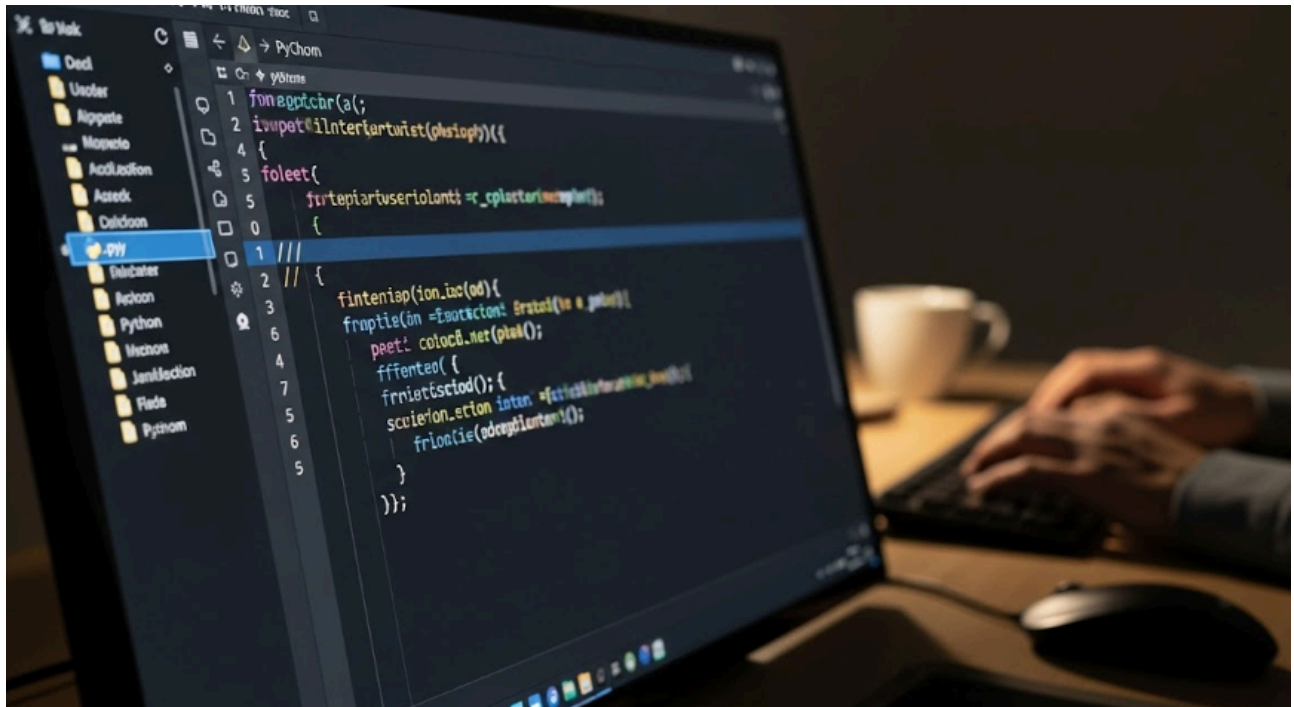
Tutor-Marked Assessments (TMAs)

1. Explain the concept of learning and its significance in personal and professional development.
2. Define machine learning and elaborate on its significance across various fields for information extraction from large datasets.
3. In machine learning, how does the concept of learning from experience correlate with traditional learning methods in humans and animals? Discuss how this analogy enhances our understanding of machine learning.
4. Analyse the role of error measurement in reinforcement learning. How does the calculation of misprediction errors guide the learning process, and what are the implications for algorithm refinement?

STUDY SESSION Two

PYTHON PROGRAMMING LANGUAGE

Introduction



In this session, we embark on a journey through the Python ecosystem for machine learning, uncovering its robust capabilities and advantages. The growth of the Python ecosystem suggests its potential to become the dominant platform for machine learning. This trajectory is fuelled by Python's versatility, which serves as both a research and development tool and a platform for production applications. As we delve into this session, we will delve into several key aspects:

- **Python's Ascendancy in Machine Learning:** We will begin by recognising Python as a general-purpose interpreted programming language that stands out for its readability. The Zen of Python encapsulates its guiding principles, promoting simplicity and clarity. Python's ubiquity is

demonstrated by its consistent presence among the top programming languages in various surveys.

- **SciPy:** The Mathematical Foundation: SciPy, an ecosystem of Python libraries for science, mathematics, and engineering, emerges as an essential component for machine learning. It encompasses core modules like NumPy for efficient data array manipulation, Matplotlib for visualisations, and Pandas for data organisation and analysis.
- **scikit-learn:** The Machine Learning Library: The crux of our exploration is the scikit-learn library, a cornerstone for developing and honing machine learning skills within Python. Building upon the SciPy foundation, Scikit-Learn furnishes a comprehensive array of machine learning algorithms. Its capabilities span classification, regression, clustering, and auxiliary tasks such as model evaluation, parameter tuning, and data preprocessing.
- **Setting Up Your Python Ecosystem:** Establishing a robust environment for machine learning is vital. We guide you through the process, from installing Python itself to integrating SciPy and Scikit-Learn. A recommended installation method is provided, ensuring you're well-prepared to embark on your machine learning journey.

The session demonstrates that Python's allure extends well beyond its readability and versatility. Its maturation as a language within the realm of machine learning is evidenced by its strong library support and commercial viability. Our voyage is illuminated by insights into Python's syntax and key

libraries, such as NumPy, Matplotlib, and Pandas. Together, these components form the bedrock upon which we can build a deep understanding of machine learning tasks within the Python ecosystem. Let's embrace this enriching experience and delve into the intricacies of this powerful toolkit.

Learning Outcomes for Study Session Two

When you have studied this session, you should be able to:

- 2.1 Discuss the growing significance of the Python ecosystem in the field of machine learning and its potential to become the dominant platform.
- 2.2 Explain the primary reasons for adopting Python in machine learning, including its general-purpose nature, which makes it suitable for both R&D and production.
- 2.3 Identify the components of the Python ecosystem for machine learning, including Python itself, SciPy, scikit-learn, and related libraries.
- 2.4 Differentiate between Python's role in the research, development, and production phases of machine learning.
- 2.5 Detail the functionalities provided by SciPy, encompassing NumPy, Matplotlib, and Pandas, and their importance in the context of machine learning.
- 2.6 Summarise the capabilities of Scikit-Learn as a comprehensive library for various machine learning algorithms and related tasks.
- 2.7 Demonstrate the importance of setting up a proper Python ecosystem for machine learning, including guidelines for version compatibility.

- 2.8 Grasp the core philosophies of the Python programming language, as reflected in the Zen of Python.
- 2.9 Utilise Python's features to navigate its language syntax, work with data structures, implement flow control structures, and define functions.
- 2.10 Explain the essential data structures in Python: tuples, lists, and dictionaries, and how they can be used effectively.
- 2.11 Create and manipulate NumPy arrays for efficient numerical operations.
- 2.12 Generate different types of plots and charts using the Matplotlib package.
- 2.13 Harness the capabilities of Pandas for data manipulation. focusing on Series and DataFrame data structures.

2.1 Python Ecosystem for Machine Learning

The Python ecosystem is growing and may become the dominant platform for machine learning. The primary rationale for adopting Python for machine learning is because it is a general-purpose programming language that you can use both for R&D and in production. In this session, you will discover the Python ecosystem for machine learning. We will learn the following:

- Python and its rising use for machine learning.
- SciPy and the functionality it provides with NumPy, Matplotlib and Pandas.
- scikit-learn that provides all of the machine learning algorithms.
- How to set up your Python ecosystem for machine learning and what versions to use

Let's get started.

Python is a general-purpose interpreted programming language. It is easy to learn and use primarily because the language focuses on readability. The philosophy of Python is captured in the Zen of Python, which includes phrases like:

Beautiful is better than ugly
Explicit is better than implicit
Simple is better than complex
Complex is better than complicated
Flat is better than nested

Sparse is better than dense

Readability counts

It is a popular language in general, consistently appearing in the top 10 programming languages in surveys on Stack Overflow. It's a dynamic language and very suited to interactive development and quick prototyping with the power to support the development of large applications. It is also widely used for machine learning and data science because of the excellent library support and because it is a general-purpose programming language (unlike R or Matlab). For example, see the results of the Kaggle platform survey results in 2011 and the KDD Nuggets 2015 tool survey results.

SciPy is an ecosystem of Python libraries for mathematics, science, and engineering. It is an add-on to Python that you will need for machine learning. The SciPy ecosystem is comprised of the following core modules relevant to machine learning:

NumPy: A foundation for SciPy that allows you to efficiently work with data in arrays.

Matplotlib: Allows you to create 2D charts and plots from data.

Pandas: Tools and data structures to organise and analyse your data.

To be effective at machine learning in Python you must install and become familiar with SciPy. Specifically:

- You will prepare your data as NumPy arrays for modelling in machine

learning algorithms.

- You will use Matplotlib (and wrappers of Matplotlib in other frameworks) to create plots and charts of your data.
- You will use Pandas to load explore and better understand your data.

Scikit-learn

The scikit-learn library is how you can develop and practise machine learning in Python. It is built upon and requires the SciPy ecosystem. The name 'scikit' suggests that it is a SciPy plug-in or toolkit. The focus of the library is machine learning algorithms for classification, regression, clustering and more. It also provides tools for related tasks such as evaluating models, tuning parameters and preprocessing data.

Like Python and SciPy, scikit-learn is open source and is usable commercially under the BSD licence. This means that you can learn about machine learning, develop models and put them into operation all with the same ecosystem and code. A powerful reason to use scikit-learn.

2.2 Python Installation

There are multiple ways to install the Python ecosystem for machine learning. In this section we cover how to install the Python ecosystem for machine learning. The first step is to install Python. We recommend Python 3.6–3.11. The instructions for installing Python will be specific to your platform. You can directly download python from <https://www.python.org/downloads/>

or just download and install Anaconda Distribution and you'll be good to start your Machine Learning Journey. From anaconda Distribution you use Jupyter notebook to write the python codes.

Download link: <https://www.anaconda.com/products/distribution>

Open a command line and type:

How To Install SciPy

There are many ways to install SciPy. For example, two popular ways are to use package management on your platform (e.g., yum on RedHat or macports on OS X) or use a Python package management tool like pip. The SciPy documentation is excellent and covers how-to instructions for many different platforms on the page Installing the SciPy Stack. When installing SciPy, ensure that you install the following packages as a minimum:

- scipy
- numpy
- matplotlib
- pandas

Once installed, you can confirm that the installation was successful. Open the Python interactive environment by typing 'python' at the command line, then type in and run the following Python code to print the versions of the installed libraries.

2.3 How To Install scikit-learn

I would suggest that you use the same method to install scikit-learn as you used to install SciPy.

There are instructions for installing scikit-learn, but they are limited to using the Python pip and conda package managers. Like SciPy, you can confirm that scikit-learn was installed successfully. Start your Python interactive environment and type and run the following code.

2.4 Course in Python and SciPy

You do not need to be a Python developer to get started using the Python ecosystem for machine learning. As a developer who already knows how to program in one or more programming languages, you are able to pick up a new language like Python very quickly. You just need to know a few properties of the language to transfer what you already know to the new language.

After completing this lesson you will know:

- How to navigate Python language syntax.
- Enough NumPy, Matplotlib and Pandas to read and write machine learning Python scripts.
- A foundation from which to build a deeper understanding of machine learning tasks in Python.

If you already know a little Python, this section will be a friendly reminder for you. Let's get started.

When getting started in Python you need to know a few key details about the language syntax to be able to read and understand Python code. This includes:

- Assignment.
- Flow Control.
- Data Structures.
- Functions.

We will cover each of these topics in turn with small standalone examples that you can type and run. Remember, whitespace has meaning in Python.

2.5 Assignment

As a programmer, assignments and types should not be surprising to you.

Strings

Python

```
# strings
data = 'hello world'
print(data[0])
print(len(data))
print(data)
```

Listing 3.1: Example of working with strings.

Notice how you can access characters in the string using array syntax. Running the example prints:

```
h
11
hello world
```

Numbers

```
Python
# Numbers

Value - 123.1
print(value)
value = 10
print(value)
```

Example of working with numbers. Running the example prints:

123.1

10

Boolean

```
# Boolean
```

```
a = True
```

```
b = false
```

```
print(a, b)
```

Example of working with booleans. Running the example prints:

True, False

Output of example working with booleans.

Multiple Assignment

```
# Multiple Assignment  
  
a, b, c = 1, 2, 3  
  
print(a, b, c)
```

Listing 3.7: Example of working with multiple assignments.

This can also be very handy for unpacking data in simple data structures.

Running the example prints:

```
1, 2, 3
```

Output of example working with multiple assignment

No Value

```
# No value  
  
a = None  
  
print(a)
```

Example of working with no value. Running the example prints:

```
None
```

Output of example working with no value.

2.6 Flow Control

There are three main types of flow control that you need to learn: If-Then-Else conditions,

For-Loops and While-Loops. If-Then-Else Conditional

Value - 99

```
if value == 99:  
    print('That is fast ')  
elif value > 200:  
    print('That is too fast ')  
else:  
    print('That is safe')
```

Example of working with an If-Then-Else conditional.

Notice the colon at the end of the condition and the meaningful tab intend for the code block under the condition. Running the example prints:

This is fast

Output of example working with an If-Then-Else conditional.

For-Loop

```
1  #for loop  
2  for i in range(10):  
3      print(i)
```

Example of working with a For-Loop. Running the example prints:

0
1
2
3

4
5
6
7
8
9

Output of example working with a For-Loop.

While - loop

```
1 # While-Loop
2 i = 0
3 while i < 10:
4     print(i)
5     i += 1
```

Example of working with a While-Loop.

0
1
2
3
4
5
6
7
8
9

Output of an example working with a While-Loop.

2.7 Data Structures

There are three data structures in Python that you will find the most used and useful. They are tuples, lists and dictionaries.

Tuple

Tuples are read-only collections of items.

```
1 a = (1, 2, 3)
2 print(a)
```

Example of working with a Tuple. Running the example prints:

```
(1, 2, 3)
```

Output of example working with a Tuple.

List

Lists use the square bracket notation and can be indexed using array notation.

Python

```
mylist = [1, 2, 3]
print("Zeroth Value: %d" % mylist[0])
mylist.append(4)
print("List Length: %d" % len(mylist))
for value in mylist:
    print(value)
```

Example of Working with a List.

Notice that we are using some simple printf-like functionality to combine strings and variables when printing.

Running the example prints

Zeroth Value: 1	
List Length: 4	
1	
2	
3	
4	

Output of example working with a List.

Dictionary

Dictionaries are mappings of names to values, like key-value pairs. Note the use of the curly bracket and colon notations when defining the dictionary.

Python

```
mydict = {'a': 1, 'b': 2, 'c': 3}
print("A value: %d" % mydict['a'])
mydict['a'] = 11
print("A value: %d" % mydict['a'])
print("Keys: %s" % mydict.keys()
)
print("Values: %s" % mydict.values
())
for key in mydict.keys():
```

```
print(mydict  
[key])
```

Example of working with a Dictionary. Running the example prints:

A value: 1
A value: 11
Keys: dict_keys(['a', 'b', 'c'])
Values: dict_values([11, 2, 3])
11
2
3

Output of an example working with a Dictionary.

2.8 Functions

The biggest gotcha with Python is the whitespace. Ensure that you have an empty new line after indented code. The example below defines a new function to calculate the sum of two values and calls the function with two arguments.

```
# Sum function
def mysum(x, y):
    return x + y

# Test sum function
result = mysum(1, 3)
print(result)
```

Example of working with a custom function. Running the example prints:

4

Output of example working with a custom

2.9 NumPy Crash Course

NumPy provides the foundation data structures and operations for SciPy. These are arrays (ndarrays) that are efficient to deny and manipulate.

Create Array

```
# define an array import
numpy mylist = [1,
                2, 3]
myarray = numpy.array(mylist)
print(myarray)
print(myarray.shape)
```

Example of creating a NumPy array.

Notice how we easily converted a Python list to a NumPy array. Running the example prints:

```
[1 2 3]
(3,)
```

Output of example creating a NumPy

Access Data

Array notation and ranges can be used to efficiently access data in a NumPy

```
# access values import
    numpy
mylist = [[1, 2, 3], [3, 4, 5]]
myarray = numpy.array(mylist)
    print(myarray)
    print(myarray.shape)
print("First row: %s" % myarray[0]) print("Last
    row: %s" % myarray[-1])
print("Specific row and col: %s" % myarray[0, 2])
print("Whole col: %s" % myarray[:, 2])
```

Example of working with a NumPy array.

Running the example prints:

```
[[1 2 3]
 [3 4 5]]
(2, 3)
First row: [1 2 3]
Last row: [3 4 5] Specific row
           and col: 3 Whole col: [3
           5]
```

Output of example working with a NumPy

Arithmetic

NumPy arrays can be used directly in arithmetic.

```
# arithmetic import
    numpy
myarray1 = numpy.array([2, 2, 2])
myarray2 = numpy.array([3, 3, 3]) print("Addition: %s"
    % (myarray1 + myarray2))
print("Multiplication: %s" % (myarray1 * myarray2))
```

Example of doing arithmetic with NumPy Running the example prints:

```
Addition: [5 5 5]
Multiplication: [6 6 6]
```

Output of an example of doing arithmetic with NumPy arrays.

There is a lot more to NumPy arrays but these examples give you a view of the efficiencies they provide when working with lots of numerical data. See Chapter 24 for resources to learn more about the NumPy API.

2.10 Matplotlib Package

Matplotlib can be used for creating plots and charts. The library is generally used as follows:

Call a plotting function with some data (e.g., `.plot()`).

Call many functions to set up the properties of the plot (e.g. labels and colours).

Make the plot visible (e.g., `.show()`).

Line Plot

The example below creates a simple line plot from one-dimensional data.

```
# basic line plot
import matplotlib.pyplot as plt
import numpy
myarray = numpy.array([1, 2, 3])
plt.plot(myarray)
plt.xlabel('some x axis')
plt.ylabel('some y axis')
plt.show()
```

Example of creating a line plot with Matplotlib. Running the example produces:

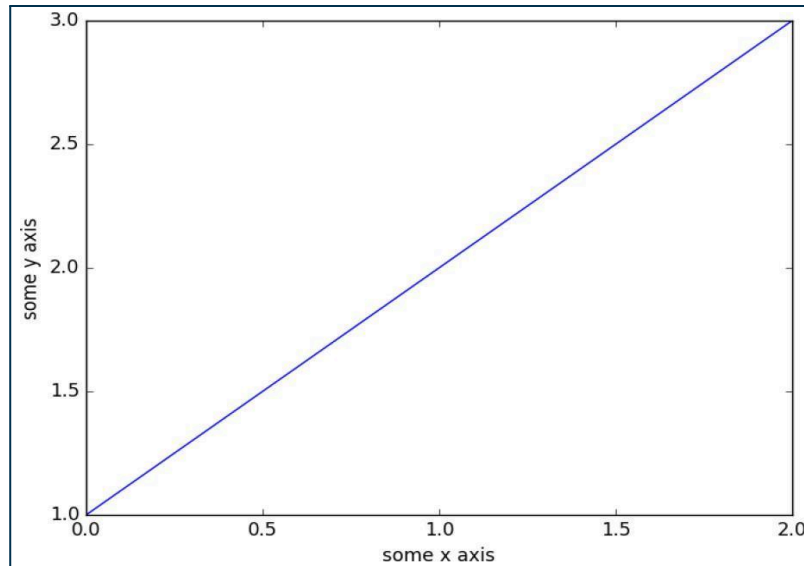


Figure 2.1: Line Plot Matplotlib

Scatter Plot

Below is a simple example of creating a scatter plot from two-dimensional data.

```
# basic scatter plot
import matplotlib.pyplot as plt
import numpy
x = numpy.array([1, 2, 3])
y = numpy.array([2, 4, 6])
plt.scatter(x,y)
plt.xlabel('some x axis')
plt.ylabel('some y axis')
```

```
plt.show()
```

Example of creating a line plot with Matplotlib.

Running the example produces:

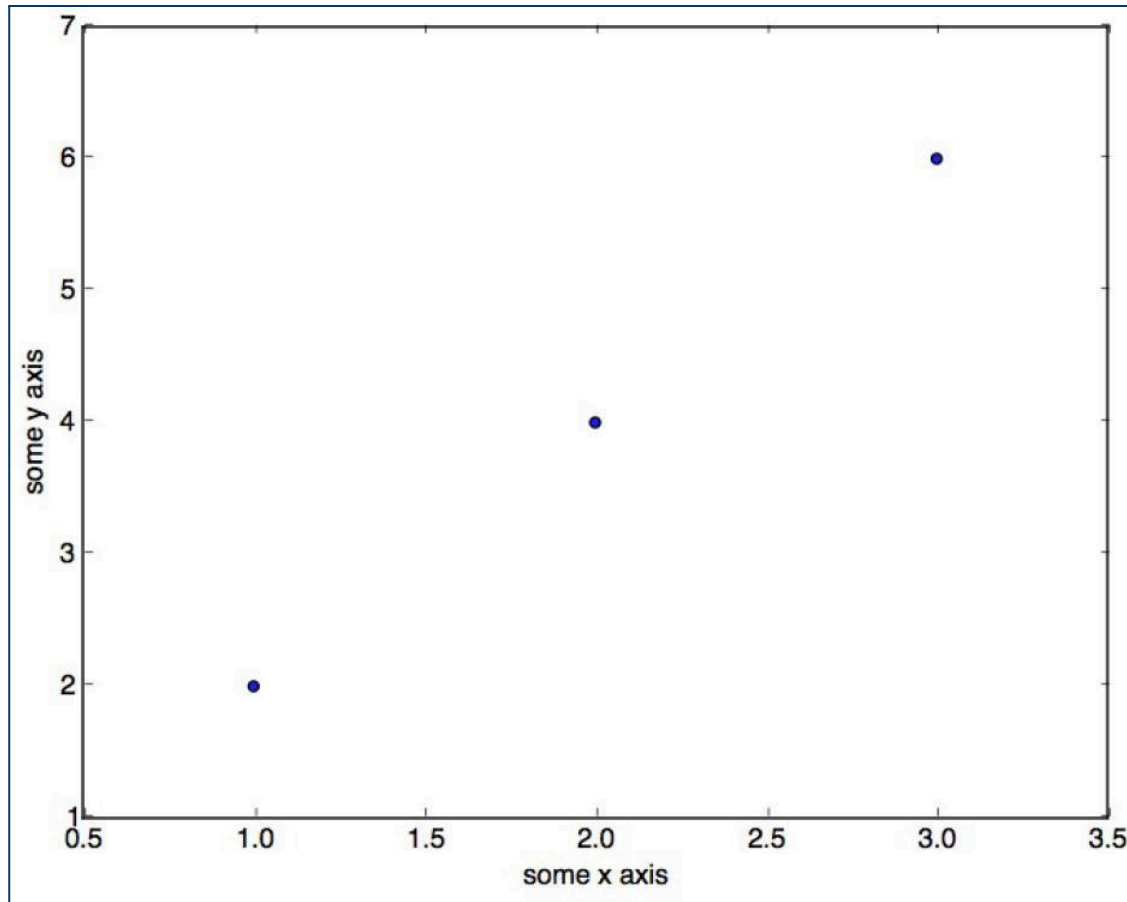


Figure 2.2: Scatter Plot with Matplotlib

There are many more plot types and many more properties that can be set on a plot to configure it. See resources

- Matplotlib gallery of plot types and sample code:
<http://matplotlib.org/gallery.html>
- Matplotlib Beginners Guide. <http://matplotlib.org/users/beginner.html>
- Matplotlib API Reference: <http://matplotlib.org/api/index.html> to learn more about the Matplotlib API.

2.11 Pandas Crash Course

Pandas provides data structures and functionality to quickly manipulate and analyse data. The key to understanding Pandas for machine learning is understanding the Series and DataFrame data structures.

Series

A series is a one-dimensional array of data where the rows are labelled using a time axis.

```
# series import
    numpy import
    pandas
myarray = numpy.array([1, 2, 3])
    rownames = ['a', 'b', 'c']
myseries = pandas.Series(myarray, index=rownames)
print(myseries)
```

Example of creating a Pandas Series.

Running the example prints:

```
a 1
b 2
c 3
```

Output of example of creating a Pandas Series.

You can access the data in a series like a NumPy array and like a dictionary, for example:

```
print(myseries[0]) print(myseries['a'])
```

Example of accessing data in a Pandas Series. Running the example prints:

```
1  
1
```

Output of example of accessing data in a Pandas Series.

DataFrame

A data frame is a multi-dimensional array where the rows and the columns can be labelled.

```
# dataframe import  
    numpy import  
    pandas  
myarray = numpy.array([[1, 2, 3], [4, 5, 6]]) rownames =  
    ['a', 'b']  
colnames = ['one', 'two', 'three']  
mydataframe = pandas.DataFrame(myarray, index=rownames,  
    columns=colnames)  
print(mydataframe)
```

Example of creating a Pandas DataFrame. Running the example prints:

```
One, two, three.
```

a	2	
1	3	6
b		
4		

Output of an example of creating a Pandas DataFrame.

Data can be indexed using column names.

```
print("method 1:")  
print("one column:\n%s" % mydataframe['one'])  
    print("method 2:")  
print("one column:\n%s" % mydataframe.one)
```

Example of accessing data in a Pandas DataFrame. Running the example prints:

```
Method 1: One  
Column:  
a    1  
B    4  
Method 2: One  
Column:  
a    1  
b    4
```

Output of an example of accessing data in a Pandas DataFrame.

Pandas is a very powerful tool for slicing and dicing your data. See SciPy Lecture Notes <http://www.scipy-lectures.org/>

NumPy

User Guide: <http://docs.scipy.org/doc/numpy/user/> for resources to learn more about the Pandas API.

You have covered a lot of ground in this lesson. You discovered basic syntax and usage of Python and three key Python libraries used for machine learning:

- NumPy.
- Matplotlib.
- Pandas.

In-Text Questions and Answers (ITQs and ITAs)

ITQ 2.1: What are the core reasons for Python's growing significance in machine learning?

ITA: Python's growing significance in machine learning can be attributed to its readability, versatility, strong library support, and ability to function well in both research and production environments.

ITQ 2.2: How does Python's general-purpose nature benefit its application in machine learning?

ITA: Python's general-purpose nature makes it an ideal choice for machine learning, as it can be used across various stages—from research and development to production—while maintaining simplicity and readability in code.

Conclusion

In conclusion, the Python ecosystem for machine learning emerges as a robust and dynamic platform that holds the promise of becoming the dominant force in the field. With its wide-ranging capabilities and growing popularity, Python presents itself as an optimal choice for machine learning endeavours, offering a balance between research and practical implementation. The comprehensive exploration of Python's core components, accompanied by pivotal libraries like SciPy, NumPy, Matplotlib, and Pandas, establishes a strong foundation for aspiring machine learning practitioners. Additionally, the accessibility of scikit-learn as a dedicated toolkit for machine learning further solidifies Python's position in this domain. By providing not only the necessary tools but also a deep understanding of the language's syntax and key libraries, this chapter paves the way for proficient engagement in machine learning tasks. As the ecosystem continues to evolve, Python's adaptability and extensive library support make it a compelling choice for individuals seeking to master machine learning and harness its potential for various applications.

Summary

In this study session, you have learnt to:

1. Understand the Python ecosystem for machine learning and its potential to become a dominant platform, as well as its suitability for both research and production in machine learning.

2. Recognise the significance of Python as a general-purpose programming language, its readability, and its use in both R&D and production environments for machine learning.
3. Explore the components of the Python ecosystem for machine learning, including Python's role and its rising prominence in machine learning. SciPy and its essential modules like NumPy, Matplotlib, and Pandas, which provide crucial functionality for data manipulation, visualisation, and analysis. In addition, you have been able to use Scikit-Learn, a comprehensive library for machine learning algorithms, which are essential tools for classification, regression, clustering, and parameter tuning.
4. Gain insights into setting up your Python ecosystem for machine learning, including recommended versions and installation methods.
5. Familiarise yourself with fundamental Python language features, including assignment, flow control (if-then-else conditions, for-loops, and while-loops), data structures (tuples, lists, and dictionaries), and functions.
6. Explore NumPy, a foundational library for efficient array-based data manipulation in Python, including creating arrays, accessing data, and performing arithmetic operations.
7. Discover Matplotlib, a library for creating various types of plots and charts, including line plots and scatter plots, and learn how to customise their properties.
8. Dive into Pandas, a powerful library for data manipulation and analysis, and learn about Series and data structures, indexing data, and accessing specific columns.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 1: Describe the primary reasons for adopting Python as the dominant platform for machine learning.

Answer: The primary reasons for adopting Python as the dominant platform for machine learning are its versatility and extensive ecosystem. Python's suitability for both research and production makes it a preferred choice. Its easy-to-learn syntax and readability align with the Zen of Python's principles, fostering rapid development. Python consistently ranks among the top programming languages on platforms like Stack Overflow.

SAQ 2: Explain the role and significance of the SciPy ecosystem in machine learning.

Answer: The SciPy ecosystem plays a crucial role in machine learning by providing a comprehensive set of libraries for mathematics, science, and engineering. It enhances Python's capabilities and efficiency, making it an indispensable component for machine learning practitioners.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **Which Python feature makes it an ideal choice for both research and production in machine learning?**

- A) Its ability to perform complex mathematical operations
- B) Its readability and versatility for quick prototyping and production deployment
- C) Its restriction to research-only applications

D) Its focus on numerical operations

Answer: B) Its readability and versatility for quick prototyping and production deployment

Explanation: Python's readability and versatility allow it to be used effectively both in research (for rapid prototyping) and production environments (for implementing machine learning models).

2. **Which Python library is essential for creating plots and visualisations in machine learning?**

A) NumPy

B) Pandas

C) Matplotlib

D) SciPy

Answer: C) Matplotlib

Explanation: Matplotlib is the key library used for creating various types of plots and visualisations in machine learning, such as line and scatter plots.

3. **What role does SciPy play in Python's ecosystem for machine learning?**

A) It provides machine learning algorithms

B) It helps in data visualisation.

C) It supports numerical and scientific computations through libraries like NumPy

D) It handles data preprocessing

Answer: C) It supports numerical and scientific computations through libraries like NumPy

Explanation: SciPy provides essential support for numerical operations and scientific computations, primarily through NumPy, which is foundational for machine learning.

4. **Which of the following is a key benefit of using Python for machine learning?**

- A) It has limited library support
- B) It's highly focused on a single domain (research only)
- C) It's an easy-to-learn language with strong community support
- D) It lacks debugging tools

Answer: C) It's an easy-to-learn language with strong community support

Explanation: Python is well-known for its ease of learning, readability, and strong community support, making it an ideal language for machine learning development.

5. **Which Python data structure is often used to store data for machine learning tasks, especially for array manipulation?**

- A) List
- B) Dictionary
- C) Tuple
- D) NumPy array

Answer: D) NumPy array

Explanation: NumPy arrays are optimised for numerical operations and are used extensively in machine learning for data manipulation and processing.

Tutor-Marked Assessments (TMAs)

1. Why is Python considered suitable for both research and production in the field of machine learning?
2. Explain the roles of NumPy, Matplotlib, and Pandas in the context of machine learning.
3. Describe the role of scikit-learn in machine learning. Provide examples of machine learning algorithms it supports and how it integrates with other Python libraries to streamline the machine learning process.
4. Explain the setup process for a Python ecosystem for machine learning, including the installation of Python, SciPy, and scikit-learn. Discuss version compatibility and why these steps are essential for effective machine learning development.

STUDY SESSION THREE

LOADING MACHINE LEARNING DATA

Introduction

In this session, we will delve into the crucial task of loading machine learning data, a prerequisite for any successful machine learning project. The predominant data format in machine learning is CSV files, and Python provides various techniques to load this data efficiently. In this lesson, we will explore three distinct methods for loading CSV data in Python, each with its own unique advantages. These methods include loading CSV files using the Python Standard Library, leveraging NumPy, and harnessing the power of Pandas. Before we dive into these techniques, we will also discuss essential considerations when loading CSV data, such as handling file headers, comments, delimiters, and quotes. Let's embark on this journey to master the art of loading and understanding machine learning data.

Learning Outcomes for Study Session Three

When you have studied this session, you should be able to:

- 3.1. Explain the importance of loading data for machine learning projects and recognise CSV files as the most common format for machine learning data.
- 3.2. Identify key considerations when loading CSV data, including file headers, comments, delimiters, and quotes. Specify whether your data has a file header and understand the role of comments, delimiters, and quotes.
- 3.3. Recognise the Pima Indians dataset and its use as a classification problem example in data loading demonstrations. Understand its attributes and the nature of its output variable.

- 3.4. Load CSV files using the Python Standard Library by utilising the CSV module and the `reader()` function. Convert CSV data to a NumPy array for machine learning applications.
- 3.5. Load CSV files using NumPy and the `numpy.loadtxt()` function. Gain familiarity with loading data from both local files and URLs.
- 3.6. Load CSV files using Pandas and the `pandas.read_csv()` function. Appreciate the flexibility and benefits of using Pandas for data loading, and learn to specify attribute names when creating a DataFrame.
- 3.7. Explain the significance of understanding your data through descriptive statistics. Learn how to use various methods to better understand your machine learning dataset.
- 3.8. Review your raw data by using the `head()` function to display the first few rows, allowing insights into the structure and format of the data.
- 3.9. Gain an understanding of the dimensions of your dataset by using the `shape` property, which provides the number of rows and columns in the dataset.
- 3.10. Examine the data types of each attribute in your dataset using the `dtypes` property, which reveals the types of data stored in each column.
- 3.11. Use the `describe()` function to obtain descriptive statistics about your dataset, including count, mean, standard deviation, and percentiles.
- 3.12. Understand the class distribution in classification problems by using the `groupby()` function to count the occurrences of each class in your dataset.
- 3.13. Calculate correlations between attributes in your dataset using the `corr()` function to determine how variables change together and identify potential

relationships.

- 3.14. Measure the skew of univariate distributions in your dataset using the `skew()` function to identify potential Gaussian distribution shifts.
- 3.15. Utilise tips for effective data review, including interpreting summary statistics, asking questions about the data, writing down observations, and noting potential relationships for future exploration.

3.1 How To Load Machine Learning Data

You must be able to load your data before you can start your machine learning project. The most common format for machine learning data is CSV files. There are a number of ways to load a CSV file in Python. In this lesson, you will learn three ways that you can use to load your CSV data in Python:

- Load CSV Files with the Python Standard Library.
- Load CSV Files with NumPy.
- Load CSV Files with Pandas. Let's get started.

Considerations When Loading CSV Data:

There are a number of considerations when loading your machine learning data from CSV files. For reference, you can learn a lot about the expectations for CSV files by reviewing the CSV request for comment titled Common Format and MIME Type for Comma-Separated Values (CSV) File.

1. **File Header:** Does your data have a file header? If so, this can help in automatically assigning names to each column of data. If not, you may need to name your attributes manually. Either way, you should explicitly specify whether or not your CSV file had a file header when loading your data.
2. **Comments:** Does your data have comments? Comments in a CSV file are indicated by a hash (#) at the start of a line. If you have comments in your file, depending on the method used to load your data, you may need to indicate whether or not to expect comments and the character to expect to signify a comment line.

3. **Delimiter:** The standard delimiter that separates values in fields is the comma (,) character. Your file could use a different delimiter like a tab or whitespace, in which case you must specify it explicitly.
4. **Quotes:** Sometimes field values can have spaces. In these CSV files the values are often quoted. The default quote character is the double quotation mark character. Other characters can be used, and you must specify the quote character used in your file.

3.2 Pima Indians Dataset

The Pima Indians dataset is used to demonstrate data loading in this lesson. It will also be used in many of the lessons to come. This dataset describes the medical records for Pima Indians and whether or not each patient will have an onset of diabetes within five years. As such, it is a classification problem. It is a good dataset for demonstration because all of the input attributes are numeric and the output variable to be predicted is binary (0 or 1). The data is freely available from the UCI Machine Learning Repository.

1. Load CSV Files with the Python Standard Library

The Python API provides the module CSV and the function `reader()` that can be used to load CSV files. Once loaded, you can convert the CSV data to a NumPy array and use it for machine learning. For example, you can download the Pima Indians dataset into your local directory with the filename `pima-indians-diabetes.data.csv`. All fields in this dataset are numeric and there is no header line.

```
# Load CSV Using Python Standard Library import
    csv
import numpy

filename = 'pima-indians-diabetes.data.csv'
    raw_data = open(filename, 'rt')
reader = csv.reader(raw_data, delimiter=',', quoting=csv.QUOTE_NONE) x =
    list(reader)
data = numpy.array(x).astype('float')

print(data.shape)
```

Example of loading a CSV file using the Python standard library.

The example loads an object that can iterate over each row of the data and can easily be converted into a NumPy array. Running the example prints the shape of the array.

```
(768, 9)
```

Output of example loading a CSV file using the Python standard library.

2. Load CSV Files with NumPy

You can load your CSV data using NumPy and the `numpy.loadtxt()` function. This function assumes no header row and all data has the same format. The example below assumes that the file `pima-indians-diabetes.data.csv` is in your current working directory.

```
# Load CSV using NumPy from
    numpy import loadtxt
filename = 'pima-indians-diabetes.data.csv'
    raw_data = open(filename, 'rt')
data = loadtxt(raw_data, delimiter=",")
    print(data.shape)
```

Example of loading a CSV file using NumPy.

Running the example will load the file as a `numpy.ndarray` and print the shape of the data:

```
(768, 9)
```

Output of example loading a CSV file using NumPy.

This example can be modified to load the same dataset directly from a URL as follows:

```
# Load CSV from URL using NumPy from
    numpy import loadtxt
from urllib.request import urlopen url =
    'https://goo.gl/vhm1eU' raw_data =
    urlopen(url)
dataset = loadtxt(raw_data, delimiter=",")
print(dataset.shape)
```

Example of loading a CSV URL using NumPy.

Again, running the example produces the same resulting shape of the data.

```
(768, 9)
```

Output of an example loading a CSV URL using NumPy.

3. Load CSV Files with Pandas

You can load your CSV data using Pandas and the `pandas` module. Read the `csv()` function. This function is very flexible and is perhaps my recommended approach for loading your machine learning data. The function returns a `pandas.DataFrame` that you can immediately start summarising and plotting. The example below assumes that the `pima-indians-diabetes.data.csv` file is in the current working directory.

```
from urllib.request import urlopen # Load CSV using Pandas from
pandas import read_csv
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = read_csv(filename, names=names)
print(data.shape)
```

Example of loading a CSV file using Pandas.

Note that in this example we explicitly specify the names of each attribute of the `DataFrame`. Running the example displays the shape of the data:

```
(768, 9)
```

Output of an example loading a CSV file using Pandas.

We can also modify this example to load CSV data directly from a URL.

```
# Load CSV using Pandas from URL from
pandas import read_csv
url = 'https://goo.gl/vhm1eU'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
      = read_csv(url, names=names)
print(data.shape)
```

Example of loading a CSV URL using Pandas.

Again, running the example downloads the CSV file, parses it and displays the shape of the

loaded DataFrame

```
(768, 9)
```

Output of an example loading a CSV URL using Pandas.

In this chapter you discovered how to load your machine learning data in Python.

You learnt three specific techniques that you can use:

- Load CSV Files with the Python Standard Library.
- Load CSV Files with NumPy.
- Load CSV Files with Pandas.

Generally, I recommend that you load your data with Pandas in practice, and all subsequent examples in this course will use this method.

3.3 Understand Your Data With Descriptive Statistics

You must understand your data in order to get the best results. Here, you will discover 7 recipes that you can use in Python to better understand your machine learning data. After reading this section, you will know how to:

- Take a peek at your raw data.
- Review the dimensions of your dataset.
- Review the types of attributes in your data.
- Summarise the distribution of instances across classes in your dataset.
- Summarise your data using descriptive statistics.
- Understand the relationships in your data using correlations.
- Review the skew of the distributions of each attribute.

Each recipe is demonstrated by loading the Pima Indians Diabetes classification dataset from the UCI Machine Learning repository. Open your Python interactive environment and try each recipe out in turn. Let's get started.

3.4 Look at Your Data

There is no substitute for looking at the raw data. Looking at the raw data can reveal insights that you cannot get any other way. It can also plant seeds that may later grow into ideas on how to better preprocess and handle the data for machine learning tasks. You can review the first 20 rows of your data using the `head()` function on the Pandas DataFrame.

```
# View first 20 rows

from pandas import read_csv

filename = "pima-indians-diabetes.data.csv"

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
        = read_csv(filename, names=names)

look = data.head(20)

print(look)
```

Example of reviewing the first few rows of data.

You can see that the first column lists the row number, which is handy for referencing a specific observation.

	preg	plas	pres	skin	test	mass	pedi	age	class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
5	5	116	74	0	0	25.6	0.201	30	0
6	3	78	50	32	88	31.0	0.248	26	1
7	10	115	0	0	0	35.3	0.134	29	0
8	2	197	70	45	543	30.5	0.158	53	1
9	8	125	96	0	0	0.0	0.232	54	1
10	4	110	92	0	0	37.6	0.191	30	0
11	10	168	74	0	0	38.0	0.537	34	1
12	10	139	80	0	0	27.1	1.441	57	0
13	1	189	60	23	846	30.1	0.398	59	1
14	5	166	72	19	175	25.8	0.587	51	1
15	7	100	0	0	0	30.0	0.484	32	1
16	0	118	84	47	230	45.8	0.551	31	1
17	7	107	74	0	0	29.6	0.254	31	1
18	1	103	30	38	83	43.3	0.183	33	0
19	1	115	70	30	96	34.6	0.529	32	1

Output of reviewing the first few rows of data.

3.5 Dimensions of Your Data

You must have a very good handle on how much data you have, both in terms of rows and columns.

- Too many rows and algorithms may take too long to train. Too few, and perhaps you do not have enough data to train the algorithms.
- Too many features and some algorithms can be distracted or suffer poor performance due to the curse of dimensionality.

You can review the shape and size of your dataset by printing the shape property on the Pandas DataFrame.

```
# Dimensions of your data from
pandas import read_csv
filename = "pima-indians-diabetes.data.csv"
```

```
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data =
    read_csv(filename, names=names)
shape = data.shape
print(shape)
```

Example of reviewing the shape of the data.

The results are listed in rows and columns. You can see that the dataset has 768 rows and 9 columns.

```
(768, 9)
```

Output of reviewing the shape of the data.

3.6 Data Type For Each Attribute

The type of each attribute is important. Strings may need to be converted to floating-point values or integers to represent categorical or ordinal values. You can get an idea of the types of attributes by peeking at the raw data, as above. You can also list the data types used by the DataFrame to characterise each attribute using the dtypes property.

```
# Data Types for Each Attribute from
pandas import read_csv
filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
        = read_csv(filename, names=names)
types = data.dtypes
print(types)
```

Example of reviewing the data types of the data.

You can see that most of the attributes are integers and that mass and pedi are floating-point types.

```
preg      int64
plas      int64
pres      int64
skin      int64
test      int64
mass      float64
pedi      float64
age       int64
class     int64
dtype: object
```

Output of reviewing the data types of the data.

3.7 Descriptive Statistics

Descriptive statistics can give you great insight into the shape of each attribute. Often you can create more summaries than you have time to review. The `describe()` function on the Pandas DataFrame lists 8 statistical properties of each attribute. They are:

- Count.
- Mean.
- Standard Deviation.
- Minimum Value.
- 25th Percentile.
- 50th Percentile (Median).
- 75th Percentile.
- Maximum Value.

```
# Statistical Summary
```

```
from pandas import read_csv from
    pandas import set_option
filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data =
    read_csv(filename, names=names) set_option('display.width', 100)
set_option('precision', 3)
description = data.describe()
print(description)
```

Example of reviewing a statistical summary of the data.

You can see that you do get a lot of data. You will note some calls to `pandas.set_option()` in the recipe to change the precision of the numbers and the preferred width of the output. This is to make it more readable for this example. When describing your data this way, it is worth taking some time and reviewing observations from the results. This might include the presence of NA values for missing data or surprising distributions for attributes.

	preg	plas	pres	skin	test	mass	pedi	age	class
count	768.000	768.000	768.000	768.000	768.000	768.000	768.000	768.000	768.000
mean	3.845	120.895	69.105	20.536	79.799	31.993	0.472	33.241	0.349
std	3.370	31.973	19.356	15.952	115.244	7.884	0.331	11.760	0.477
min	0.000	0.000	0.000	0.000	0.000	0.000	0.078	21.000	0.000
25%	1.000	99.000	62.000	0.000	0.000	27.300	0.244	24.000	0.000
50%	3.000	117.000	72.000	23.000	30.500	32.000	0.372	29.000	0.000
75%	6.000	140.250	80.000	32.000	127.250	36.600	0.626	41.000	1.000
max	17.000	199.000	122.000	99.000	846.000	67.100	2.420	81.000	1.000

Output of reviewing a statistical summary of the data.

3.8 Class Distribution (Classification Only)

On classification problems you need to know how balanced the class values are. Highly imbalanced problems (a lot more observations for one class than another) are common and may need special handling in the data preparation stage of your project. You can quickly get an idea of the distribution of the class attribute in Pandas

```
# Class Distribution
from pandas import read_csv
filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
        = read_csv(filename, names=names)
class_counts = data.groupby('class').size()
        print(class_counts)
```

Example of reviewing a class breakdown of the data.

You can see that there are nearly double the number of observations with class 0 (no onset of diabetes) than there are with class 1 (onset of diabetes).

class	
0	500
1	268

Output of reviewing a class breakdown of the data.

3.9 Correlations Between Attributes

Correlation refers to the relationship between two variables and how they may or may not change together. The most common method for calculating correlation is Pearson's Correlation Coefficient, which assumes a normal distribution of the attributes involved. A correlation of -1 or 1 shows a full negative or positive correlation, respectively. Whereas a value of 0 shows no correlation at all. Some machine learning algorithms like linear and logistic regression can suffer.

poor performance if there are highly correlated attributes in your dataset. As such, it is a good idea to review all of the pairwise correlations of the attributes in your dataset. You can use the `corr()` function on the Pandas DataFrame to calculate a correlation matrix.

```
# Pairwise Pearson correlations from
pandas import read_csv from
pandas import set_option
filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
        = read_csv(filename, names=names) set_option('display.width', 100)
set_option('precision', 3)
correlations = data.corr(method='pearson')
print(correlations)
```

Example of reviewing correlations of attributes in the data.

The matrix lists all attributes across the top and down the side to give the correlation between all pairs of attributes (twice, because the matrix is symmetrical). You can see the diagonal line through the matrix from the top left to bottom right corners of the matrix shows perfect correlation of each attribute with itself.

Output of reviewing correlations of attributes in the data.

3.10 Skew of Univariate Distributions

Skew refers to a distribution that is assumed to be Gaussian (normal or bell curve) and that is shifted or squashed in one direction or another. Many machine learning algorithms assume a Gaussian distribution. Knowing that an attribute has a skew may allow you to perform data preparation to correct the skew and later improve the accuracy of your models. You can calculate the skew of each attribute using the `skew()` function on the Pandas DataFrame.

```
# Skew for each attribute from
pandas import read_csv
filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']

data = read_csv(filename, names=names) skew =
    data.skew()
print(skew)
```

Example of reviewing the skew of attribute distributions in the data.

The skew result shows a positive (right) or negative (left) skew. Values closer to zero show less skew.

preg	0.901674
plas	0.173754
pres	-1.843608
skin	0.109372
test	2.272251
mass	-0.428982
pedi	1.919911
age	1.129597
class	0.635017

Output of reviewing the skew of attribute distributions in the data.

3.11 Tips To Remember

1. This section gives you some tips to remember when reviewing your data using summary statistics.
2. Review the numbers.
3. Generating the summary statistics is not enough.
4. Take a moment to pause, read and really think about the numbers you are seeing. Ask why.
5. Review your numbers and ask a lot of questions.
6. How and why are you seeing specific numbers?
7. Think about how the numbers relate to the problem domain in general and specific entities that observations relate to.
8. Write down your ideas.
9. Write down your observations and ideas.
10. Keep a small notebook or notepad and jot down all of the ideas for how variables may relate, for what numbers mean, and ideas for techniques to

try later.

11. The things you write down now while the data is fresh will be very valuable later when you are trying to think up new things to try.

In-Text Questions and Answers (ITQs and ITAs)

ITQ 3.1: Why is it important to understand the considerations when loading machine learning data, especially from CSV files?

ITA: It is important to understand considerations such as file headers, comments, delimiters, and quotes when loading data because these factors can impact how the data is read and processed, ensuring that it is correctly interpreted and used for machine learning tasks.

ITQ 3.2: What is the significance of the Pima Indians dataset in the context of machine learning data loading?

ITA: The Pima Indians dataset is significant because it serves as a classification problem example, where the goal is to predict whether a patient will have diabetes within five years based on various medical attributes. It is commonly used to demonstrate data loading and machine learning techniques.

Conclusion

In conclusion, this chapter has equipped you with essential techniques for loading and understanding machine learning data using Python. By introducing multiple methods for loading CSV files, including the Python Standard Library, NumPy, and Pandas, the chapter ensures a thorough grasp of the data loading

process. The provided insights into handling considerations such as file headers, delimiters, and quotes further empower readers to preprocess data effectively. Moreover, the chapter's focus on the Pima Indians dataset demonstrates the practical application of these techniques in a real-world context.

The chapter's exploration of data understanding through descriptive statistics, dimensions, attribute types, class distribution, correlations, and skewness enhances readers' ability to glean valuable insights from their datasets. By emphasising critical thinking and careful observation, the chapter not only imparts technical skills but also promotes a deeper understanding of data analysis. These skills are crucial for developing successful machine learning models and making informed decisions throughout the data science process. Thus, the chapter serves as an invaluable resource for practitioners seeking to master the art of loading and comprehending machine learning data.

Summary

In this study session, you have learned to:

1. Load CSV Files with the Python Standard Library
2. Load CSV Files with NumPy.
3. Various Considerations when Loading CSV Data
4. Use Pima Indian Dataset
5. Understand your data with descriptive statistics:
6. Visually inspect raw data using the `head()` function in Pandas to view the first few rows of the dataset.

7. Use dimensions of Your Data
8. Use data type for each attribute
9. Use `describe()` function in Pandas, which provides useful statistical summaries for each attribute, including count, mean, standard deviation, and percentiles.
10. Use class distribution
11. Use correlations between attributes
12. Use skewness of attribute distributions to calculate skew using the `skew()` function.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 3.1: What are some considerations you need to keep in mind when loading CSV data for a machine learning project?

Answer: Some considerations you need to keep in mind when loading CSV data are:

- The number of columns in the CSV file.
- The presence of comments in the CSV file.
- d) The size of the dataset.

SAQ 3.2: Why is the Pima Indians dataset commonly used in machine learning tutorials?

Answer: The Pima Indians dataset is widely used because it is a simple classification problem with numeric input attributes and a binary output variable (diabetes prediction), making it an ideal choice for demonstrating data loading,

preprocessing, and machine learning techniques.

SAQ 3.3: How can you load CSV data using NumPy?

Answer: CSV data can be loaded using the `numpy.loadtxt()` function, which assumes no header row and that all data has the same format. It can handle both local files and URLs, returning the data as a NumPy array for numerical analysis.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **What is the primary advantage of using Pandas for loading CSV files in machine learning?**

- A) It requires no installation
- B) It automatically handles headers and missing values
- C) It is slower than NumPy.
- D) It can only load small datasets

Answer: B) It automatically handles headers and missing values

Explanation: Pandas automatically handles file headers, data types, and missing values, making it much easier and more flexible for working with CSV data compared to other methods.

2. **What is the Pima Indians dataset typically used for in machine learning?**

- A) Regression
- B) Classification
- C) Clustering
- D) Dimensionality reduction

Answer: B) Classification

Explanation: The Pima Indians dataset is used for a classification problem, where the goal is to predict whether a person has diabetes within five years based on medical records.

3. **Which Python function is used to check the first few rows of a dataset loaded with Pandas?**

- A) head()
- B) tail()
- C) describe()
- D) shape()

Answer: A) head()

Explanation: The head() function is used to display the first few rows of the dataset, allowing for an initial inspection of the data.

4. **When loading CSV files with NumPy, which function is used?**

- A) loadtxt()
- B) read_csv()
- C) csv.reader()
- D) pandas.load()

Answer: A) loadtxt()

Explanation: The loadtxt() function from NumPy is used to load CSV data into an ndarray, assuming no headers and a consistent data format.

5. **What type of data is used in the Pima Indians dataset?**

- A) Textual data
- B) Numeric data with a binary outcome

C) Image data

D) Audio data

Answer: B) Numeric data with a binary outcome

Explanation: The Pima Indians dataset contains numeric medical data, and the outcome variable is binary, indicating whether or not the patient has diabetes.

Tutor-Marked Assessments (TMAs)

1. What are the common formats used for machine learning data, and why is the CSV format widely used?
2. Describe the considerations that need to be taken into account when loading CSV data for machine learning.
3. After loading the Pima Indians dataset, demonstrate how you would use Pandas to explore and understand the data. Discuss the use of the `head()`, `shape()`, `dtypes`, and `describe()` functions in your data exploration process.
4. Explain the significance of understanding the class distribution in classification problems. How would you use Pandas to analyse the class distribution in the Pima Indians dataset?

STUDY SESSION

FOUR

UNDERSTANDING YOUR DATA WITH
VISUALISATION

Introduction

In this session, we will delve into the realm of data visualisation, a powerful tool for gaining insights and understanding your machine learning data. Understanding your data is essential to extracting the best possible results from machine learning algorithms, and data visualisation provides a fast and intuitive way to achieve this. In this chapter, we'll focus on harnessing the capabilities of Pandas to visualise your machine learning data in Python. Through a series of techniques, we'll explore how to create various types of visualisations that shed light on different aspects of your dataset. These techniques will include creating univariate plots to understand individual attributes, density plots for distribution insights, and box and whisker plots for summarising attribute distributions. Additionally, we'll delve into multivariate plots like correlation matrix plots and scatter plot matrices, which help reveal interactions between multiple variables in your dataset. This chapter's practical approach, using the Pima Indians' onset of diabetes dataset, will equip you with invaluable skills for effectively visualising your machine learning data. Let's embark on this visualisation journey and uncover the stories hidden within your data.

Learning Outcomes for Study Session Four

When you have studied this session, you should be able to:

- 4.1. Recognise the importance of understanding your data for effective machine learning outcomes, and understand how data visualisation can aid in gaining insights.

- 4.2. Explain the concept of univariate plots and their significance in analysing individual attributes of your dataset.
- 4.3. Create histograms to visualise attribute distributions, and interpret the shape of bins to identify Gaussian, skewed, or exponential distributions.
- 4.4. Discuss density plots as an alternative visualisation technique for understanding attribute distributions, and comprehend the abstract representation they provide.
- 4.5. Gain insights into attribute distributions using box and whisker plots, and understand their components, including medians, percentiles, and outlier values.
- 4.6. Grasp the concept of multivariate plots and their value in illustrating interactions between multiple variables in your dataset.
- 4.7. Create a correlation matrix plot to identify relationships between attributes and interpret positive, negative, and strong correlations.
- 4.8. Generate a scatter plot matrix to visualise pairwise relationships between attributes and identify structured patterns, potentially indicating correlations or relationships.
- 4.9. Interpret scatter plot matrices and understand the significance of symmetrical plots in visualising relationships between attributes and attribute distributions.

4.1 Visualisation

You must understand your data in order to get the best results from machine learning algorithms. The fastest way to learn more about your data is to use data visualisation. In this chapter, you will discover exactly how you can visualise your machine learning data in Python using Pandas. Recipes in this chapter use the Pima Indians' onset of diabetes dataset introduced in Session 3.

4.2 Univariate Plots

In this section, we will look at three techniques that you can use to understand each attribute of your dataset independently.

- Histograms.
- Density Plots.
- Box and Whisker Plots.

4.2.1 Histograms

A fast way to get an idea of the distribution of each attribute is to look at histograms. Histograms group data into bins and provide you a count of the number of observations in each bin. From the shape of the bins you can quickly get a feel for whether an attribute is Gaussian, skewed or even has an exponential distribution. It can also help you see possible outliers.

```
# Univariate Histograms from
    matplotlib import pyplot from
    pandas import read_csv
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
        = read_csv(filename, names=names)
data.hist()

pyplot.show()
```

Example of creating histogram plots.

We can see that perhaps the attributes age, pedi and test may have an exponential distribution. We can also see that perhaps the mass and pres and plas attributes may have a Gaussian or nearly Gaussian distribution. This is interesting because many machine learning techniques assume a Gaussian univariate distribution on the input variables.

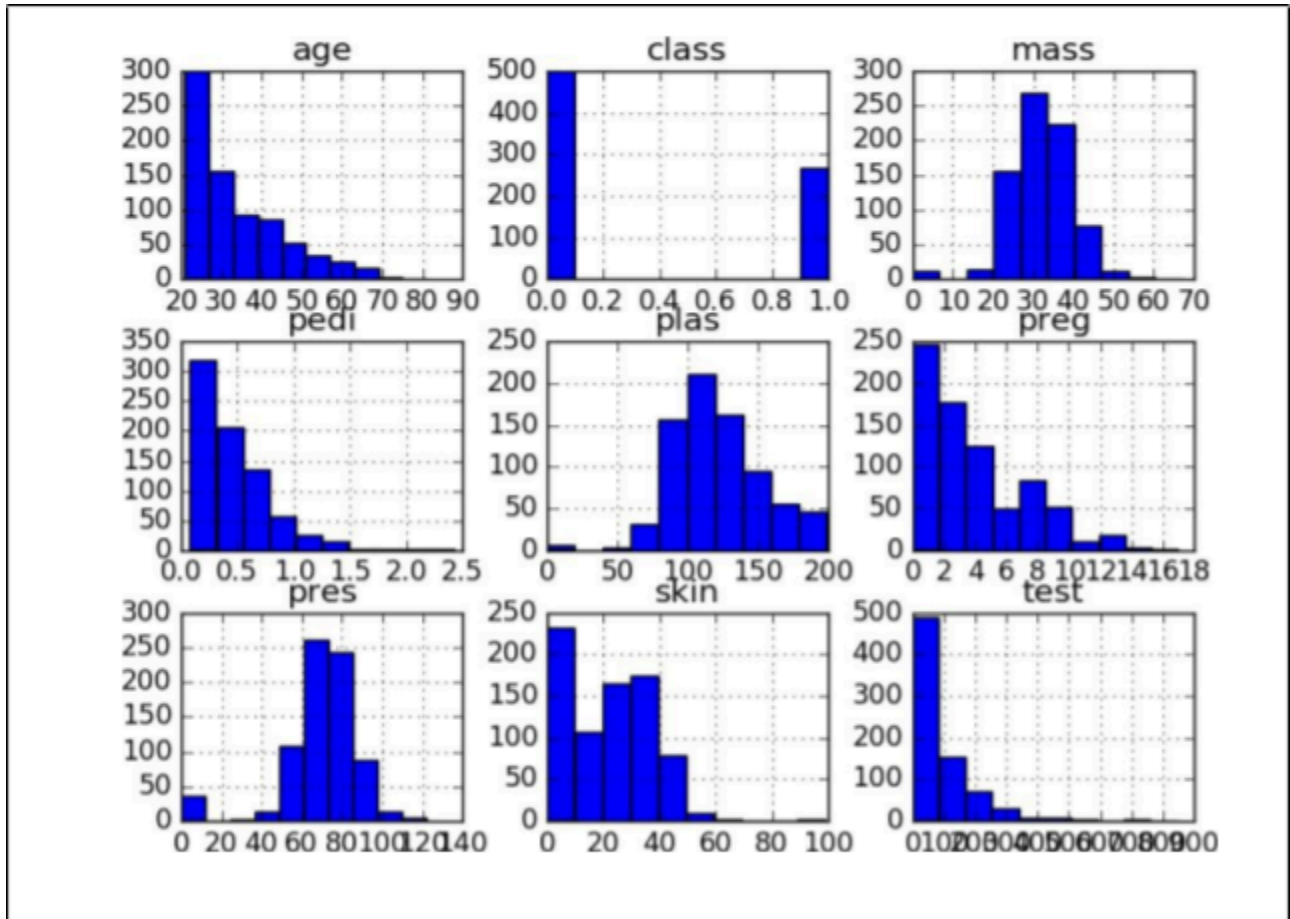


Figure 4.1: Histograms of each attribute

4.2.2 Density Plots

Density plots are another way of getting a quick idea of the distribution of each attribute. The plots look like an abstracted histogram with a smooth curve drawn through the top of each bin, much like your eye tried to do with the histograms.

```
# Univariate Density Plots from
    matplotlib import pyplot from
    pandas import read_csv
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
    = read_csv(filename, names=names)
data.plot(kind='density', subplots=True, layout=(3,3), sharex=False)
pyplot.show()
```

Example of creating density plots.

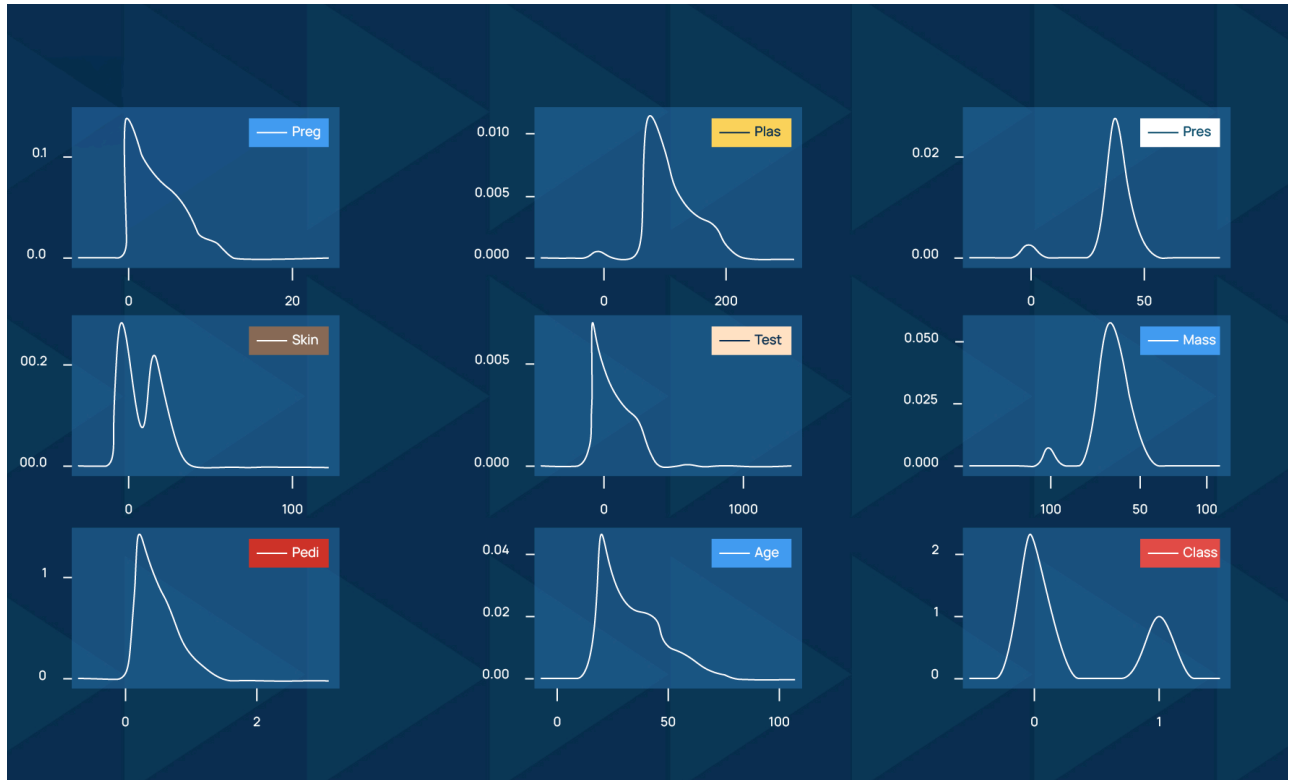


Figure 4.2: Density plots of each attribute

4.2.3 Box and Whisker Plots

Another useful way to review the distribution of each attribute is to use Box and Whisker Plots or boxplots for short. Boxplots summarise the distribution of each attribute, drawing a line for the median (middle value) and a box around the 25th and 75th percentiles (the middle 50% of the data). The whiskers give an idea of the spread of the data and dots outside of the whiskers show candidate outlier values (values that are 1.5 times greater than the size of the spread of the middle 50% of the data).

```
# Box and Whisker Plots from
    matplotlib import pyplot from
    pandas import read_csv
filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
    = read_csv(filename, names=names)
data.plot(kind='box', subplots=True, layout=(3,3), sharex=False,
    sharey=False)
pyplot.show()
```

Example of creating box and whisker plots.

We can see that the spread of attributes is quite different. Some like age, test and skin, appear quite skewed towards smaller values.

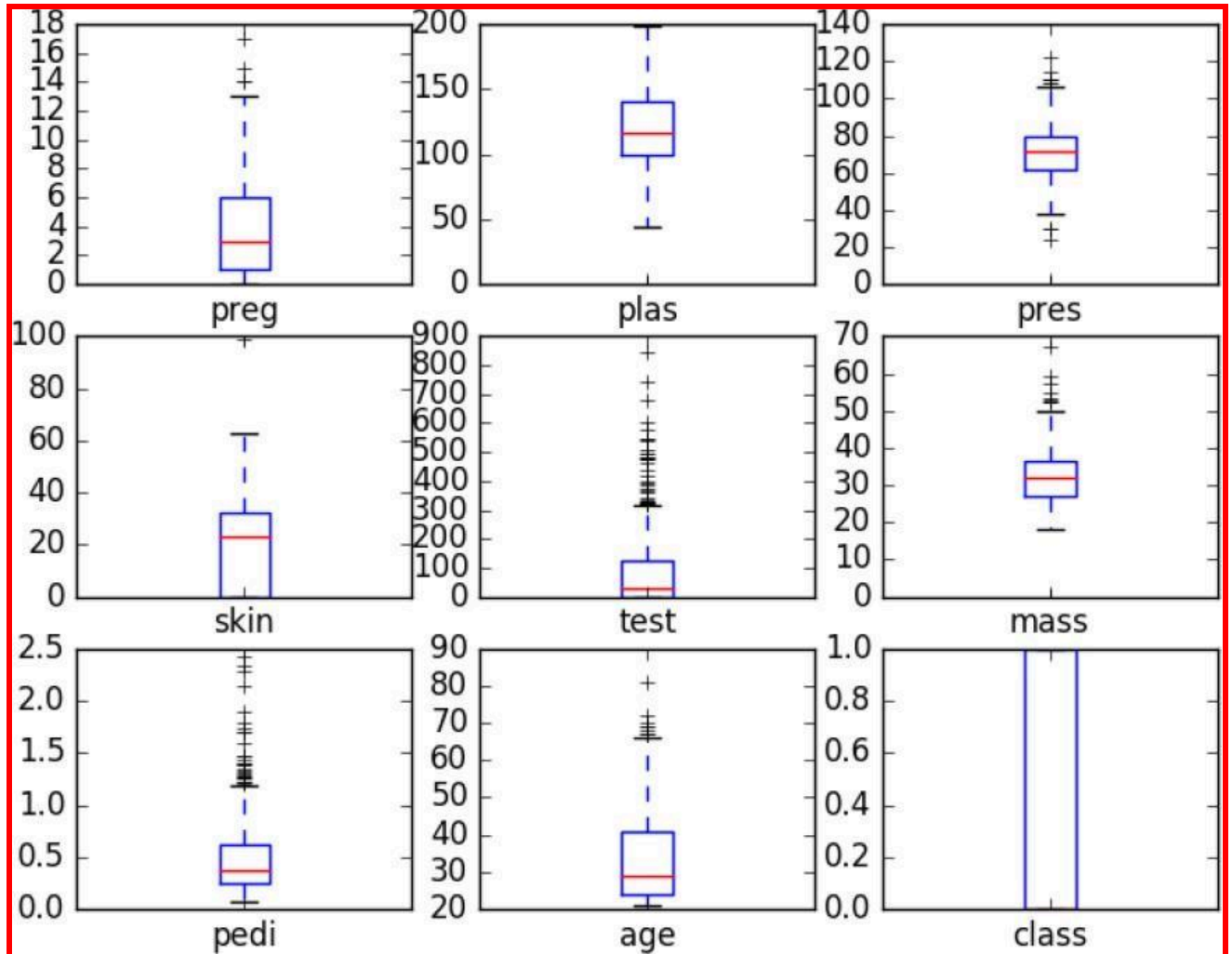


Figure 4.3: Box and whisker plots of each attribute

4.3 Multivariate Plots

This section provides examples of two plots that show the interactions between multiple variables in your dataset.

- Correlation Matrix Plot.
- Scatter Plot Matrix.

4.3.1 Correlation Matrix Plot

Correlation gives an indication of how related the changes are between two variables. If two variables change in the same direction, they are positively correlated. If they change in opposite directions together (one goes up, one goes down), then they are negatively correlated. You can calculate the correlation between each pair of attributes. This is called a correlation matrix. You can then plot the correlation matrix and get an idea of which variables have a high correlation with each other. This is useful to know, because some machine learning algorithms, like linear and logistic regression, can have poor performance if there are highly correlated input variables in your data.

```
# Correlation Matrix Plot from
    matplotlib import pyplot from
    pandas import read_csv
    import numpy
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
    = read_csv(filename, names=names)
correlations = data.corr() # plot
    correlation matrix fig =
        pyplot.figure()
ax = fig.add_subplot(111)
cax = ax.matshow(correlations, vmin=-1, vmax=1)
    fig.colorbar(cax)
ticks = numpy.arange(0,9,1)
```

```
ax.set_xticks(ticks) ax.set_yticks(ticks)

ax.set_xticklabels(names)
ax.set_yticklabels(names)

pyplot.show()
```

Example of creating a correlation matrix plot.

We can see that the matrix is symmetrical, i.e., the bottom left of the matrix is the same as the top right. This is useful, as we can see two different views on the same data in one plot. We can also see that each variable is perfectly positively correlated with each other (as you would have expected) in the diagonal line from top left to bottom right.

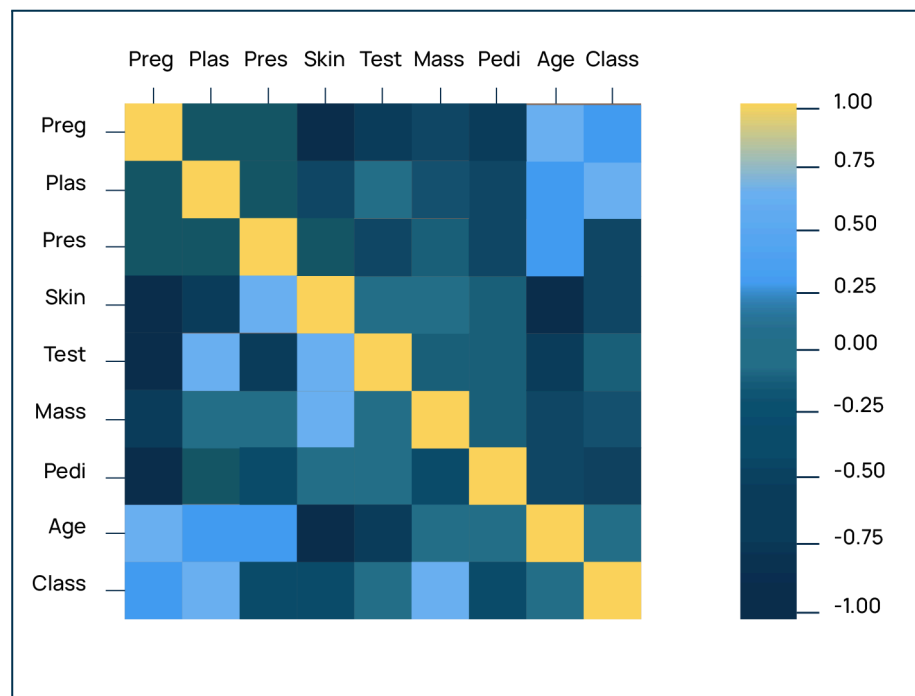


Figure 4.4: Correlation matrix plot

The example is not generic in that it specifies the names for the attributes along the axes as well as the number of ticks. This recipe can be made more generic by removing these aspects as follows:

```
# Correlation Matrix Plot (generic) from
    matplotlib import pyplot
from pandas import read_csv

filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] data
    = read_csv(filename, names=names)
correlations = data.corr()
# plot correlation matrix fig =
    pyplot.figure()

ax = fig.add_subplot(111)
cax = ax.matshow(correlations, vmin=-1, vmax=1)
    fig.colorbar(cax)
pyplot.show()
```

Example of creating a generic correlation matrix plot.

Generating the plot, you can see that it gives the same information, although making it a little harder to see what attributes are correlated by name. Use this generic plot as a first cut to understand the correlations in your dataset and customise it like the first example in order to read more specific data if needed.

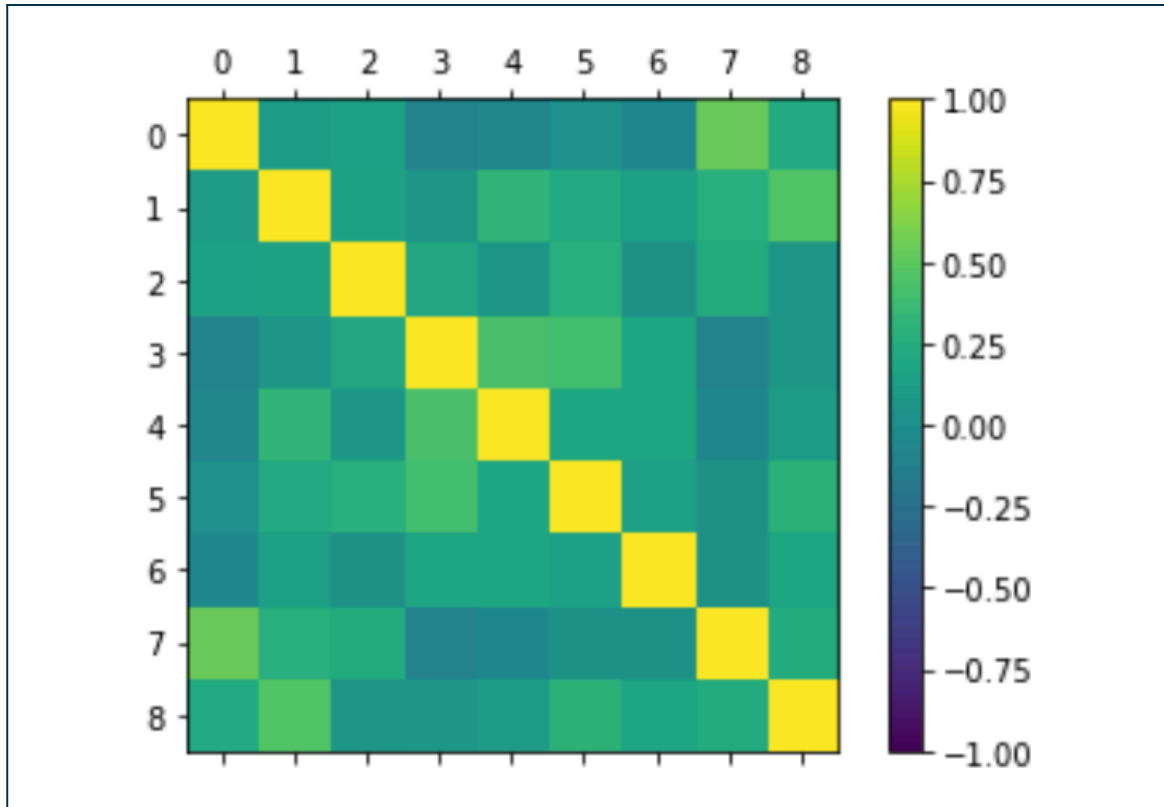


Figure 4.5 Generic correlation matrix plot

4.3.2 Scatter Plot Matrix

A scatter plot shows the relationship between two variables as dots in two dimensions, with one axis for each attribute. You can create a scatter plot for each pair of attributes in your data. Drawing all these scatter plots together is called a scatter plot matrix. Scatter plots are useful for spotting structured relationships between variables, like whether you could summarise the relationship between two variables with a line. Attributes with structured relationships may also be correlated and good candidates for removal from your dataset.

```
# Scatterplot Matrix
from matplotlib import pyplot
from pandas import read_csv
from pandas.plotting import scatter_matrix

filename = "pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = read_csv(filename, names=names)
scatter_matrix(data)
pyplot.show()
```

Example of creating a scatter plot matrix.

Like the Correlation Matrix Plot above, the scatter plot matrix is symmetrical. This is useful to look at the pairwise relationships from different perspectives. Because there is little point in drawing a scatter plot of each variable with itself, the diagonal shows histograms of each attribute.

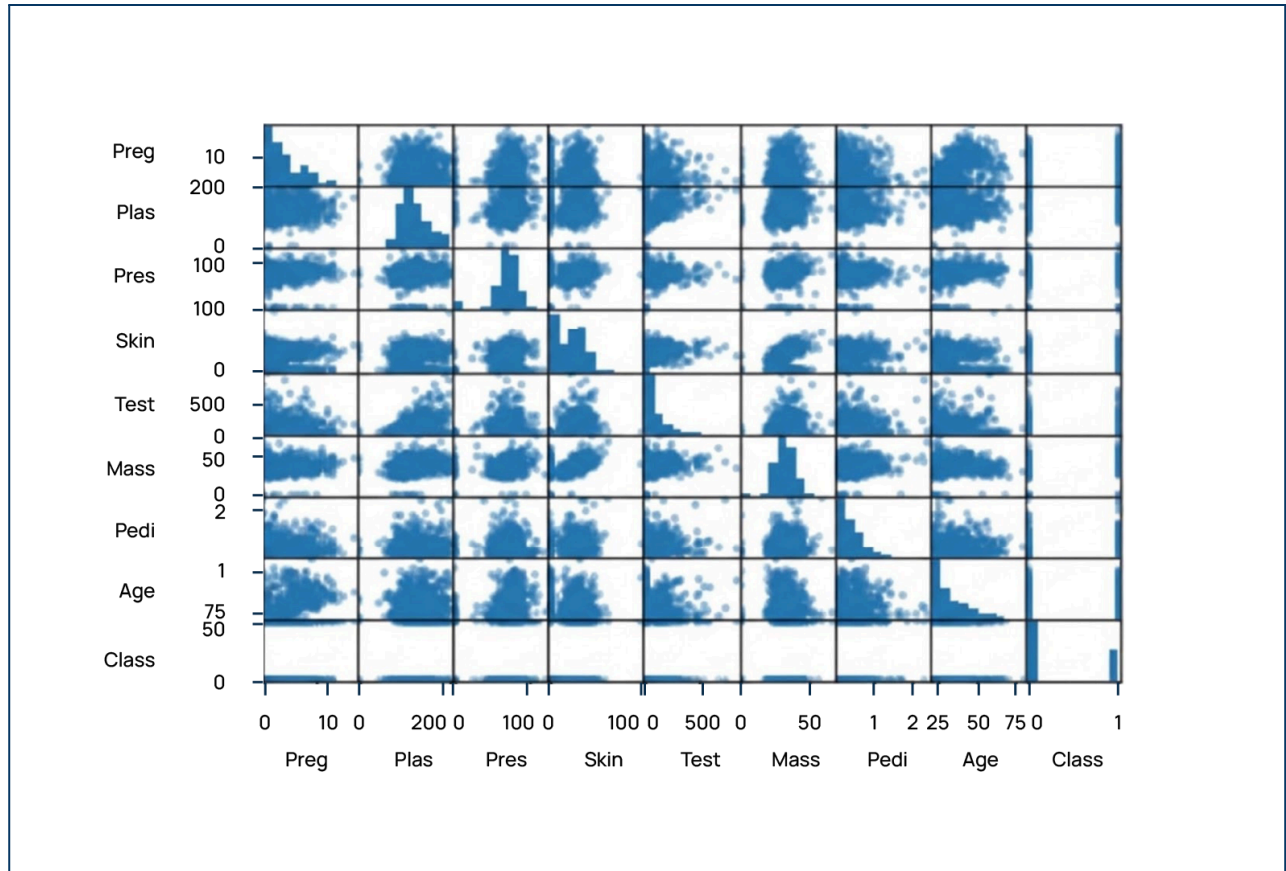


Figure 4.6: Scatter plot matrix of the data

In-Text Questions and Answers (ITQs and ITAs)

ITQ 4.1: Why is data visualisation crucial for machine learning outcomes?

ITA: Data visualisation is crucial because it allows practitioners to explore, understand, and identify patterns in the data quickly. It helps in making informed decisions, detecting outliers, and understanding the distributions and relationships between variables, which are essential for building effective machine learning models.

ITQ 4.2: What insights can histograms provide when exploring a dataset?

ITA: Histograms provide a visual representation of the distribution of each attribute. They show how data is spread across different bins, helping to identify if the distribution is Gaussian, skewed, or exponential. They also highlight the presence of outliers in the dataset.

Conclusion

To conclude, this chapter has delved into the crucial realm of data visualisation, an indispensable tool for gaining insights from machine learning datasets. The paramount significance of understanding data before embarking on machine learning endeavours has been emphasised. By harnessing the power of data visualisation, practitioners are equipped to explore and comprehend their datasets more effectively. The chapter has presented a comprehensive guide to univariate plots, featuring histograms, density plots, and box and whisker plots, providing a deeper understanding of individual attribute distributions and potential outliers.

Moving beyond univariate analysis, the chapter has illuminated the realm of multivariate plots, introducing the concepts of correlation matrix plots and scatter plot matrices. These techniques facilitate the examination of interrelationships between variables and reveal patterns that may influence the performance of machine learning algorithms. Through practical examples and insightful commentary, the chapter equips readers with the skills to analyse attribute correlations and identify potential redundancies.

By emphasising the hands-on implementation of visualisation techniques using Python's Pandas library, the chapter empowers practitioners to translate theoretical concepts into tangible actions. Ultimately, the chapter serves as an indispensable guide for data scientists and machine learning enthusiasts seeking to unravel the intricate layers of their datasets, fostering a deeper understanding that is fundamental to effective model building and decision-making.

Summary

In this session, you discovered a number of ways that you can better understand your machine learning data in Python using Pandas. Specifically, you learnt how to plot your data using:

- Histograms
- Density Plots
- Box and Whisker Plots
- Correlation Matrix Plot
- Scatter Plot Matrix

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 4.1: What are univariate plots, and why are they important in data exploration?

Answer: Univariate plots, including histograms, density plots, and box and whisker plots, focus on understanding the distribution of individual attributes. They help identify the shape, spread, and outliers of each attribute, which is essential for preprocessing and choosing appropriate machine learning models.

SAQ 4.2: How can a histogram help in identifying the nature of an attribute's distribution?

Answer: A histogram groups data into bins, providing a visual representation of how values are distributed. By examining the shape of the bins, you can identify whether the attribute follows a Gaussian, skewed, or exponential distribution, which informs the choice of machine learning models and transformations.

SAQ 4.3: What is the primary advantage of using a density plot over a histogram?

Answer: The primary advantage of a density plot is that it provides a smoother, continuous representation of the distribution, making it easier to understand the underlying trend of the data without being influenced by bin sizes, which can sometimes distort the interpretation in histograms.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. What is the purpose of using histograms in data visualisation?

- A) To show correlations between attributes
- B) To display attribute distributions
- C) To create smooth curves for data
- D) To summarise statistical properties of attributes

Answer: B) To display attribute distributions

Explanation: Histograms group data into bins and visually show the distribution of an attribute, helping to identify patterns like Gaussian, skewed, or exponential distributions.

2. Which plot would you use to visualise the distribution of an attribute while smoothing out the noise?

- A) Scatter plot
- B) Box and whisker plot
- C) Density plot
- D) Histogram

Answer: C) Density plot

Explanation: Density plots smooth the data, providing a continuous representation of the distribution and highlighting the overall pattern, unlike histograms, which can be affected by bin sizes.

3. What does a box and whisker plot help you understand?

- A) The mean of an attribute
- B) The distribution, spread, and outliers of an attribute
- C) Pairwise relationships between multiple variables
- D) The correlation between attributes

Answer: B) The distribution, spread, and outliers of an attribute

Explanation: Box and whisker plots summarise an attribute's distribution by showing the median, quartiles, and outliers, helping to understand the spread and detect unusual data points.

4. Which of the following visualisation techniques is used to examine interactions between multiple variables?

- A) Histogram
- B) Box plot

- C) Scatter plot matrix
- D) Line plot

Answer: C) Scatter plot matrix

Explanation: A scatter plot matrix shows pairwise relationships between multiple variables, helping to identify structured patterns or correlations between attributes.

5. **What does a correlation matrix plot reveal about your data?**

- A) The individual distributions of each attribute
- B) The relationships between pairs of attributes
- C) The outliers in the data
- D) The time distribution of the data

Answer: B) The relationships between pairs of attributes

Explanation: A correlation matrix plot shows how variables relate to each other, indicating positive or negative correlations, which can help in feature selection and improving model performance.

Tutor Marked Assessments (TMAs)

1. How does visualising data help in gaining insights and understanding the underlying patterns?
2. Explain how histograms can provide insights into the distribution of data and the presence of outliers.
3. How can you identify potential outliers using box and whisker plots?
Discuss how outliers can impact machine learning models and how they can be handled.

4. Provide a detailed explanation of how data visualisation aids in the process of feature engineering. Use the Pima Indians dataset to showcase your answer with appropriate plots.

STUDY SESSION

FIVE

DATA PREPARATION FOR MACHINE LEARNING

Introduction

In this session, our focus shifts to the critical domain of feature selection in the context of machine learning, a pivotal step aimed at enhancing model performance by harnessing the most pertinent data attributes. The quality and relevance of features employed for training machine learning models profoundly influence their effectiveness. By eliminating irrelevant or partially relevant features, we can substantially boost model performance. Throughout this chapter, we will uncover automatic feature selection techniques in Python, capitalising on the capabilities of scikit-learn. As this session unfolds, you'll cultivate a deep comprehension of methods such as Univariate Selection, Recursive Feature Elimination, Principal Component Analysis (PCA), and Feature Importance. Armed with these techniques, we will streamline the feature set and enhance data compatibility with machine learning algorithms. Practical demonstrations using the renowned Pima Indian diabetes dataset will solidify your ability to implement these strategies adeptly. Let's embark on this journey into the realm of feature selection, equipping ourselves with the tools to refine our data for more accurate and efficient machine learning models.

Learning Outcomes for Study Session Five

When you have studied this session, you should be able to:

- 5.1. Discuss the critical role of data preparation in machine learning and its impact on model training, including how data assumptions can affect algorithm performance.

- 5.2. Master data preparation techniques such as rescaling, standardising, normalising, and binarising using scikit-learn in Python, and recognise their significance in optimising data for machine learning algorithms.
- 5.3. Apply hands-on examples with the Pima Indian diabetes dataset to effectively implement data transformation methods and make data compatible with machine learning algorithms.
- 5.4. Grasp the importance of feature selection in enhancing model performance and efficiency, and understand how the quality and relevance of features influence machine learning models.
- 5.5. Explore automatic feature selection techniques such as Univariate Selection, Recursive Feature Elimination, Principal Component Analysis (PCA), and Feature Importance using scikit-learn in Python.
- 5.6. Utilise practical examples with the Pima Indian diabetes dataset to gain a deep understanding of feature selection methods and their application.
- 5.7. Discover evaluation techniques for assessing machine learning algorithm performance, including cross-validation and train/test splits, to estimate model generalisation on new data.
- 5.8. Interpret a variety of classification and regression metrics, including classification accuracy, logarithmic loss, ROC curve area, confusion matrix, mean absolute error, mean squared error, and R-squared, to evaluate different aspects of model performance.
- 5.9. Gain the ability to make informed decisions about the effectiveness of machine learning algorithms based on the techniques and metrics explored in this session.

5.1 Prepare Your Data For Machine Learning

Many machine learning algorithms make assumptions about your data. It is often a very good idea to prepare your data in such a way as to best expose the structure of the problem to the machine learning algorithms that you intend to use. In this chapter, you will discover how to prepare your data for machine learning in Python using scikit-learn. After completing this lesson, you will know how to:

- Rescale data.
- Standardise data.
- Normalise data.
- Binarise data.

5.2 Need For Data Pre-Processing

You almost always need to preprocess your data. It is a required step. A difficulty is that different algorithms make different assumptions about your data and may require different transformations. Further, when you follow all of the rules and prepare your data, sometimes algorithms can deliver better results without preprocessing.

Generally, It is recommended to start by creating many different views and transforms of your data, then exercise a handful of algorithms on each view of your dataset. This will help you figure out which data transformations might be better at exposing the structure of your problem in general.

Data Transforms

In this lesson, you will work through 4 different data pre-processing recipes for machine learning. The Pima Indian diabetes dataset is used in each recipe.

Each recipe follows the same structure:

- Load the dataset from a URL.
- Split the dataset into the input and output variables for machine learning.
- Apply a preprocessing transform to the input variables.
- Summarise the data to show the change.

The scikit-learn library provides two standard idioms for transforming data. Each is useful in different circumstances. The transforms are calculated in such a way that they can be applied to your training data and any samples of data you may have in the future. The scikit-learn documentation has some information on how to use various different preprocessing methods:

- Fit and Multiple Transform
- Combined Fit-And-Transform

The Fit and Multiple Transform method is the preferred approach. You call the `fit()` function to prepare the parameters of the transform once on your data. Then later, you can use the `transform()` function on the same data to prepare it for modelling and again on the test or validation dataset or new data that you may see in the future. The Combined Fit-And-Transform is a convenience that you can use for one-off tasks. This might be useful if you are interested in plotting or

summarising the transformed data. You can review the preprocess API in [scikit-learn](#) here.

5.3 Rescale Data

When your data is composed of attributes with varying scales, many machine learning algorithms can benefit from rescaling the attributes to all have the same scale. Often this is referred to as normalisation, and attributes are often rescaled into the range between 0 and 1. This is useful for optimisation algorithms used in the core of machine learning algorithms like gradient descent. It is also useful for algorithms that weight inputs like regression and neural networks and algorithms that use distance measures like k-Nearest Neighbours. You can rescale your data using scikit-learn using the `MinMaxScaler` class.

```
# Rescale data (between 0 and 1) from pandas
import read_csv
from numpy import set_printoptions
from sklearn.preprocessing import MinMaxScaler

filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(filename, names=names)
array = dataframe.values

# separate array into input and output components.
X = array[:,0:8]
Y = array[:,8]

scaler = MinMaxScaler(feature_range=(0, 1))
```

```
rescaledX = scaler.fit_transform(X) #  
  
    summarize transformed data  
  
    set_printoptions(precision=3)  
  
print(rescaledX[0:5,:])
```

Example of rescaling data.

After rescaling, you can see that all of the values are in the range between 0 and 1.

```
[[ 0.353 0.744 0.59  0.354 0.      0.501 0.234 0.483]  
 [ 0.059 0.427 0.541 0.293 0.      0.396 0.117 0.167]  
 [ 0.471 0.92  0.525 0.      0.      0.347 0.254 0.183]  
 [ 0.059 0.447 0.541 0.232 0.111 0.419 0.038 0.   ]  
 [ 0.      0.688 0.328 0.354 0.199 0.642 0.944 0.2  ]]
```

Output of rescaling data.

5.4 Standardise Data

Standardisation is a useful technique to transform attributes with a Gaussian distribution and differing means and standard deviations into a standard Gaussian distribution with a mean of 0 and a standard deviation of 1. It is most suitable for techniques that assume a Gaussian distribution in the input variables and work better with rescaled data, such as linear regression, logistic regression, and linear discriminant analysis. You can standardise data using scikit-learn with the StandardScaler class.

```
# Standardize data (0 mean, 1 stdev)  
from sklearn.preprocessing import StandardScaler from  
pandas import read_csv
```

```
from numpy import set_printoptions filename =  
    'pima-indians-diabetes.data.csv'  
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =  
    read_csv(filename, names=names)  
array = dataframe.values  
  
# separate array into input and output components X =  
    array[:,0:8]  
Y = array[:,8]  
  
scaler = StandardScaler().fit(X)  
    rescaledX = scaler.transform(X)  
  
# summarize transformed data  
    set_printoptions(precision=3)  
    print(rescaledX[0:5,:])
```

Example of standardising data.

The values for each attribute now have a mean value of 0 and a standard deviation of 1.

```
[[ 0.64  0.848 0.15  0.907 -0.693 0.204 0.468 1.426]  
 [-0.845 -1.123 -0.161 0.531 -0.693 -0.684 -0.365 -0.191]  
 [ 1.234 1.944 -0.264 -1.288 -0.693 -1.103 0.604 -0.106]  
 [-0.845 -0.998 -0.161 0.155 0.123 -0.494 -0.921 -1.042]  
 [-1.142 0.504 -1.505 0.907 0.766 1.41  5.485 -0.02 ]]
```

Output of rescaling data.

5.5 Normalise Data

Normalising in scikit-learn refers to rescaling each observation (row) to have a length of 1 (called a unit norm or a vector with the length of 1 in linear algebra). This pre-processing method can be useful for sparse datasets (lots of zeros) with attributes of varying scales when using algorithms that weight input values, such as neural networks, and algorithms that use distance measures, such as k-Nearest Neighbours. can normalise data in Python with scikit-learn using the Normaliser class.

```
# Normalize data (length of 1)
from sklearn.preprocessing import Normalizer from
pandas import read_csv
from numpy import set_printoptions filename =
    'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values

# separate array into input and output components X =
    array[:,0:8]
Y = array[:,8]
scaler = Normalizer().fit(X) normalizedX
    = scaler.transform(X) # summarize
    transformed data
    set_printoptions(precision=3)
    print(normalizedX[0:5,:])
```

Example of normalising data.

The rows are normalised to length 1.

```
[ [ 0.034 0.828 0.403 0.196 0.    0.188 0.004 0.28 ]  
  [ 0.008 0.716 0.556 0.244 0.    0.224 0.003 0.261]  
  [ 0.04  0.924 0.323 0.    0.    0.118 0.003 0.162]  
  [ 0.007 0.588 0.436 0.152 0.622 0.186 0.001 0.139]  
  [ 0.    0.596 0.174 0.152 0.731 0.188 0.01  0.144]]
```

Output of normalising data.

5.6 Binarise Data (Make Binary)

You can transform your data using a binary threshold. All values above the threshold are marked 1 and all values equal to or below the threshold are marked as 0. This is called 'binarising your data' or 'thresholding your data'. It can be useful when you have probabilities that you want to make into crisp values. It is also useful when you are feature engineering and want to add new features that indicate something meaningful. You can create new binary attributes in Python using scikit-learn with the Binarizer class.

```
# binarization  
from sklearn.preprocessing import Binarizer from  
    pandas import read_csv  
from numpy import set_printoptions filename =  
    'pima-indians-diabetes.data.csv'  
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =  
    read_csv(filename, names=names)  
array = dataframe.values  
  
# separate array into input and output components
```

```
X = array[:,0:8]
Y = array[:,8]
binarizer = Binarizer(threshold=0.0).fit(X) binaryX
            = binarizer.transform(X)
# summarize transformed data
set_printoptions(precision=3)
print(binaryX[0:5,:])
```

Example of binarising data.

You can see that all values equal to or less than 0 are marked 0 and all of those above 0 are marked 1.

Output of binarising data.

```
[[ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  0.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  1.  1.  1.  1.]
 [ 0.  1.  1.  1.  1.  1.  1.  1.]
```

5.7 Feature Selection For Machine Learning

The data features that you use to train your machine learning models have a huge influence on the performance you can achieve. Irrelevant or partially relevant features can negatively impact model performance. In this chapter, you will discover automatic feature selection techniques that you can use to prepare your machine learning data in Python with scikit-learn. After completing this lesson, you will know how to use:

- a. Univariate Selection.
- b. Recursive Feature Elimination.
- c. Principal Component Analysis.
- d. Feature Importance. Let's get started.

Feature Selection

Feature selection is a process where you automatically select those features in your data that contribute most to the prediction variable or output in which you are interested. Having irrelevant features in your data can decrease the accuracy of many models, especially linear algorithms like linear and logistic regression.

Three benefits of performing feature selection before modelling your data are:

- **Reduces Overfitting:** Less redundant data means less opportunity to make decisions based on noise.
- **Improves Accuracy:** Less misleading data means modelling accuracy improves.
- **Reduces Training Time:** Less data means that algorithms train faster.

You can learn more about feature selection with scikit-learn in the article [Feature Selection](#). Each feature selection recipe will use the Pima Indians onset of diabetes dataset.

5.7.1 Univariate Selection

Statistical tests can be used to select those features that have the strongest relationship with the output variable. The scikit-learn library provides the `SelectKBest` class that can be used with a suite of different statistical tests to select a specific number of features. The example below uses the chi-squared

(chi2) statistical test for non-negative features to select 4 of the best features from the Pima Indians onset of diabetes dataset.

```
# summarize selected features  
  
print(features[0:5,:])
```

Example of univariate feature selection.

You can see the scores for each attribute and the 4 attributes chosen (those with the highest scores): plas, test, mass and age. I got the names for the chosen attributes by manually mapping the index of the 4 highest scores to the index of the attribute names.

Output of univariate feature selection.

```
[ 111.52  1411.887   17.605   53.108  2175.565  127.669    5.393  
 181.304]  
[[ 148.    0.   33.6  50. ]  
 [  85.    0.   26.6  31. ]  
 [ 183.    0.   23.3  32. ]  
 [  89.   94.   28.1  21. ]  
 [ 137.  168.   43.1  33. ]]
```

5.7.2 Recursive Feature Elimination

The Recursive Feature Elimination (or RFE) works by recursively removing attributes and building a model on those attributes that remain. It uses the model accuracy to identify which attributes (and combinations of attributes) contribute the most to predicting the target attribute. You can learn more about the RFE class in the scikit-learn documentation. The example below uses RFE with the logistic regression algorithm to select the top 3 features. The choice of algorithm does not matter too much as long as it is skilful and consistent.

Feature Extraction with RFE

```
from pandas import read_csv
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression #
    load data
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8]
# feature extraction
model = LogisticRegression() rfe =
    RFE(model, 3)
fit = rfe.fit(X, Y)
print("Num Features: %d" % fit.n_features_)
    print("Selected Features: %s" % fit.support_)
    print("Feature Ranking: %s" % fit.ranking_)
```

Example of RFE feature selection.

You can see that RFE chose the top 3 features as preg, mass and pedi. These are marked 'True' in the support array and marked with a choice 1 in the ranking array. Again, you can manually map the feature indexes to the indexes of attribute names.

```
Num Features: 3  
Selected Features: [ True False False False True True False]  
Feature Ranking: [1 2 3 5 6 1 1 4]
```

Output of RFE feature selection.

5.7.2 Principal Component Analysis

Principal Component Analysis (or PCA) uses linear algebra to transform the dataset into a compressed form. Generally this is called a data reduction technique. A property of PCA is that you can choose the number of dimensions or principal components in the transformed result. In the example below, we use PCA and select 3 principal components. Learn more about the PCA class in scikit-learn by reviewing the API.

```
# Feature Extraction with PCA from  
pandas import read_csv  
from sklearn.decomposition import PCA # load  
data  
filename = 'pima-indians-diabetes.data.csv'  
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =  
    read_csv(filename, names=names)  
array = dataframe.values X =  
    array[:,0:8]  
Y = array[:,8]  
# feature extraction  
pca = PCA(n_components=3) fit =  
    pca.fit(X)
```

```
# summarize components
print("Explained Variance: %s" % fit.explained_variance_ratio_)

print(fit.components_)
```

Example of PCA feature extraction.

You can see that the transformed dataset (3 principal components) bears little resemblance to the source data.

```
Explained Variance: [ 0.88854663 0.06159078 0.02579012]
[[ -2.02176587e-03  9.78115765e-02  1.60930503e-02  6.07566861e-02
   9.93110844e-01  1.40108085e-02  5.37167919e-04 -3.56474430e-03]
 [ 2.26488861e-02  9.72210040e-01  1.41909330e-01 -5.78614699e-02
  -9.46266913e-02  4.69729766e-02  8.16804621e-04  1.40168181e-01]
 [ -2.24649003e-02  1.43428710e-01 -9.22467192e-01 -3.07013055e-01
   2.09773019e-02 -1.32444542e-01 -6.39983017e-04 -1.25454310e-01]]
```

Output of PCA feature extraction.

5.7.3 Feature Importance

Bagged decision trees like Random Forest and Extra Trees can be used to estimate the importance of features. In the example below we construct an ExtraTreesClassifier classifier for the Pima Indians onset of diabetes dataset. You can learn more about the ExtraTreesClassifier class in the scikit-learn API.

```
# Feature Importance with Extra Trees Classifier from
    pandas import read_csv
from sklearn.ensemble import ExtraTreesClassifier # load
    data
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
```

```
dataframe = read_csv(filename, names=names) array =  
    dataframe.values  
X = array[:,0:8]  
Y = array[:,8]  
# feature extraction  
model = ExtraTreesClassifier() model.fit(X,  
    Y)  
print(model.feature_importances_)
```

Example of feature importance.

You can see that we are given an importance score for each attribute, where the larger the score, the more important the attribute. The scores suggest the importance of plans, age and mass.

```
[ 0.11070069  0.2213717  0.08824115  0.08068703  0.07281761  0.14548537  0.12654214  0.15415431]
```

Output of feature importance.

Evaluate the Performance of Machine Learning Algorithms with Resampling

You need to know how well your algorithms perform on unseen data. The best way to evaluate the performance of an algorithm would be to make predictions for new data for which you already know the answers. The second best way is to use clever techniques from statistics called resampling methods that allow you to make accurate estimates for how well your algorithm will perform on new data. In this chapter, you will discover how you can estimate the accuracy of your machine

learning algorithms using resampling methods in Python and scikit-learn on the Pima Indians dataset.

5.8 Evaluate Machine Learning Algorithms

Why can't you prepare your machine learning algorithm on your training dataset and use predictions from this same dataset to evaluate performance? The simple answer is overfitting. Imagine an algorithm that remembers every observation it is shown during training. If you evaluated your machine learning algorithm on the same dataset used to train the algorithm, then an algorithm like this would have a perfect score on the training dataset. But the predictions it made on new data would be terrible. We must evaluate our machine learning algorithms on data that is not used to train the algorithm.

The evaluation is an estimate that we can use to talk about how well we think the algorithm may actually work in practice. It is not a guarantee of performance. Once we estimate the performance of our algorithm, we can then re-train the final algorithm on the entire training dataset and get it ready for operational use. Next up we are going to look at four different techniques that we can use to split up our training dataset and create useful estimates of performance for our machine learning algorithms:

- Train and Test Sets.
- k-fold Cross-Validation.
- Leave-One-Out Cross-Validation.
- Repeated Random Test-Train Splits.

5.9 Split into Train and Test Sets

The simplest method that we can use to evaluate the performance of a machine learning algorithm is to use different training and testing datasets. We can take our original dataset and split it into two parts. Train the algorithm on the first part, make predictions on the second part and evaluate the predictions against the expected results. The size of the split can depend on the size and specifics of your dataset, although it is common to use 67% of the data for training and the remaining 33% for testing.

This algorithm evaluation technique is very fast. It is ideal for large datasets (millions of records) where there is strong evidence that both splits of the data are representative of the underlying problem. Because of the speed, it is useful to use this approach when the algorithm you are investigating is slow to train. A downside of this technique is that it can have a high variance. This means that differences in the training and test datasets can result in meaningful differences in the estimate of accuracy. In the example below, we split the Pima Indians dataset into 67%/33% splits for training and testing and evaluate the accuracy of a Logistic Regression model.

```
# Evaluate using a train and a test set
```

```
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(filename, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
test_size = 0.33
seed = 7
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
                                                    test_size=test_size,
                                                    random_state=seed)
model = LogisticRegression()
model.fit(X_train, Y_train)
result = model.score(X_test, Y_test)
print("Accuracy: %.3f%%" % (result*100.0))
```

Example of evaluating an algorithm with a train and test set.

We can see that the estimated accuracy for the model was approximately 75%.

Note that in addition to specifying the size of the split, we also specify the random seed. Because the split of the data is random, we want to ensure that the results are reproducible. By specifying the random seed we ensure that we

get the same random numbers each time we run the code and, in turn, the same split of data. This is important if we want to compare this result to the estimated accuracy of another machine learning algorithm or the same algorithm with a different configuration. To ensure the comparison was apples-for-apples, we must ensure that they are trained and tested on exactly the same data.

Accuracy: 75.591%

Output of evaluating an algorithm with a train and test set.

5.10 K-fold Cross-Validation

Cross-validation is an approach that you can use to estimate the performance of a machine learning algorithm with less variance than a single train-test set split. It works by splitting the dataset into k parts (e.g., $k = 5$ or $k = 10$). Each split of the data is called a fold. The algorithm is trained on $k - 1$ folds with one held back and tested on the held-back fold. This is repeated so that each fold of the dataset is given a chance to be the held-back test set. After running cross-validation, you end up with k different performance scores that you can summarise using a mean and a standard deviation. The result is a more reliable estimate of the performance of the algorithm on new data. It is more accurate because the algorithm is trained and evaluated multiple times on different types of data. The choice of k must allow the size of each test partition to be large enough to be a reasonable sample of the problem, whilst allowing enough repetitions of the train-test evaluation of the algorithm to provide a fair estimate of the algorithm's performance on unseen data. For modest-sized datasets in the thousands or

tens of thousands of records, k values of 3, 5 and 10 are common. In the example below we use 10-fold cross-validation.

Example of evaluating an algorithm with k -fold Cross-Validation.

You can see that we report both the mean and the standard deviation of the performance measure. When summarising performance measures, it is a good practice to summarise the distribution of the measures, in this case assuming a Gaussian distribution of performance (a very reasonable assumption) and recording the mean and standard deviation.

Accuracy: 76.951% (4.841%)

Output of evaluating an algorithm with k -fold Cross-validation

5.11 Leave-One-Out Cross-Validation

You can configure cross-validation so that the size of the fold is 1 (k is set to the number of observations in your dataset). This variation of cross-validation is called leave-one-out cross-validation. The result is a large number of performance measures that can be summarised in an effort to give a more reasonable estimate of the accuracy of your model on unseen data. A downside is that it can be a computationally more expensive procedure than k -fold cross-validation. In the example below we use leave-one-out cross-validation.

```
# Evaluate using Leave One Out Cross Validation from
pandas import read_csv
from sklearn.model_selection import LeaveOneOut from
sklearn.model_selection import cross_val_score from
sklearn.linear_model import LogisticRegression filename
= 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values
```

```
X = array[:,0:8]
Y = array[:,8]
loocv = LeaveOneOut() model
= LogisticRegression()
results = cross_val_score(model, X, Y, cv=loocv)
print("Accuracy:      %.3f%      (%.3f%%)      %      (results.mean()*100.0,
      %              "
      results.std()*100.0))
```

Example of evaluating an algorithm with Leave One Out Cross-Validation

You can see in the standard deviation that the score has more variance than the k-fold cross-validation results described above.

```
Accuracy: 76.823% (42.196%)
```

Output of evaluating an algorithm with Leave One Out Cross-Validation.

5.12 Repeated Random Test-Train Splits

Another variation on k-fold cross-validation is to create a random split of the data like the train/test split described above but repeat the process of splitting and evaluation of the algorithm multiple times, like cross-validation. This has the speed of using a train/test split and the reduction in variance in the estimated performance of k-fold cross-validation. You can also repeat the process many more times as needed to improve the accuracy. A downside is that repetitions may include much of the same data in the train or the test split from run to run, introducing redundancy into the evaluation. The example below splits the data into a 67%/33% train/test split and repeats the process 10 times.

```
# Evaluate using Shuffle Split Cross Validation from
pandas import read_csv
from sklearn.model_selection import ShuffleSplit from
sklearn.model_selection import cross_val_score from
sklearn.linear_model import LogisticRegression filename
= 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(filename, names=names)
array = dataframe.values X
= array[:,0:8]
Y = array[:,8]
n_splits = 10
test_size = 0.33
seed = 7
kfold = ShuffleSplit(n_splits=n_splits, test_size=test_size,
random_state=seed)
model = LogisticRegression()
results = cross_val_score(model, X, Y, cv=kfold)
print("Accuracy:   %.3f%%   (%.3f%%)   %   (results.mean()*100.0,
      %.3f%%   "
      results.std()*100.0))
```

Example of evaluating an algorithm with Shuffle Split Cross-Validation.

We can see that in this case the distribution of the performance measure is on par with k-fold cross-validation above.

Accuracy: 76.496% (1.698%)

Output of evaluating an algorithm with Shuffle Split Cross-Validation.

What Techniques to Use When

This section lists some tips to consider when deciding what resampling technique to use in different circumstances.

- Generally, k-fold cross-validation is the gold standard for evaluating the performance of a machine learning algorithm on unseen data with k set to 3, 5, or 10.
- Using a train/test split is good for speed when using a slow algorithm and produces performance estimates with lower bias when using large datasets.
- Techniques like leave-one-out cross-validation and repeated random splits can be useful intermediates when trying to balance variance in the estimated performance, model training speed and dataset size.

The best advice is to experiment and find a technique for your problem that is fast and produces reasonable estimates of performance that you can use to make decisions. If in doubt, use 10-fold cross-validation.

5.13 Machine Learning Algorithm Performance Metrics

The metrics that you choose to evaluate your machine learning algorithms are very important. The choice of metrics influences how the performance of machine Learning algorithms are measured and compared. They influence how you weigh the importance of different characteristics in the results and your ultimate choice of which algorithm to choose. In this chapter you will discover how to select and use different machine learning performance metrics in Python with scikit-learn. Let's get started.

5.14 Algorithm Evaluation Metrics

In this lesson, various algorithm evaluation metrics are demonstrated for both classification and regression-type machine learning problems. For classification metrics, the Pima Indians onset of diabetes dataset is used as a demonstration. This is a binary classification problem where all of the input variables are numeric. For regression metrics, the Boston House Price dataset is used as a demonstration. this is a regression problem where all of the input variables are also numeric. All recipes evaluate the same algorithms, Logistic Regression for classification and Linear Regression for the regression problems. A 10-fold cross-validation test harness is used to demonstrate each metric because this is the most likely scenario you will use when employing different algorithm evaluation metrics.

A caveat in these recipes is the cross-validation. The `cross_val_score` function is used to report the performance of each recipe. It does allow the use of different scoring metrics that will be discussed, but all scores are reported so that they

can be sorted in ascending order (largest score is best). Some evaluation metrics (like mean squared error) are naturally descending scores (the smallest score is best) and as such are reported as negative by the cross validation. `Cross_val_score()` function.

This is important to note, because some scores will be reported as negative that can never be negative. I will remind you about this caveat as we work through the lesson. You can learn more about machine learning algorithm performance metrics supported by scikit-learn on the page [Model evaluation: quantifying the quality of predictions](#). Let's get on with the evaluation metrics.

5.15 Classification Metrics

Classification problems are perhaps the most common type of machine learning problem, and as such, there are a myriad of metrics that can be used to evaluate predictions for these problems.

In this section we will review how to use the following metrics:

- Classification Accuracy
- Logarithmic Loss
- Area Under ROC Curve
- Confusion Matrix
- Classification Report

5.15.1 Classification Accuracy

Classification accuracy is the number of correct predictions made as a ratio of all predictions made. This is the most common evaluation metric for classification

problems; it is also the most misused. It is really only suitable when there are an equal number of observations in each class (which is rarely the case) and that all predictions and prediction errors are equally important, which is often not the case. Below is an example of calculating classification accuracy.

```
# Cross Validation Classification Accuracy from pandas import read_csv

from sklearn.model_selection import KFold

from sklearn.model_selection import cross_val_score from sklearn.linear_model
import LogisticRegression filename = 'pima-indians-diabetes.data.csv'

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
read_csv(filename, names=names)

array = dataframe.values X = array[:,0:8]

Y = array[:,8]

kfold = KFold(n_splits=10, random_state=7) model = LogisticRegression()

scoring = 'accuracy'

results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)
print("Accuracy: %.3f (%3f)" % (results.mean(), results.std()))
```

Example of evaluating an algorithm by classification accuracy.

You can see that the ratio is reported. This can be converted into a percentage by multiplying the value by 100, giving an accuracy score of approximately 77% accurate.

Accuracy: 0.770 (0.048)

Output of evaluating an algorithm by classification accuracy.

5.15.2 Logarithmic Loss

Logarithmic loss (or logloss) is a performance metric for evaluating the predictions of probabilities of membership to a given class. The scalar probability between 0 and 1 can be seen as a measure of confidence for a prediction by an algorithm. Predictions that are correct or incorrect are rewarded or punished proportionally to the confidence of the prediction. Below is an example of calculating logloss for Logistic regression predictions on the Pima Indians onset of diabetes dataset.

```
# Cross Validation Classification LogLoss from pandas import read_csv

from sklearn.model_selection import KFold

from sklearn.model_selection import cross_val_score from sklearn.linear_model
import LogisticRegression filename = 'pima-indians-diabetes.data.csv'

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
read_csv(filename, names=names)

array = dataframe.values X = array[:,0:8]

Y = array[:,8]

kfold = KFold(n_splits=10, random_state=7) model = LogisticRegression()

scoring = 'neg_log_loss'
```

```
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring) print("Logloss:  
%.3f (%.3f)" % (results.mean(), results.std()))
```

Example of evaluating an algorithm by logloss

Smaller logloss is better, with 0 representing a perfect logloss. As mentioned above, the measure is inverted to be ascending when using the `cross_val_score()` function.

`Logloss: -0.493 (0.047)`

Output of evaluating an algorithm by logloss

5.15.3 Area Under ROC Curve

Area under the ROC Curve (or AUC for short) is a performance metric for binary classification problems. The AUC represents a model's ability to discriminate between positive and negative classes. An area of 1.0 represents a model that made all predictions perfectly. An area of 0.5 represents a model that is as good as random. ROC can be broken down into sensitivity and specificity. A binary classification problem is really a trade-off between sensitivity and specificity.

- Sensitivity is the true positive rate, also called the recall. It is the number of instances from the positive (first) class that were actually predicted correctly.
- Specificity is also called the true negative rate. Is the number of instances from the negative (second) class that were actually predicted correctly.

The example below provides a demonstration of calculating AUC.

Cross Validation Classification ROC AUC

```
from pandas import read_csv
```

```
from sklearn.model_selection import KFold
```

```
from sklearn.model_selection import cross_val_score from sklearn.linear_model
```

```
import LogisticRegression filename = 'pima-indians-diabetes.data.csv'
```

```
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =  
read_csv(filename, names=names)
```

```
array = dataframe.values X = array[:,0:8]
```

```
Y = array[:,8]
```

```
kfold = KFold(n_splits=10, random_state=7) model = LogisticRegression()
```

```
scoring = 'roc_auc'
```

```
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring) print("AUC:  
%.3f (%.3f)" % (results.mean(), results.std()))
```

Example of evaluating an algorithm by AUC

You can see the AUC is relatively close to 1 and greater than 0.5, suggesting some skill in the predictions

```
AUC: 0.824 (0.041)
```

Output of evaluating an algorithm by AUC.

```
# Cross Validation Classification Confusion Matrix from
```

```
pandas import read_csv
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression from
```

```
sklearn.metrics import confusion_matrix filename =
```

```
'pima-indians-diabetes.data.csv'
```

```
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
```

```
read_csv(filename, names=names)
```

```
array = dataframe.values X =
```

```
array[:,0:8]
```

```
Y = array[:,8] test_size =
```

```
0.33
```

```
seed = 7
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
```

```
test_size=test_size,
```

```
random_state=seed)
```

```
model = LogisticRegression() model.
```

```
fit(X_train, Y_train) predicted =
```

```
model.predict(X_test)
```

```
matrix = confusion_matrix(Y_test, predicted)
```

```
print(matrix)
```

Example of evaluating an algorithm by a confusion matrix.

Although the array is printed without headings, you can see that the majority of the predictions fall on the diagonal line of the matrix (which are correct predictions).

```
[[141 21]
 [ 41 51]]
```

Output of evaluating an algorithm by confusion matrix.

5.15.4 Classification Report

The scikit-learn library provides a convenience report when working on classification problems to give you a quick idea of the accuracy of a model using a number of measures. The `classification_report()` function displays the precision, recall, F1-score and support for each class. The example below demonstrates the report on the binary classification problem.

```
# Cross Validation Classification Report from
pandas import read_csv
from sklearn.model_selection import train_test_split from
sklearn.linear_model import LogisticRegression from
sklearn.metrics import classification_report filename =
'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8] test_size
    = 0.33
seed = 7
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
    test_size=test_size,
    random_state=seed)

model = LogisticRegression()
model.fit(X_train, Y_train)
predicted = model.predict(X_test)

report = classification_report(Y_test, predicted)
print(report)
```

Example of evaluating an algorithm by classification report.

You can see good predictions and recall for the algorithm.

	precision	recall	f1-score	support
0.0	0.81	0.88	0.84	162
1.0	0.74	0.63	0.68	92
accuracy			0.79	254
macro avg	0.78	0.75	0.76	254
weighted avg	0.78	0.79	0.78	254

Output of evaluating an algorithm by classification report.

5.16 Regression Metrics

In this section we will review 3 of the most common metrics for evaluating predictions on regression machine learning problems:

- Mean Absolute Error.
- Mean Squared Error.

- R2.

5.16.1 Mean Absolute Error

The Mean Absolute Error (or MAE) is the sum of the absolute differences between predictions and actual values. It gives an idea of how wrong the predictions were. The measure gives an idea of the magnitude of the error but no idea of the direction (e.g., over- or under-predicting). The example below demonstrates calculating the mean absolute error on the Boston house price dataset.

```
# Cross Validation Regression MAE from
pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
sklearn.linear_model import LinearRegression filename =
'housing.csv'
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
'PTRATIO',
'B', 'LSTAT', 'MEDV']
dataframe = read_csv(filename, delim_whitespace=True, names=names) array =
dataframe.values
X = array[:,0:13]
Y = array[:,13]
kfold = KFold(n_splits=10, random_state=7) model =
LinearRegression()
scoring = 'neg_mean_absolute_error'
```

```
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring) print("MAE: %.3f  
%.3f" % (results.mean(), results.std()))
```

Example of evaluating an algorithm by Mean Absolute Error.

A value of 0 indicates no error or perfect predictions. Like logloss, this metric is inverted by the `cross_val_score()` function.

```
MAE: -4.005 (2.084)
```

Output of evaluating an algorithm by Mean Absolute Error.

5.16.2 Mean Squared Error

The Mean Squared Error (or MSE) is much like the mean absolute error in that it provides a gross idea of the magnitude of error. Taking the square root of the mean squared error converts the units back to the original units of the output variable and can be meaningful for description and presentation. This is called the Root Mean Squared Error (or RMSE). The example below provides a demonstration of calculating mean squared error.

```
# Cross Validation Regression MSE from  
pandas import read_csv  
from sklearn.model_selection import KFold  
from sklearn.model_selection import cross_val_score from  
sklearn.linear_model import LinearRegression filename =  
'housing.csv'
```

```
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',  
         'PTRATIO',  
         'B', 'LSTAT', 'MEDV']  
  
dataframe = read_csv(filename, delim_whitespace=True, names=names) array =  
    dataframe.values  
  
X = array[:,0:13]  
  
Y = array[:,13]  
  
    num_folds = 10  
  
kfold = KFold(n_splits=10, random_state=7) model =  
    LinearRegression()  
  
scoring = 'neg_mean_squared_error'  
  
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring) print("MSE: %.3f  
    (%.3f)" % (results.mean(), results.std()))
```

Example of evaluating an algorithm by Mean Squared Error.

This metric too is inverted so that the results are increasing. Remember to take the absolute value before taking the square root if you are interested in calculating the RMSE.

```
MSE: -34.705 (45.574)
```

Output of evaluating an algorithm by Mean Squared Error.

5.16.3 R^2 Metric

The R^2 (or R Squared) metric provides an indication of the goodness of fit of a set of predictions to the actual values. In statistical literature this measure is called

the coefficient of determination. This is a value between 0 and 1 for no fit and perfect fit, respectively. The example below provides a demonstration of calculating the mean R^2 for a set of predictions.

Example of evaluating an algorithm by R Squared.

You can see the predictions have a poor fit to the actual values, with a value closer to zero and less than 0.5.

R^2 : 0.203 (0.595)

Output of evaluating an algorithm by R Squared.

In-Text Questions and Answers (ITQs and ITAs)

ITQ 5.1: What role does data preparation play in machine learning, and how can it affect the performance of machine learning models?

ITA: Data preparation is critical, as it ensures that the data is in the right format and scale for machine learning algorithms to perform optimally. Poorly prepared data can lead to suboptimal model performance, as many algorithms make assumptions about the data, such as the scale of attributes or the distribution of features. Proper preparation can enhance accuracy, speed, and efficiency.

ITQ 5.2: What is the purpose of rescaling data, and which machine learning algorithms benefit from it?

ITA: Rescaling data ensures that all features have the same scale, typically between 0 and 1. Algorithms that rely on distance measures, such as k-Nearest Neighbours and gradient descent-based methods, benefit from rescaling

because it prevents attributes with larger ranges from dominating the model's learning process.

Conclusion

In conclusion, this session has been a comprehensive exploration of key aspects in the realm of machine learning. We started by delving into the critical realm of data preparation, understanding how it sets the foundation for successful model training. The techniques of rescaling, standardising, normalising, and binarising data were expertly covered, ensuring data compatibility with machine learning algorithms. Moving on to feature selection, we learnt how to enhance model performance by selecting pertinent attributes and leveraging techniques like Univariate Selection, Recursive Feature Elimination, PCA, and Feature Importance. Finally, we ventured into performance evaluation, uncovering methods such as cross-validation, train/test splits, and a range of classification and regression metrics. This comprehensive journey equips us with the tools to not only build models but also refine and optimise them, making informed decisions and achieving accuracy and efficacy in our machine learning endeavours.

Summary

1. In this study session, you have learnt about the critical process of data preparation for machine learning, which lays the foundation for successful model training. Machine learning algorithms rely on well-prepared data to perform effectively. This chapter focuses on data preparation using scikit-learn

2. in Python. Techniques such as rescaling, standardising, normalising, and binarising data have been explored with hands-on examples using the Pima Indian diabetes dataset. The goal is to optimise the data structure to enhance compatibility with machine learning algorithms.
3. Moving forward, we delve into feature selection for machine learning, a pivotal step in enhancing model performance by identifying and using the most relevant data attributes. Irrelevant features can be detrimental to model efficacy, so techniques like Univariate Selection, Recursive Feature Elimination, PCA, and Feature Importance will be covered using scikit-learn. Practical examples with the Pima Indian diabetes dataset will illustrate how to implement these techniques effectively.
4. Finally, we explore techniques and metrics for evaluating machine learning algorithm performance. Cross-validation, train/test splits, and a range of classification and regression metrics (e.g., accuracy, logarithmic loss, ROC curve, confusion matrix, mean absolute error, mean squared error, and R-squared) will be discussed. These evaluation methods empower you to make informed decisions about algorithm effectiveness.
5. Summarily, these study sessions have equipped you with essential knowledge and practical skills for data preparation, feature selection, and performance evaluation in the realm of machine learning.

Self-Assessments Questions and Answers (SAQs and SAAs)

SAQ 1: What is the primary purpose of data preparation in the context of machine learning?

Answer: To enhance the compatibility between data and machine learning algorithms. Data preparation ensures that the data is in a suitable format, without missing values or inconsistencies, and is ready to be used by machine learning algorithms.

SAQ 2: Which technique can be used to estimate a machine learning model's performance on new, unseen data?

Answer: Cross-validation is a technique where the dataset is divided into subsets for training and testing, allowing multiple estimates of a model's performance on unseen data.

SAQ 3: Why is PCA considered a feature selection technique, and how does it reduce the dimensionality of data?

Answer: Principal Component Analysis (PCA) transforms the original data into a set of linearly uncorrelated components, each representing a combination of the original features. By selecting the top principal components, PCA reduces the dimensionality of the data while retaining most of the variance, helping to simplify the dataset and improve model performance.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. What is the main benefit of data standardisation in machine learning?

- A) It transforms data to a range between 0 and 1.
- B) It adjusts the data to have a mean of 0 and a standard deviation of 1.
- C) It scales the data into a unit norm.
- D) It removes irrelevant features.

Answer: B) It adjusts the data to have a mean of 0 and a standard deviation of 1.

Explanation: Standardisation ensures that the data has a mean of 0 and a standard deviation of 1, making it suitable for algorithms that assume Gaussian distributions.

2. Which feature selection method recursively eliminates the least important features?

- A) Univariate Selection
- B) Principal Component Analysis
- C) Recursive Feature Elimination
- D) Feature Importance

Answer: C) Recursive Feature Elimination

Explanation: Recursive Feature Elimination (RFE) removes the least important features iteratively, using model accuracy to identify which features contribute the most to prediction.

3. **What does PCA (Principal Component Analysis) do in the context of feature selection?**

- A) It eliminates irrelevant features by statistical tests.
- B) It selects the most important features based on feature importance scores.
- C) It transforms the data into a smaller set of components, retaining the most variance.
- D) It groups similar features into a single category.

Answer: C) It transforms the data into a smaller set of components, retaining the most variance.

Explanation: PCA reduces the dimensionality of the data by transforming the original features into a set of uncorrelated principal components, retaining the most important variance in the data.

4. **Why is feature selection important in machine learning?**

- A) It reduces the amount of data available for training the model.
- B) It increases the complexity of the model, improving accuracy.
- C) It helps to remove irrelevant or redundant features, improving model efficiency.
- D) It eliminates the need for data preprocessing.

Answer: C) It helps to remove irrelevant or redundant features, improving model efficiency.

Explanation: Feature selection reduces the number of input variables,

which can help avoid overfitting, reduce training time, and improve model performance by focusing on the most relevant data.

5. **Which of the following techniques is most suitable for algorithms that assume a Gaussian distribution in the input variables?**

- A) Rescaling
- B) Standardisation
- C) Normalisation
- D) Binarisation

Answer: B) Standardisation

Explanation: Standardisation transforms data to have a mean of 0 and a standard deviation of 1, making it suitable for algorithms that assume Gaussian distributions.

Tutor-Marked Assessments (TMAs)

1. Explain how feature selection techniques enhance model performance and streamline the feature set.
2. What is the rationale behind Univariate Selection, and how does it work?
3. How do you determine the importance of features in your dataset? Discuss the methods used for feature importance estimation, such as decision tree-based algorithms, and explain how they help in model improvement.
4. Interpret the role of performance metrics such as classification accuracy, logarithmic loss, and AUC. Use the Pima Indians diabetes dataset to explain how these metrics help assess model performance.

STUDY SESSION

SIX

SPOT-CHECK CLASSIFICATION ALGORITHMS

Introduction

In this session, we will explore the process of spot-checking classification algorithms to determine their performance on a machine learning problem. Spot-checking involves evaluating various algorithms to identify the most promising ones for further optimisation. We cannot know in advance which algorithms will work best for our dataset, so trial and error is necessary. This chapter introduces six classification algorithms that we will evaluate using Python's scikit-learn library. These algorithms include both linear and nonlinear approaches. The goal is to identify algorithms that perform well in classifying data and picking out the underlying structure. Let's begin the journey of discovering and comparing these machine learning algorithms to enhance our classification tasks.

Learning Outcomes for Study Session Six

When you have studied this session, you should be able to:

- 6.1. Explore the concept of spot-checking classification algorithms and their importance in discovering suitable algorithms for your machine learning problem.
- 6.2. Identify that spot-checking involves trying out various algorithms and focusing on the most promising ones based on their performance.
- 6.3. Recognise the distinction between knowing which algorithm to use and knowing which algorithms to spot-check on your dataset.

- 6.4. Implement a mixture of algorithm representations and learning algorithms when spot-checking.
- 6.5. Distinguish between linear and nonlinear machine learning algorithms and their characteristics.
- 6.6. Demonstrates minimal recipes for using linear regression algorithms.
- 6.7. Demonstrates minimal recipes for using three nonlinear regression algorithms.
- 6.8. Compare Machine Learning Algorithms
- 6.9. Automate Machine Learning

6.1 Spot-Check Classification Algorithms

Spot-checking is a way of discovering which algorithms perform well on your machine learning problem. You cannot know which algorithms are best suited to your problem beforehand. You must trial a number of methods and focus attention on those that prove themselves to be the most promising. In this chapter, you will discover six machine learning algorithms that you can use when spot-checking your classification problem in Python with scikit-learn. After completing this lesson, you will know:

- How to spot-check machine learning algorithms on a classification problem.
- How to spot-check two linear classification algorithms.
- How to spot-check four nonlinear classification algorithms. Let's get started.

6.2 Algorithm Spot-Checking

You cannot know which algorithm will work best on your dataset beforehand. You must use trial and error to discover a shortlist of algorithms that do well on your problem, which you can then double down on and tune further. I call this process spot-checking. The question is not:

What algorithm should I use on my dataset? Instead, it is:

What algorithms should I spot-check on my dataset?

You can guess at what algorithms might do well on your dataset, and this can be a good starting point. I recommend trying a mixture of algorithms to see what is

good at picking out the structure in your data. Below are some suggestions for spot-checking algorithms on your dataset:

- Try a mixture of algorithm representations (e.g. instances and trees).
- Try a mixture of learning algorithms (e.g., different algorithms for learning the same type of representation).
- Try a mixture of modelling types (e.g. linear and nonlinear functions or parametric and nonparametric).

In the next section, we will look at algorithms that you can use to check on your next classification machine learning project in Python.

6.3 Algorithms Overview

We are going to take a look at six classification algorithms that you can spot-check on your dataset. Starting with two linear machine learning algorithms:

- Logistic Regression.
- Linear Discriminant Analysis.

Then, looking at four nonlinear machine learning algorithms:

- k-Nearest Neighbours.
- Naive Bayes.
- Classification and Regression Trees.
- Support Vector Machines.

Each recipe is demonstrated on the Pima Indians' onset of Diabetes dataset. A test harness using 10-fold cross-validation is used to demonstrate how to spot-check each machine learning algorithm, and mean accuracy measures are used to indicate algorithm performance. The recipes assume that you know about

each machine learning algorithm and how to use it. We will not go into the API or parameterisation of each algorithm.

6.4 Linear Machine Learning Algorithms

This section demonstrates minimal recipes for how to use two linear machine learning algorithms: logistic regression and linear discriminant analysis.

6.4.1 Logistic Regression

Logistic regression assumes a Gaussian distribution for the numeric input variables and can model binary classification problems. You can construct a logistic regression model using the Logistic Regression class.

```
# Logistic Regression Classification from  
pandas import read_csv
```

```
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.linear_model import LogisticRegression filename
    = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8] num_folds
    = 10
kfold = KFold(n_splits=10, random_state=7) model =
    LogisticRegression()
results = cross_val_score(model, X, Y, cv=kfold)
    print(results.mean())
```

Example of the logistic regression algorithm.

Running the example prints the mean estimated accuracy.

```
0.76951469583
```

Output of the logistic regression algorithm.

6.4.2 Linear Discriminant Analysis

Linear Discriminant Analysis or LDA is a statistical technique for binary and multiclass classification. It too assumes a Gaussian distribution for the numerical input variables. You can construct an LDA model using the Linear Discriminant Analysis class.

```
# LDA Classification
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis filename =
    'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8] num_folds
    = 10
kfold = KFold(n_splits=10, random_state=7) model =
    LinearDiscriminantAnalysis()
results = cross_val_score(model, X, Y, cv=kfold)
    print(results.mean())
```

Example of the LDA algorithm.

Running the example prints the mean estimated accuracy.

```
0.773462064252
```

Output of the LDA algorithm.

6.5 Nonlinear Machine Learning Algorithms

This section demonstrates minimal recipes for how to use 4 nonlinear machine learning algorithms.

6.5.1 k-Nearest Neighbours

The k-Nearest Neighbours algorithm (or KNN) uses a distance metric to find the k most similar instances in the training data for a new instance and takes the mean outcome of the neighbours as the prediction. You can construct a KNN model using the KNeighborsClassifier class.

```
# KNN Classification
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.neighbors import KNeighborsClassifier filename
    = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8] num_folds
    = 10
kfold = KFold(n_splits=10, random_state=7) model =
    KNeighborsClassifier()
results = cross_val_score(model, X, Y, cv=kfold)
    print(results.mean())
```

Example of the KNN algorithm.

Running the example prints the mean estimated accuracy.

```
0.726555023923
```

Output of the KNN algorithm.

6.5.2 Naive Bayes

Naive Bayes calculates the probability of each class and the conditional probability of each class given each input value. These probabilities are estimated for new data and multiplied together, assuming that they are all independent (a simple or naive assumption). When working with real-valued data, a Gaussian distribution is assumed to easily estimate the probabilities for input variables using the Gaussian Probability Density Function. You can construct a Naïve Bayes model using the GaussianNB class.

```
# Gaussian Naive Bayes Classification from pandas import read_csv

from sklearn.model_selection import KFold

from sklearn.model_selection import cross_val_score from sklearn.naive_bayes
import GaussianNB filename = 'pima-indians-diabetes.data.csv'

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
read_csv(filename, names=names)

array = dataframe.values X = array[:,0:8]

Y = array[:,8]

kfold = KFold(n_splits=10, random_state=7) model = GaussianNB()

results = cross_val_score(model, X, Y, cv=kfold)

print(results.mean())
```

Example of the Naive Bayes algorithm.

Running the example prints the mean estimated accuracy.

```
0.75517771702
```

Output of the Naive Bayes algorithm.

6.5.3 Classification and Regression Trees

Classification and Regression Trees (CART or just decision trees) construct a binary tree from the training data. Split points are chosen greedily by evaluating each attribute and each value of each attribute in the training data in order to minimise a cost function (like the Gini index). You can construct a CART model using the `DecisionTreeClassifier` class.

```
# CART Classification
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.tree import DecisionTreeClassifier filename =
    'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
```

```
Y = array[:,8]
kfold = KFold(n_splits=10, random_state=7) model =
    DecisionTreeClassifier()
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Example of the CART algorithm.

Running the example prints the mean estimated accuracy.

```
0.692600820232
```

Output of the CART algorithm.

6.5.4 Support Vector Machines

Support Vector Machines (or SVM) seek a line that best separates two classes. Those data instances that are closest to the line that best separates the classes are called support vectors and influence where the line is placed. SVM has been extended to support multiple classes. Of particular importance is the use of different kernel functions via the kernel parameter. Powerful Radial Basis Function is used by default. You can construct an SVM model using the SVC class.

```
# SVM Classification
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
```

```
from sklearn.svm import SVC
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8]
kfold = KFold(n_splits=10, random_state=7) model =
    SVC()
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Example of the SVM algorithm.

Running the example prints the mean estimated accuracy.

```
0.651025290499
```

Output of the SVM algorithm.

6.6 Spot-Check Regression Algorithms

Spot-checking is a way of discovering which algorithms perform well on your machine learning problem. You cannot know which algorithms are best suited to your problem beforehand. You must trial a number of methods and focus attention on those that prove themselves to be the most promising. In this chapter, you will discover six machine learning algorithms that you can use when

spot-checking your regression problem in Python with scikit-learn. After completing this lesson, you will know:

1. How to spot-check machine learning algorithms on a regression problem.
2. How to spot-check four linear regression algorithms.
3. How to spot-check three nonlinear regression algorithms.

6.7 Algorithms Overview

In this lesson, we are going to take a look at seven regression algorithms that you can spot-check on your dataset. Starting with four linear machine learning algorithms:

- Linear Regression.
- Ridge Regression.
- LASSO Linear Regression.
- Elastic Net Regression.

Then, looking at three nonlinear machine learning algorithms:

1. k-Nearest Neighbours.
2. Classification and Regression Trees.
3. Support Vector Machines.

Each recipe is demonstrated on the Boston House Price dataset. This is a regression problem where all attributes are numeric. A test harness with 10-fold cross-validation is used to demonstrate how to spot-check each machine learning algorithm and mean squared error measures are used to indicate algorithm performance. Note that mean squared error values are inverted

(negative). This is a quirk of the `cross_val_score()` function used that requires all algorithm metrics to be sorted in ascending order (larger value is better). The recipes assume that you know about each machine learning algorithm and how to use them. We will not go into the API or parameterisation of each algorithm.

6.8 Linear Machine Learning Algorithms

This section provides examples of how to use four different linear machine learning algorithms for regression in Python with scikit-learn.

6.8.1 Linear Regression

Linear regression assumes that the input variables have a Gaussian distribution. It is also assumed that input variables are relevant to the output variable and that they are not highly correlated with each other (a problem called collinearity). You can construct a linear regression model using the linear regression class.

```
# Linear Regression
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.linear_model import LinearRegression filename
    = 'housing.csv'
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
        'PTRATIO',
        'B', 'LSTAT', 'MEDV']
dataframe = read_csv(filename, delim_whitespace=True, names=names) array
    = dataframe.values
X = array[:,0:13]
```

```
Y = array[:,13]
kfold = KFold(n_splits=10, random_state=7) model =
    LinearRegression()
scoring = 'neg_mean_squared_error'
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)
print(results.mean())
```

Example of the linear regression algorithm.

Running the example provides an estimate of mean squared error.

```
-34.7052559445
```

Output of the linear regression algorithm.

6.8.2 Ridge Regression

Ridge regression is an extension of linear regression where the loss function is modified to minimise the complexity of the model, measured as the sum squared value of the coefficient values (also called the L2-norm). You can construct a ridge regression model by using the Ridge class.

```
# Ridge Regression
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.linear_model import Ridge
filename = 'housing.csv'
```

```
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',  
         'PTRATIO',  
         'B', 'LSTAT', 'MEDV']  
  
dataframe = read_csv(filename, delim_whitespace=True, names=names) array =  
    dataframe.values  
  
X = array[:,0:13]  
  
Y = array[:,13]  
  
kfold = KFold(n_splits=10, random_state=7) model =  
    Ridge()  
  
scoring = 'neg_mean_squared_error'  
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)  
print(results.mean())
```

Example of the ridge regression algorithm.

Running the example provides an estimate of the mean squared error.

```
-34.0782462093
```

Output of the ridge regression algorithm.

6.8.3 LASSO Regression

The Least Absolute Shrinkage and Selection Operator (or LASSO for short) is a modification of linear regression, like ridge regression, where the loss function is modified to minimise the complexity of the model measured as the sum absolute value of the coefficient values (also called the L1-norm). You can construct a LASSO model by using the Lasso class

```
# Lasso Regression
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.linear_model import Lasso
filename = 'housing.csv'
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
        'PTRATIO',
        'B', 'LSTAT', 'MEDV']
dataframe = read_csv(filename, delim_whitespace=True, names=names) array =
    dataframe.values
X = array[:,0:13]
Y = array[:,13]
kfold = KFold(n_splits=10, random_state=7) model =
    Lasso()
scoring = 'neg_mean_squared_error'
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)
    print(results.mean())
```

Example of the LASSO regression algorithm.

Running the example provides an estimate of the mean squared error.

```
-34.4640845883
```

Output of the LASSO regression algorithm.

6.8.4 ElasticNet Regression

ElasticNet is a form of regularisation regression that combines the properties of both Ridge Regression and LASSO regression. It seeks to minimise the complexity of the regression model (magnitude and number of regression coefficients) by penalising the model using both the L2-norm (sum of squared coefficient values) and the L1-norm (sum of absolute coefficient values).

You can construct an ElasticNet model using the ElasticNet class.

```
# ElasticNet Regression from
    pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.linear_model import ElasticNet
filename = 'housing.csv'
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
        'PTRATIO',
        'B', 'LSTAT', 'MEDV']
dataframe = read_csv(filename, delim_whitespace=True, names=names) array
    = dataframe.values
X = array[:,0:13]
Y = array[:,13]
kfold = KFold(n_splits=10, random_state=7) model =
    ElasticNet()
scoring = 'neg_mean_squared_error'
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)
```

```
print(results.mean())
```

Example of the ElasticNet regression algorithm.

Running the example provides an estimate of the mean squared error.

```
-34.4640845883
```

Output of the ElasticNet regression algorithm.

6.9 Nonlinear Machine Learning Algorithms

This section provides examples of how to use three different nonlinear machine learning algorithms for regression in Python with scikit-learn.

6.9.1 K-Nearest Neighbours

The k-Nearest Neighbours algorithm (or KNN) locates the k most similar instances in the training dataset for a new data instance. From the k neighbours, a mean or median output variable is taken as the prediction. Of note is the distance metric used (the metric argument). The Minkowski distance is used by default, which is a generalisation of both the Euclidean distance (used when all inputs have the same scale) and the Manhattan distance (used when the scales of the input variables differ). You can construct a KNN model for regression using the KNeighborsRegressor class.

```
# KNN Regression
from pandas import read_csv
from sklearn.model_selection import KFold
```

```
from sklearn.model_selection import cross_val_score from
sklearn.neighbors import KNeighborsRegressor filename
= 'housing.csv'
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
        'PTRATIO',
        'B', 'LSTAT', 'MEDV']
dataframe = read_csv(filename, delim_whitespace=True, names=names) array =
        dataframe.values
X = array[:,0:13]
Y = array[:,13]
kfold = KFold(n_splits=10, random_state=7) model =
        KNeighborsRegressor()
scoring = 'neg_mean_squared_error'
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)
print(results.mean())
```

Example of the KNN regression algorithm.

Running the example provides an estimate of the mean squared error.

```
-107.28683898
```

Output of the KNN regression algorithm.

6.9.2 Classification and Regression Trees

Decision trees, or Classification and Regression Trees (CART, as they are known), use the training data to select the best points to split the data in order to

minimise a cost metric. The default cost metric for regression decision trees is the mean squared error, specified in the criterion parameter. You can create a CART model for regression using the `DecisionTreeRegressor` class.

Example of the CART regression algorithm.

Running the example provides an estimate of the mean squared error.

```
-35.4906027451
```

Output of the CART regression algorithm.

6.9.3 Support Vector Machines

Support Vector Machines (SVM) were developed for binary classification. The technique has been extended for the prediction of real-valued problems called Support Vector Regression (SVR). Like the classification example, SVR is built upon the LIBSVM library. You can create an SVM model for regression using the `SVR` class.

```
# SVM Regression
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.svm import SVR
filename = 'housing.csv'
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
        'PTRATIO',
        'B', 'LSTAT', 'MEDV']
dataframe = read_csv(filename, delim_whitespace=True, names=names) array =
    dataframe.values
X = array[:,0:13]
Y = array[:,13]
kfold = KFold(n_splits=10, random_state=7) model =
    SVR()
scoring = 'neg_mean_squared_error'
results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring)
```

```
print(results.mean())
```

Example of the SVM regression algorithm.

Running the example provides an estimate of the mean squared error.

```
-91.0478243332
```

Output of the SVM regression algorithm.

6.10 Compare Machine Learning Algorithms

It is important to compare the performance of multiple different machine learning algorithms consistently. In this chapter, you will discover how you can create a test harness to compare multiple different machine learning algorithms in Python with scikit-learn. You can use this test harness as a template for your own machine learning problems and add more and different algorithms to compare.

After completing this lesson, you will know:

- How to formulate an experiment to directly compare machine learning algorithms.
- A reusable template for evaluating the performance of multiple algorithms on one dataset.
- How to report and visualise the results when comparing algorithm performance.

6.11 Choose The Best Machine Learning Model

When you work on a machine learning project, you often end up with multiple good models to choose from. Each model will have different performance

characteristics. Using resampling methods like cross-validation, you can get an estimate for how accurate each model may be on unseen data. You need to be able to use these estimates to choose one or two best models from the suite of models that you have created.

When you have a new dataset, it is a good idea to visualise the data using different techniques in order to look at the data from different perspectives. The same idea applies to model selection. You should use a number of different ways of looking at the estimated accuracy of your machine learning algorithms in order to choose the one or two algorithms to finalise. A way to do this is to use visualisation methods to show the average accuracy, variance, and other properties of the distribution of model accuracies. In the next section you will discover exactly how you can do that in Python with scikit-learn.

6.12 Compare Machine Learning Algorithms Consistently

The key to a fair comparison of machine learning algorithms is ensuring that each algorithm is evaluated in the same way on the same data. You can achieve this by forcing each algorithm to be evaluated on a consistent test harness. In the example below, six different classification algorithms are compared on a single dataset:

- Logistic Regression.
- Linear Discriminant Analysis.
- k-Nearest Neighbours.
- classification and Regression Trees.
- Naive Bayes.

- Support Vector Machines.

The dataset is the Pima Indians' onset of diabetes problem. The problem has two classes and eight numeric input variables of varying scales. The 10-fold cross-validation procedure is used to evaluate each algorithm, and it is important to configure it with the same random seed to ensure that the same splits to the training data are performed and that each algorithm is evaluated in precisely the same way. Each algorithm is given a short name, useful for summarising results afterward.

Compare Algorithms

```
from pandas import read_csv from matplotlib import pyplot
from sklearn. model_selection import KFold
from sklearn.model_selection import cross_val_score from sklearn.linear_model
import LogisticRegression from sklearn. tree import DecisionTreeClassifier
from sklearn. neighbors import KNeighbors Classifier
from sklearn. discriminant_analysis import LinearDiscriminantAnalysis from
sklearn. naive_bayes import GaussianNB
from sklearn.svm import SVC # load dataset
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
read_csv(filename, names=names)
array = dataframe.values X = array[:,0:8]
Y = array[:,8]
# prepare models models = []
```

```
models.append(('LR', LogisticRegression())) models.append(('LDA',  
LinearDiscriminantAnalysis())) models.append(('KNN', KNeighborsClassifier()))  
append(('CART', DecisionTreeClassifier())) models.append(('NB',  
GaussianNB())) models.append(('SVM', SVC()))  
# evaluate each model in turn. results = []  
names = []  
scoring = 'accuracy'  
for name, model in models:  
    kfold = KFold(n_splits=10, random_state=7)  
    cv_results = cross_val_score(model, X, Y, cv=kfold, scoring=scoring) results.  
    append(cv_results)  
    Names.append(name)  
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
```

```
print(msg)  
# boxplot algorithm comparison fig =  
    pyplot.figure()  
fig.suptitle('Algorithm Comparison') ax =  
    fig.add_subplot(111)  
    pyplot.boxplot(results)  
    ax.set_xticklabels(names)  
    pyplot.show()
```

Example of comparing multiple algorithms.

Running the example provides a list of each algorithm short name, the mean accuracy, and the standard deviation accuracy.

```
LR: 0.769515 (0.048411)
LDA: 0.773462 (0.051592)
KNN: 0.726555 (0.061821)
CART: 0.695232 (0.062517)
NB: 0.755178 (0.042766)
SVM: 0.651025 (0.072141)
```

Output of comparing multiple algorithms.

The example also provides a box and whisker plot showing the spread of the accuracy scores across each cross-validation fold for each algorithm.

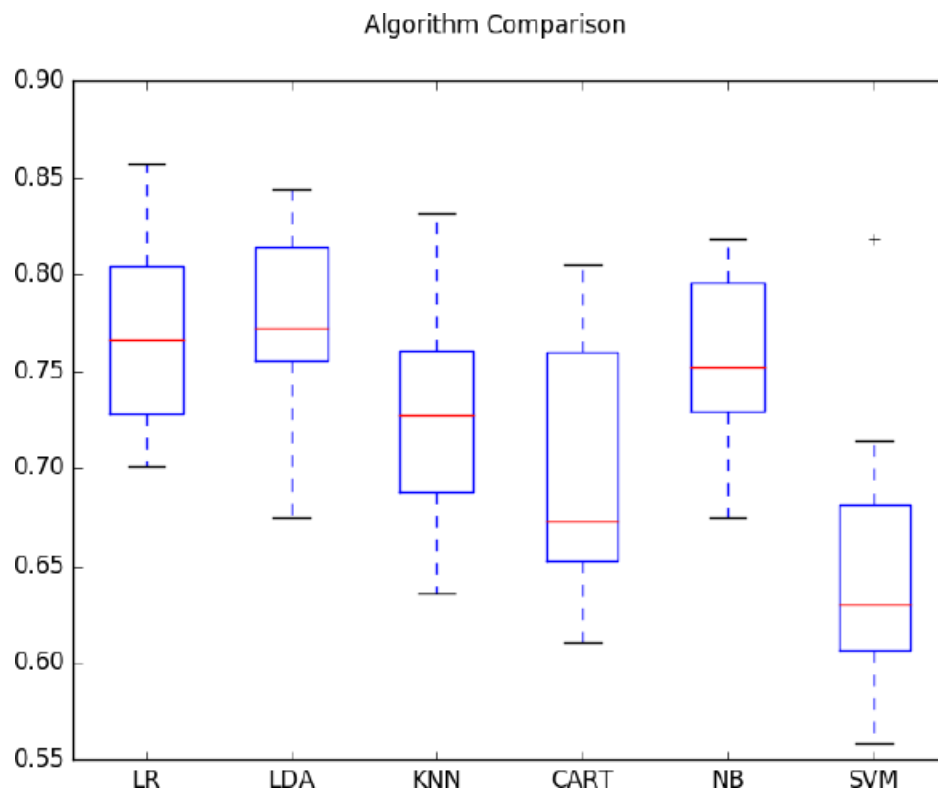


Figure 6.1: Algorithm comparison

Box and Whisker Plots Comparing Algorithm Performance

From these results, it would suggest that both logistic regression and linear discriminant analysis are perhaps worthy of further study on this problem.

6.13 Automate Machine Learning

6.13.1 Workflows with Pipelines

There are standard workflows in a machine learning project that can be automated. In Python scikit-learn, Pipelines help to clearly define and automate these workflows. In this chapter, you will discover Pipelines in scikit-learn and how you can automate common machine learning workflows. After completing this lesson, you will know:

- How to use pipelines to minimise data leakage.
- How to construct a data preparation and modelling pipeline.
- How to construct a feature extraction and modelling pipeline.

6.13.2 Automating Machine Learning Workflows

There are standard workflows in applied machine learning. Standard because they overcome common problems like data leakage in your test harness. Python's scikit-learn provides a Pipeline utility to help automate machine learning workflows. Pipelines work by allowing for a linear sequence of data transformations to be chained together, culminating in a modelling process that can be evaluated. The goal is to ensure that all of the steps in the pipeline are constrained by the data available for evaluation, such as the training dataset or each fold of the cross-validation procedure. You can learn more about Pipelines in scikit-learn by reading Pipeline Section 1 of the user guide. You can also review

the API documentation for the Pipeline and FeatureUnion classes and the pipeline module.

6.13.3 Data Preparation and Modelling Pipeline

An easy trap to fall into in applied machine learning is leaking data from your training dataset to your test dataset. To avoid this trap, you need a robust test harness with a strong separation of training and testing. This includes data preparation. Data preparation is one easy way to leak knowledge of the whole training dataset to the algorithm. For example, preparing your data using normalisation or standardisation on the entire training dataset before learning would not be a valid test because the training dataset would have been influenced by the scale of the data in the test set.

Pipelines help you prevent data leakage in your test harness by ensuring that data preparation, like standardisation, is constrained to each fold of your cross-validation procedure. The example below demonstrates this important data preparation and model evaluation workflow on the Pima Indians onset of diabetes dataset. The pipeline is defined in two steps:

- Standardise the data.
- Learn a Linear Discriminant Analysis model.

The pipeline is then evaluated using 10-fold cross-validation.

```
# Create a pipeline that standardizes the data then creates a model from
pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
sklearn.preprocessing import StandardScaler from
sklearn.pipeline import Pipeline
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis # load
data
```

```
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe
= read_csv(filename, names=names)
array = dataframe.values X =
array[:,0:8]
Y = array[:,8]
# create pipeline estimators =
[]
estimators.append(('standardize', StandardScaler()))
estimators.append(('lda', LinearDiscriminantAnalysis())) model =
Pipeline(estimators)
# evaluate pipeline
kfold = KFold(n_splits=10, random_state=7) results =
cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Example of a Pipeline to standardise and model data.

Notice how we create a Python list of steps that are provided to the Pipeline for processing the data. Also notice how the Pipeline itself is treated like an estimator and is evaluated in its entirety by the k-fold cross-validation procedure. Running the example provides a summary of the accuracy of the setup on the dataset.

```
0.773462064252
```

Output of a Pipeline to standardise and model data.

6.13.4 Feature Extraction and Modelling Pipeline

Feature extraction is another procedure that is susceptible to data leakage. Like data preparation, feature extraction procedures must be restricted to the data in your training dataset. The pipeline provides a handy tool called the Feature Union, which allows the results of multiple feature selection and extraction procedures to be combined into a larger dataset on which a model can be trained. Importantly, all the feature extraction and feature union occur within each fold of the cross-validation procedure. The example below demonstrates the pipeline defined by four steps:

- Feature Extraction with Principal Component Analysis (3 features).
- Feature Extraction with Statistical Selection (6 features).
- Feature Union.
- Learn a Logistic Regression Model.

The pipeline is then evaluated using 10-fold cross-validation.

```
# Create a pipeline that extracts features from the data then creates a model
```

```
from pandas import read_csv

from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
    sklearn.pipeline import Pipeline
from sklearn.pipeline import FeatureUnion
from sklearn.linear_model import LogisticRegression from
    sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest # load
    data
```

```
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe
    = read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8]

# create feature union features
    = []
features.append(('pca', PCA(n_components=3)))
    features.append(('select_best', SelectKBest(k=6)))
    feature_union = FeatureUnion(features)

# create pipeline estimators =
    []
estimators.append(('feature_union', feature_union))
    estimators.append(('logistic', LogisticRegression()))
    model = Pipeline(estimators)

# evaluate pipeline
kfold = KFold(n_splits=10, random_state=7) results =
    cross_val_score(model, X, Y, cv=kfold)

    print(results.mean())
```

Example of a Pipeline extract and combines features before modelling.

Notice how the FeatureUnion is its own Pipeline that in turn is a single step in the final Pipeline used to feed Logistic Regression. This might get you thinking about how you can start embedding pipelines within pipelines. Running the example provides a summary of the accuracy of the setup on the dataset.

Output of a Pipeline extract and combines features before modelling.

In-Text Questions and Answers (ITQs and ITAs)

ITQ 6.1: What is spot-checking in machine learning, and why is it important?

ITA: Spot-checking is the process of trial and error in which you evaluate multiple machine learning algorithms to identify the most promising ones for your problem. It is important because it helps determine which algorithms work best for your dataset, allowing you to focus on those that perform well before further optimisation.

ITQ 6.2: How do linear and nonlinear machine learning algorithms differ?

ITA: Linear algorithms assume a straight-line relationship between inputs and outputs, making them suitable for problems where the data is linearly separable. Nonlinear algorithms, on the other hand, can capture more complex relationships and can model interactions between variables that are not represented by a straight line.

Conclusion

In conclusion, this chapter provided a comprehensive overview of spot-checking classification algorithms as a means to determine their performance on a given machine learning problem. By utilising techniques like logistic regression, linear discriminant analysis, k-nearest neighbours, classification and regression trees, naive Bayes, and support vector machines: the chapter demonstrated how to evaluate and compare these algorithms using cross-validation. The importance of data preparation, model selection, and visualisation in the context of algorithm comparison was emphasised. Additionally, the chapter introduced the concept of pipelines, which automate workflows and help prevent data leakage by ensuring that data transformations are confined to each fold of cross-validation. Through these insights, readers gained a deeper understanding of how to systematically select the most suitable machine learning algorithms for their classification tasks while maintaining good practices for reliable evaluations.

Summary

1. In this study session, you have learnt to apply spot-checking techniques to evaluate and compare different machine learning algorithms for classification and regression tasks. The process of spot-checking involves experimenting with various algorithms to identify those that perform well on your specific problem. The session covered both linear and nonlinear algorithms for classification and regression tasks, using Python's scikit-learn library.

2. For classification, you learnt about spot-checking six machine learning algorithms: Logistic Regression, Linear Discriminant Analysis, k-Nearest neighbours, Naive Bayes, classification and Regression Trees (CART), and Support Vector Machines (SVM). Each algorithm was demonstrated using code examples, including data loading, model creation, cross-validation, and performance evaluation.
3. Similarly, for regression tasks, you explored spot-checking four linear algorithms: Linear Regression, Ridge Regression, LASSO Regression, and elastic net Regression, as well as three nonlinear algorithms: k-Nearest Neighbours, Decision Trees, and Support Vector Machines. The steps to create, evaluate, and compare models using cross-validation were shown for each algorithm.
4. To ensure fair comparisons and avoid data leakage, the session introduced the concept of pipelines in scikit-learn. Pipelines allow you to chain together data transformation and modelling steps while maintaining data integrity during cross-validation. Two types of pipelines were covered: one for data preparation and modelling and another for feature extraction and modelling. Each step within a pipeline is constrained by the training data, ensuring accurate evaluations.
5. Furthermore, the session delved into comparing machine learning algorithms consistently using box and whisker plots, providing a visual representation of each algorithm's performance across different cross-validation folds.

6. In summary, this study session provided a comprehensive overview of spot-checking techniques, algorithm comparisons, and the use of pipelines for proper data preparation and modelling, enabling you to make informed decisions when selecting the most suitable machine learning algorithms for your specific classification and regression tasks.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 1: What is the purpose of spot-checking classification algorithms in machine learning, and why is it important for model selection?

Answer: Spot-checking classification algorithms is a method to discover which algorithms perform well on a given machine learning problem. It involves trying out a variety of algorithms and identifying the ones that show promising results. This process is crucial because you cannot know beforehand which algorithm will work best for your specific dataset. By spot-checking, you can systematically evaluate different algorithms and select the most suitable ones for further tuning and optimisation.

SAQ 2: Describe the role of pipelines in automating machine learning workflows. How do pipelines help prevent data leakage and ensure a fair evaluation of models?

Answer: Pipelines in machine learning are sequences of data transformations and model-fitting steps that are chained together. They help automate workflows by ensuring a consistent and reproducible process from data preprocessing to model evaluation. Pipelines are essential for preventing data leakage because

they encapsulate each step within a fold of cross-validation, ensuring that information from the test set doesn't leak into the training process. This separation is crucial for unbiased model evaluation and effective feature extraction. Pipelines improve code readability and maintainability and make it easier to reproduce experiments.

SAQ 3: Describe how a decision tree (CART) works in a classification task.

Answer: A decision tree (CART) works by recursively splitting the data based on the feature that results in the best separation of the target variable. It evaluates each feature and its values, minimising a cost function (like the Gini index) to create branches, and continues splitting until a stopping criterion is met, creating a tree-like structure.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **What is the primary purpose of spot-checking algorithms in machine learning?**
 - A) To fine-tune a model for better accuracy
 - B) To evaluate various algorithms and identify the most promising ones for further optimisation
 - C) To choose the best algorithm for deployment
 - D) To reduce the time required for training algorithms

Answer: B) To evaluate various algorithms and identify the most promising ones for further optimisation

Explanation: Spot-checking helps identify the best-performing algorithms for a given problem, allowing for further tuning and optimisation.

2. **Which of the following algorithms is a nonlinear machine learning algorithm?**

- A) Logistic Regression
- B) Linear Discriminant Analysis
- C) k-Nearest Neighbors
- D) Linear Regression

Answer: C) k-Nearest Neighbors

Explanation: k-Nearest Neighbours (KNN) is a nonlinear algorithm that classifies instances based on the majority class of their nearest neighbours in the feature space.

3. **Which machine learning technique works by recursively eliminating features and using the remaining ones to build a model?**

- A) Univariate Feature Selection
- B) Recursive Feature Elimination
- C) Feature Importance
- D) Principal Component Analysis

Answer: B) Recursive Feature Elimination

Explanation: Recursive Feature Elimination (RFE) iteratively removes the least important features to build a model using only the most relevant features.

4. **What is the main advantage of using cross-validation in model evaluation?**

- A) It guarantees better performance than a single train-test split.
- B) It reduces overfitting by training and testing on different subsets of data.
- C) It speeds up the training process.
- D) It eliminates the need for a validation set

Answer: B) It reduces overfitting by training and testing on different subsets of data

Explanation: Cross-validation helps mitigate overfitting by ensuring that the model is evaluated on different data subsets, providing a more reliable estimate of its performance.

5. **Which of the following algorithms is based on the assumption that features are conditionally independent given the class label?**

- A) Naive Bayes
- B) Support Vector Machines
- C) Logistic Regression
- D) Decision Trees

Answer: A) Naive Bayes

Explanation: Naive Bayes assumes that the features are conditionally independent given the class label, making it particularly effective for problems with high-dimensional data.

Tutor-Marked Assessments (TMAs)

1. Explain the concept of spot-checking classification algorithms. Why is it important to perform spot-checking when dealing with a classification problem in machine learning?
2. List the six machine learning algorithms mentioned in the text that are used for spot-checking classification problems. Briefly describe each algorithm's characteristics.
3. Describe how a pipeline can be used to automate the machine learning workflow. Include examples of how pipelines can prevent data leakage and ensure consistent data transformation across training and testing datasets.
4. Compare and contrast the different feature selection techniques, such as Univariate Selection, Recursive Feature Elimination, PCA, and Feature Importance. Provide examples of when each technique would be most beneficial.

STUDY SESSION SEVEN

IMPROVE PERFORMANCE

Introduction

In this session, we will delve into the concept of ensemble learning, a powerful approach that can significantly enhance the performance and accuracy of machine learning models. Ensemble methods involve combining multiple individual models to create a stronger and more robust predictive model. By leveraging the strengths of different algorithms, ensemble methods can mitigate the weaknesses of individual models and provide more accurate and reliable predictions. Throughout this session, we will explore various ensemble techniques, including Bagging, Boosting, and Voting, and understand how to implement them using popular Python libraries such as scikit-learn. Through hands-on examples and practical insights, you will gain a comprehensive understanding of how ensemble methods work and how to effectively apply them to your own machine learning projects. Let's embark on this journey to unlock the potential of ensemble learning and elevate the performance of our models.

Learning Outcomes for Study Session Seven

When you have studied this session, you should be able to:

- 7.1. Discuss the concept of ensemble learning and its role in enhancing machine learning model performance.
- 7.2. Identify the strengths and weaknesses of individual models and how ensembles mitigate these weaknesses.
- 7.3. Explain the key ensemble techniques: Bagging, Boosting, and Voting, and their respective benefits.

- 7.4. Implement ensemble methods using Python's scikit-learn library.
- 7.5. Create bagging ensembles using techniques like bagged decision trees, random forests, and extra trees.
- 7.6. Utilise boosting techniques including AdaBoost and stochastic gradient boosting.
- 7.7. Employ voting ensembles to combine predictions from multiple algorithms.
- 7.8. Grasp the importance of algorithm parameter tuning to optimise model performance.
- 7.9. Apply grid search and random search strategies for algorithm parameter tuning.
- 7.10. Gain insights into saving and loading machine learning models using pickle and Joblib.

7.1 Improve Performance with Ensemble

Ensembles can give you a boost in accuracy for your dataset. In this chapter, you will discover how you can create some of the most powerful types of ensembles in Python using scikit-learn. This lesson will step you through Boosting, Bagging and Majority Voting and show you how you can continue to ratchet up the accuracy of the models on your own datasets. After completing this lesson, you will know:

- How to use bagging ensemble methods such as bagged decision trees, random forests, and extra trees.
- How to use boosting ensemble methods such as AdaBoost and stochastic gradient boosting.
- How to use voting ensemble methods to combine the predictions from multiple algorithms.

7.2 Combine Models Into Ensemble Predictions

The three most popular methods for combining the predictions from different models are:

7.2.1 Bagging

Building multiple models (typically of the same type) from different subsamples of the training dataset.

7.2.2 Boosting

Building multiple models (typically of the same type), each of which learns to fix the prediction errors of a prior model in the sequence of models.

7.2.3 Voting

Building multiple models (typically of differing types) and simple statistics (like calculating the mean) are used to combine predictions. This assumes you are generally familiar with machine learning algorithms and ensemble methods and will not go into the details of how the algorithms work or their parameters. The Pima Indians onset of Diabetes dataset is used to demonstrate each algorithm. Each ensemble algorithm is demonstrated using 10-fold cross-validation and the classification accuracy performance metric.

7.2.4 Bagging Algorithms

Bootstrap Aggregation (or Bagging) involves taking multiple samples from your training dataset (with replacement) and training a model for each sample. The final output prediction is averaged across the predictions of all of the sub-models. The three bagging models covered in this section are as follows:

- Bagged Decision Trees.
- Random Forest.
- Extra Trees.
- Bagged Decision Trees

Bagging performs best with algorithms that have high variance. A popular example is decision trees, often constructed without pruning. In the example below, there is an example of using the Bagging Classifier with the Classification and Regression Trees algorithm (`DecisionTreeClassifier`). A total of 100 trees are created.

```
# Bagged Decision Trees for Classification
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(filename, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
seed = 7
kfold = KFold(n_splits=10, random_state=seed)
cart = DecisionTreeClassifier()
num_trees = 100
model = BaggingClassifier(base_estimator=cart, n_estimators=num_trees, random_state=seed)
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Example of Bagged Decision Trees Ensemble Algorithm.

Running the example, we get a robust estimate of model accuracy.

```
0.770745044429
```

Output of Bagged Decision Trees Ensemble Algorithm.

7.2.5 Random Forest

Random Forests is an extension of bagged decision trees. Samples of the training dataset are taken with replacement, but the trees are constructed in a way that reduces the correlation between individual classifiers. Specifically, rather than greedily choosing the best split point in the construction of each tree, only a random subset of features are considered for each split. You can construct a Random Forest model for classification using the `RandomForestClassifier` class. The example below demonstrates using Random Forest for classification with 100 trees and split points chosen from a random selection of 3 features.

```
# Random Forest Classification
from pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier
filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(filename, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
num_trees = 100
max_features = 3
kfold = KFold(n_splits=10, random_state=7)
model = RandomForestClassifier(n_estimators=num_trees, max_features=max_features)
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Example of Random Forest Ensemble Algorithm.

```
0.770727956254
```

Output of Random Forest Ensemble Algorithm.

7.2.6 Extra Trees

Extra Trees are another modification of bagging where random trees are constructed from samples of the training dataset. You can construct an Extra Trees model for classification using the `ExtraTreesClassifier` class. The example below provides a demonstration of extra trees with the number of trees set to 100 and splits chosen from 7 random features.

```
# Extra Trees Classification from
pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
sklearn.ensemble import ExtraTreesClassifier filename =
'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8] num_trees
    = 100
max_features = 7
kfold = KFold(n_splits=10, random_state=7)
model = ExtraTreesClassifier(n_estimators=num_trees,
    max_features=max_features)
results = cross_val_score(model, X, Y, cv=kfold)

print(results.mean())
```

Example of Extra Trees Ensemble Algorithm.

Running the example provides a mean estimate of classification accuracy.

```
0.760269993165
```

Output of Extra Trees Ensemble Algorithm.

7.3 Boosting Algorithms

Boosting ensemble algorithms create a sequence of models that attempt to correct the mistakes of the models before them in the sequence. Once created, the models make predictions, which may be weighted by their demonstrated accuracy, and the results are combined to create a final output prediction. The two most common boosting ensemble machine learning algorithms are:

- AdaBoost.
- Stochastic Gradient Boosting.

7.3.1 AdaBoost

AdaBoost was perhaps the first successful boosting ensemble algorithm. It generally works by weighting instances in the dataset by how easy or difficult they are to classify, allowing the algorithm to pay less attention to them in the construction of subsequent models. You can construct an AdaBoost model for classification using the `AdaBoostClassifier` class. The example below demonstrates the construction of 30 decision trees in sequence using the AdaBoost algorithm

```
# AdaBoost Classification from
pandas import read_csv
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score from
sklearn.ensemble import AdaBoostClassifier filename =
'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
read_csv(filename, names=names)
array = dataframe.values X =
array[:,0:8]
Y = array[:,8] num_trees
= 30 seed=7
kfold = KFold(n_splits=10, random_state=seed)
model = AdaBoostClassifier(n_estimators=num_trees, random_state=seed)
results = cross_val_score(model, X, Y, cv=kfold)
print(results.mean())
```

Example of AdaBoost Ensemble Algorithm.

Running the example provides a mean estimate of classification accuracy.

```
0.76045796309
```

Output of AdaBoost Ensemble Algorithm.

7.3.2 Stochastic Gradient Boosting

Stochastic Gradient Boosting (also called Gradient Boosting Machines) is one of the most sophisticated ensemble techniques. It is also a technique that is proving to be perhaps one of the best techniques available for improving performance via ensembles. You can construct a Gradient Boosting model for classification using the `GradientBoostingClassifier` class. The example below demonstrates Stochastic Gradient Boosting for classification with 100 trees.

```
# Stochastic Gradient Boosting Classification from pandas
import read_csv

from sklearn.model_selection import KFold

from sklearn.model_selection import cross_val_score
from sklearn.ensemble
import GradientBoostingClassifier
filename = 'pima-indians-diabetes.data.csv'

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = read_csv(filename, names=names)

array = dataframe.values
X = array[:,0:8]

Y = array[:,8]; seed = 7

num_trees = 100

kfold = KFold(n_splits=10, random_state=seed)

model = GradientBoostingClassifier(n_estimators=num_trees,
random_state=seed)
```

```
results = cross_val_score(model, X, Y, cv=kfold)
```

```
print(results.mean())
```

Example of Stochastic Gradient Boosting Ensemble Algorithm.

Running the example provides a mean estimate of classification accuracy.

```
0.764285714286
```

Output of Stochastic Gradient Boosting Ensemble Algorithm.

7.3.3 Voting Ensemble

Voting is one of the simplest ways of combining the predictions from multiple machine learning algorithms. It works by first creating two or more standalone models from your training dataset. A Voting Classifier can then be used to wrap your models and average the predictions of the sub-models when asked to make predictions for new data. The predictions of the sub-models can be weighted, but specifying the weights for classifiers manually or even heuristically is difficult. More advanced methods can learn how to best weight the predictions from sub-models, but this is called stacking (stacked aggregation) and is currently not provided in scikit-learn. You can create a voting ensemble model for classification using the `VotingClassifier` class. The code below provides an example of combining the predictions of logistic regression, classification and regression trees and support vector machines together for a classification problem.

```
# Voting Ensemble for Classification from
```

```
pandas import read_csv
```

```
from sklearn.model_selection import KFold
```

```
from sklearn.model_selection import cross_val_score from
```

```
sklearn.linear_model import LogisticRegression
```

```
from sklearn. tree import DecisionTreeClassifier from sklearn.svm import SVC
```

```
from sklearn.ensemble import VotingClassifier filename =
```

```
'pima-indians-diabetes.data.csv'
```

```
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
```

```
read_csv(filename, names=names)
```

```
array = dataframe.values X = array[:,0:8]
```

```
Y = array[:,8]
```

```
kfold = KFold(n_splits=10, random_state= 7) # create the sub-models
```

```
estimators = []
```

```
model1 = LogisticRegression() estimators.append(('logistic', model1)) model2 =
```

```
DecisionTreeClassifier() estimators.append(('cart', model2)) model3 = SVC()
```

```
estimators.append(('svm', model3))
```

```
# create the ensemble model
```

```
ensemble = VotingClassifier(estimators)
```

```
results = cross_val_score(ensemble, X, Y, cv=kfold) print(results.mean())
```

Example of the Voting Ensemble Algorithm.

Running the example provides a mean estimate of classification accuracy.

0.729049897471

Output of the Voting Ensemble Algorithm.

7.4 Improve Performance with Algorithm Tuning

Machine learning models are parameterised so that their behaviour can be tuned for a given problem. Models can have many parameters, and finding the best combination of parameters can be treated as a search problem. In this chapter you will discover how to tune the parameters of machine learning algorithms in Python using scikit-learn.

7.5 Machine Learning Algorithm Parameters

Algorithm tuning is a final step in the process of applying machine learning before finalising your model. It is sometimes called hyperparameter optimisation, where the algorithm parameters are referred to as hyperparameters, whereas the coefficients found by the machine learning algorithm itself are referred to as parameters. Optimisation suggests the search nature of the problem. As a search problem, you can use different search strategies to find a good and robust parameter or set of parameters for an algorithm on a given problem. Python scikit-learn provides two simple methods for algorithm parameter tuning:

- Grid Search Parameter Tuning.
- Random Search Parameter Tuning.

7.5.1 Grid Search Parameter Tuning

Grid search is an approach to parameter tuning that will methodically build and evaluate a model for each combination of algorithm parameters specified in a grid.

You can perform a grid search using the `GridSearchCV` class. The example below evaluates different alpha values for the Ridge Regression algorithm on the standard diabetes dataset. This is a one-dimensional grid search.

Example of a grid search for algorithm parameters.

Running the example lists out the optimal score achieved and the set of parameters in the grid that achieved that score. In this case the alpha value is 1.0.

```
0.279617559313  
1.0
```

Output of a grid search for algorithm parameters.

7.5.2 Random Search Parameter Tuning

Random search is an approach to parameter tuning that will sample algorithm parameters from a random distribution (i.e., uniform) for a fixed number of iterations. A model is constructed and evaluated for each combination of parameters chosen. You can perform a random search for algorithm parameters using the `RandomizedSearchCV` class. The example below evaluates different random alpha values between 0 and 1 for the Ridge Regression algorithm on the standard diabetes dataset. A total of 100 iterations are performed with uniformly random alpha values selected in the range between 0 and 1 (the range that alpha values can take).

```
# Randomized for Algorithm Tuning from
    pandas import read_csv
from scipy.stats import uniform
from sklearn.linear_model import Ridge
from sklearn.model_selection import RandomizedSearchCV filename =
    'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']

dataframe = read_csv(filename, names=names) array =
    dataframe.values
X = array[:,0:8]
Y = array[:,8]
param_grid = {'alpha': uniform()} model
    = Ridge()
rsearch = RandomizedSearchCV(estimator=model,
    param_distributions=param_grid, n_iter=100,
    random_state=7)
rsearch.fit(X, Y)
print(rsearch.best_score_)
print(rsearch.best_estimator_.alpha)
```

Example of a random search for algorithm parameters.

Running the example produces results much like those in the grid search example above. An optimal alpha value near 1.0 is discovered.

```
0.279617354112  
0.989527376274
```

Output of a random search for algorithm parameters.

Save and Load Machine Learning Models

Finding an accurate machine learning model is not the end of the project. In this chapter you will discover how to save and load your machine learning model in Python using scikit-learn. This allows you to save your model to a file and load it later in order to make predictions.

7.6 Finalise Your Model with Pickle

Pickle is the standard way of serialising objects in Python. You can use the pickle operation to serialise your machine learning algorithms and save the serialised format to a file. Later you can load this file to deserialise your model and use it to make new predictions. The example below demonstrates how you can train a logistic regression model on the Pima Indians onset of diabetes dataset, save the model to file and load it to make predictions on the unseen test set.

```
# Save Model Using Pickle from
    pandas import read_csv
from sklearn.model_selection import train_test_split from
    sklearn.linear_model import LogisticRegression from
    pickle import dump
from pickle import load

filename = 'pima-indians-diabetes.data.csv'
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =
    read_csv(filename, names=names)
array = dataframe.values X =
    array[:,0:8]
Y = array[:,8]
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.33,
    random_state=7)
# Fit the model on 33% model =
    LogisticRegression()
```

```
model.fit(X_train, Y_train) #  
    save the model to disk  
filename = 'finalized_model.sav'  
    dump(model, open(filename, 'wb'))  
    # some time later...  
# load the model from disk  
loaded_model = load(open(filename, 'rb')) result =  
    loaded_model.score(X_test, Y_test) print(result)
```

Example of using pickle to serialise and deserialise a model.

Running the example saves the model to `finalised_model.sav` in your local working directory. Loading the saved model and evaluating it provides an estimate of the accuracy of the model on unseen data.

```
0.755905511811
```

Output of using pickle to serialise and deserialise a model.

7.7 Finalise Your Model with Joblib

The Joblib library is part of the SciPy ecosystem and provides utilities for pipelining Python jobs. It provides utilities for saving and loading Python objects that make use of NumPy data structures efficiently. This can be useful for some machine learning algorithms that require a lot of parameters or store the entire dataset (e.g., k-Nearest Neighbours). The example below demonstrates how you can train a logistic regression model on the Pima Indians' onset of diabetes

dataset, save the model to a file using Joblib and load it to make predictions on the unseen test set.

```
# Save Model Using joblib from pandas import read_csv
```

```
from sklearn. model_selection import train_test_split from sklearn. linear_model  
import LogisticRegression from sklearn. externals.joblib import dump
```

```
from sklearn. externals.joblib import load filename =  
'pima-indians-diabetes.data.csv'
```

```
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class'] dataframe =  
read_csv(filename, names=names)
```

```
array = dataframe.values X = array[:,0:8]
```

```
Y = array[:,8]
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.33,  
random_state=7)
```

```
# Fit the model on 33% of the model = LogisticRegression() model. fit(X_train,  
Y_train)
```

```
# save the model to disc. filename = 'finalized_model.sav' dump(model,  
filename)
```

```
# some time later...
```

```
# load the model from disk
```

```
loaded_model = load(filename)
result = loaded_model.score(X_test, Y_test)
print(result)
```

Example of using joblib to serialise and deserialise a model.

Running the example saves the model to a file as 'finalized_model.sav' and also creates one file for each NumPy array in the model. After the model is loaded, an estimate of the accuracy of the model on unseen data is reported.

```
0.755905511811
```

Output of using Joblib to serialize and deserialize a model

In-Text Questions and Answers (ITQs and ITAs)

ITQ 7.1: What is the main goal of boosting algorithms in machine learning?

ITA: The main goal of boosting algorithms is to create a sequence of models that correct the errors of the models before them, with the final prediction being a weighted combination of the predictions from each model. This process improves the overall performance of the model.

ITQ 7.2: How does AdaBoost differ from Stochastic Gradient Boosting?

ITA: AdaBoost assigns weights to instances based on their difficulty to classify, giving more weight to misclassified instances, whereas Stochastic Gradient Boosting builds multiple decision trees and uses stochastic (random) methods to improve performance by adding random perturbations during training.

Conclusion

In conclusion, this session has provided a comprehensive exploration of ensemble learning, a technique that wields significant potential in enhancing the effectiveness of machine learning models. By amalgamating the strengths of diverse individual models, ensemble methods offer a robust approach to improving predictive accuracy. Throughout our discussion, we've delved into various ensemble techniques, including Bagging, Boosting, and Voting, while also gaining insight into their practical implementation using popular Python libraries like scikit-learn. This journey has equipped us with a solid understanding of how ensemble methods function and how they can be harnessed to optimise machine learning endeavours.

Summary

1. In this study session, you have learnt to enhance your machine learning models' performance through ensemble methods. Ensemble techniques involve combining multiple models to achieve higher accuracy and predictive power.
2. By exploring concepts like Bagging, Boosting, and Voting, and implementing them using Python's scikit-learn library, you've gained a practical understanding of how to apply these methods. Bagging includes creating multiple models from different subsets of the training dataset, while Boosting focuses on correcting the errors of prior models in a sequence. Voting combines predictions from various algorithms to create a final

output. With these insights, you're now equipped to harness the potential of ensemble learning for improved machine learning outcomes.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 1: What is the main goal of ensemble learning, and how does it contribute to improving the accuracy of machine learning models?

Answer: The main goal of ensemble learning is to enhance the predictive accuracy and robustness of machine learning models by combining the predictions of multiple individual models. This is achieved by leveraging the diversity of these models, each of which might have different strengths and weaknesses. Ensemble methods aim to reduce the potential for overfitting and increase generalisation performance by aggregating the predictions of multiple models. By combining the knowledge from various models, ensemble learning can lead to more accurate and reliable predictions compared to using a single model.

SAQ 2: Can you explain the difference between Bagging and Boosting ensemble techniques and provide an example of a scenario where each method would be particularly effective?

Answer: Bagging (Bootstrap Aggregating) and Boosting are two distinct ensemble techniques with different strategies. Bagging involves creating multiple models using different subsets of the training data, often by using the same algorithm. These models' predictions are then averaged or combined through majority voting. Bagging is effective when the base algorithm has high variance and is prone to overfitting, like decision trees. An example scenario for Bagging

could be in medical diagnosis, where multiple decision tree models trained on different subsets of patient data are combined to predict a disease diagnosis.

SAQ 3: Explain the concept of "data leakage" in machine learning and how Pipelines can prevent it.

Answer: Data leakage occurs when information from outside the training dataset is used to create the model, leading to overly optimistic performance estimates. Pipelines prevent this by ensuring that data preprocessing steps, like standardisation or feature extraction, are applied separately to the training and test datasets during cross-validation, preventing leakage.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **What is the primary function of the AdaBoost algorithm in machine learning?**

- A) To weight features based on importance
- B) To create a sequence of models that correct the errors of previous models
- C) To optimise hyperparameters
- D) To reduce the complexity of a model

Answer: B) To create a sequence of models that correct the errors of previous models

Explanation: AdaBoost is a boosting algorithm that sequentially creates models that correct the mistakes of previous models, leading to improved performance.

2. **Which of the following is a characteristic of Stochastic Gradient Boosting?**

- A) It uses only one decision tree to make predictions
- B) It introduces randomness into the model-building process to improve performance
- C) It doesn't combine multiple models
- D) It assigns weights to instances after every iteration

Answer: B) It introduces randomness into the model-building process to improve performance

Explanation: Stochastic Gradient Boosting adds randomness (stochasticity) to the model-building process, which improves the model's ability to generalise.

3. **Which ensemble method combines the predictions of multiple algorithms and averages the results?**

- A) Bagging
- B) Boosting
- C) Voting
- D) Stacking

Answer: C) Voting

Explanation: The Voting Ensemble method combines predictions from multiple models and averages them to produce a final prediction.

4. **What is the key purpose of using Grid Search in algorithm tuning?**

- A) To randomly choose a set of parameters
- B) To systematically evaluate all combinations of parameters within a grid

- C) To optimise the number of algorithms used in ensemble methods
- D) To select the best algorithm for a given dataset

Answer: B) To systematically evaluate all combinations of parameters within a grid

Explanation: Grid Search is used to evaluate all possible combinations of a set of algorithm parameters to find the best-performing configuration.

5. **Which of the following methods helps in preventing data leakage during cross-validation?**

- A) Using a fixed dataset for training and testing
- B) Creating separate datasets for training and testing
- C) Using Pipelines to constrain data transformations to the training set
- D) Randomly selecting features for each fold

Answer: C) Using Pipelines to constrain data transformations to the training set

Explanation: Pipelines ensure that transformations like scaling or feature extraction are applied to the training data only, preventing data leakage.

Tutor-Marked Assessments (TMAs)

1. Name three ensemble techniques that will be explored in this session.
2. Which Python library will be used to implement ensemble methods?
3. Describe the boosting algorithm, AdaBoost, and explain how it improves the performance of machine learning models. Provide an example use case where AdaBoost can be applied effectively.

4. Discuss the differences between Stochastic Gradient Boosting and other ensemble methods such as Bagging and Voting. Highlight when each of these methods would be most useful.

STUDY SESSION

EIGHT

HANDLING PROJECTS

Introduction

In this session, we will examine applied machine learning, focusing on the practical aspects of turning theoretical knowledge into actionable skills. While reading books and articles is valuable, true mastery of machine learning comes through hands-on practice. In this lesson, we introduce a simple yet effective six-step machine learning project template that serves as a roadmap for creating end-to-end predictive modelling projects in Python. By the end of this session, you will be equipped with the tools to structure your projects, apply the skills you've learnt, and successfully navigate the complexities of real-world machine learning tasks. With a clear understanding of the project template and its components, you'll be well-prepared to tackle a variety of machine learning challenges and drive accurate results for your datasets. Let's kick off this session and embark on the journey of honing your machine learning skills through practical projects.

Learning Outcomes for Study Session Eight

When you have studied this session, you should be able to:

- 8.1. Structure an end-to-end predictive modelling project.
- 8.2. Map the tasks you learnt about in Part II onto a project.
- 8.3. Use the structured project template to ensure an accurate result for your dataset.
- 8.4. Practice Machine Learning With Projects

8.1 Use A Structured Step-By-Step Process

Any predictive modelling machine learning project can be broken down into six common tasks:

1. Define Problem.
2. Summarise Data.
3. Prepare Data.
4. Evaluate Algorithms.
5. Improve Results.
6. Present Results.

Tasks can be combined or broken down further, but this is the general structure. To work through predictive modelling machine learning problems in Python, you need to map Python onto this process. The tasks may need to be adapted or renamed slightly to suit the Python way of doing things (e.g., Pandas for data loading and scikit-learn for modelling). The next section provides exactly this mapping and elaborates on each task and the types of sub-tasks and libraries that you can use.

8.2 Machine Learning Project Template in Python

This section presents a project template that you can use to work through machine learning problems in Python end-to-end.

8.2.1 Template Summary

Below is the project template that you can use in your machine learning projects in Python.

Python Project Template # 1. Prepare Problem

a) Load libraries # b) Load dataset

2. Summarise Data

a) Descriptive statistics # b) Data Visualisations # 3. Prepare Data

a) Data Cleaning

b) Feature Selection # c) Data Transforms

4. Evaluate Algorithms

a) Split-out validation dataset

b) Test options and evaluation metric # c) Spot Check Algorithms

d) Compare Algorithms # 5. Improve Accuracy

a) Algorithm Tuning # b) Ensembles

6. Finalise Model

a) Predictions on validation dataset

b) Create standalone model on entire training dataset # c) Save model for later use

Predictive modelling machine learning project template.

8.2.2 How To Use The Project Template

- Create a new file for your project (e.g., projectname.py).

- Copy the project template.
- Paste it into your empty project file.
- Start to fill it in, using recipes from this book and others.

8.3 Prepare Problem

This step is about loading everything you need to start working on your problem.

This includes:

- Python modules, classes and functions that you intend to use.
- Loading your dataset from CSV.

This is also the home of any global configuration you might need to do. It is also the place where you might need to make a reduced sample of your dataset if it is too large to work with. Ideally, your dataset should be small enough to build a model or create a visualisation within a minute, ideally 30 seconds. You can always scale up well-performing models later.

8.4 Summarise Data

This step is about better understanding the data that you have available. This includes understanding your data using:

- Descriptive statistics such as summaries.
- Data visualisations such as plots with Matplotlib, ideally using convenience functions from Pandas.

- Take your time and use the results to prompt a lot of questions, assumptions and hypotheses that you can investigate later with specialised models.

8.5 Prepare Data

- This step is about preparing the data in such a way that it best exposes the structure of the problem and the relationships between your input attributes and the output variable. This includes tasks such as:
- Cleaning data by removing duplicates, marking missing values and even imputing missing values.
- Feature selection where redundant features may be removed and new features developed.
- Data transforms where attributes are scaled or redistributed in order to best expose the structure of the problem later to learning algorithms.
- Start simple.
- Revisit this step often and cycle with the next step until you converge on a subset of algorithms and a presentation of the data that results in accurate or accurate-enough models to proceed.

8.6 Evaluate Algorithms

This step is about finding a subset of machine learning algorithms that are good at exploiting the structure of your data (e.g., have better than average skill). This involves steps such as:

- Separating out a validation dataset to use for later confirmation of the skill of your developed model.
- Defining test options using scikit-learn, such as cross-validation and the evaluation metric to use.
- Spot-checking a suite of linear and nonlinear machine learning algorithms.
- Comparing the estimated accuracy of algorithms.
- On a given problem you will likely spend most of your time on this and the previous step until you converge on a set of 3 to 5 well-performing machine learning algorithms.

8.7 Improve Accuracy

Once you have a shortlist of machine learning algorithms, you need to get the most out of them. There are two different ways to improve the accuracy of your models:

- Search for a combination of parameters for each algorithm using scikit-learn that yields the best results.
- Combine the predictions of multiple models into an ensemble prediction using ensemble techniques.
- The line between this and the previous step can blur when a project becomes concrete.
- There may be a little algorithm tuning in the previous step. And in the case of ensembles, you may bring more than a shortlist of algorithms forward to combine their predictions.

8.8 Finalise Model

Once you have found a model that you believe can make accurate predictions on unseen data, you are ready to finalise it. Finalising a model may involve sub-tasks such as:

- Using an optimal model tuned by scikit-learn to make predictions on unseen data.
- Creating a standalone model using the parameters tuned by scikit-learn.
- Saving an optimal model to file for later use.
- Once you make it this far, you are ready to present results to stakeholders and/or deploy your model to start making predictions on unseen data

In-Text Questions and Answers (ITQs and ITAs)

ITQ 8.1: What is the primary goal of using the six-step machine learning project template in Python?

ITA: The primary goal is to provide a structured approach for handling machine learning tasks, from problem definition and data preparation to model evaluation and finalising the model for deployment, ensuring systematic and efficient project completion.

ITQ 8.2: Why is data preparation an essential step in the machine learning project template?

ITA: Data preparation is critical because it ensures that the data is clean, relevant, and properly formatted for model building. Properly prepared data can

significantly enhance the accuracy and efficiency of machine learning models by making the relationships between features and the output variable clearer.

Conclusion

In conclusion, this session has provided you with a practical approach to machine learning through project-based learning. The importance of hands-on practice in machine learning cannot be overstated, as it enables you to apply theoretical knowledge to real-world datasets. The six-step machine learning project template introduced here serves as a structured guide to help you tackle predictive modelling projects effectively. By following this template, you have learnt how to define the problem, summarise and prepare data, evaluate algorithms, improve results, and present your findings. Through this process, you've gained the skills to work through machine learning challenges from start to finish, enhancing your ability to create accurate and reliable models. Armed with this knowledge, you are now equipped to confidently approach and excel in real-world machine learning projects.

Summary

In this study session, you have learnt to embrace a practical approach to machine learning by engaging in project-based learning. This hands-on experience is crucial for honing your skills and applying theoretical concepts to real-world datasets. The six-step machine learning project template introduced here has equipped you with a structured framework to effectively address predictive modelling challenges. By following this template, you've gained insights into

defining problems, summarising and preparing data, evaluating algorithms, improving results, and presenting findings. This comprehensive process empowers you to confidently navigate and excel in various machine learning projects, enhancing your ability to create accurate and robust models.

Self-Assessment Questions and Answers (SAQs and SAAs)

SAQ 1: How does project-based learning contribute to your understanding of machine learning concepts?

Answer: Project-based learning enriches understanding of machine learning by providing practical experience, bridging theoretical concepts with real-world applications. Learners actively engage in projects, developing problem-solving skills and adaptability while seeing how abstract concepts translate into tangible results. This approach also promotes interdisciplinary collaboration, mirroring real-world scenarios and equipping learners with both technical skills and teamwork abilities.

SAQ 2: Explain the importance of the "Prepare Problem" and "Summarise Data" steps in the machine learning project template.

Answer: The "Prepare Problem" step defines project objectives, aligning tasks with goals, while the "Summarise Data" step offers insights into data quality and distribution. Together, they establish a clear project direction and ensure informed decision-making throughout. These steps create a strong foundation for subsequent tasks and enable practitioners to navigate challenges and opportunities effectively.

SAQ 3: In what situations would you consider using ensemble techniques to improve model accuracy?

Answer: Ensemble techniques are particularly useful when you have multiple models that perform reasonably well but need to combine their strengths to achieve better performance. They can reduce variance, bias, and overfitting, making them ideal for improving the robustness of your model.

Multiple-Choice Questions and Answers (MCQs and MCAs)

1. **What is the first step in the machine learning project template?**

- A) Evaluate Algorithms
- B) Summarise Data
- C) Define Problem
- D) Prepare Data

Answer: C) Define Problem

Explanation: The first step is defining the problem, which sets the foundation for the entire project and determines the direction for data collection, model selection, and evaluation.

2. **Why is it necessary to perform data cleaning in the "Prepare Data" step of the machine learning project?**

- A) To identify the most relevant algorithms
- B) To remove irrelevant features from the dataset
- C) To ensure the data is free from errors and inconsistencies that could skew model results

D) To visualise data distributions

Answer: C) To ensure the data is free from errors and inconsistencies that could skew model results

Explanation: Data cleaning ensures that the dataset is accurate and consistent, preventing errors and inconsistencies that could negatively impact the model's performance.

3. **Which method in the project template helps compare the performance of different machine learning algorithms?**

A) Summarise Data

B) Evaluate Algorithms

C) Prepare Data

D) Finalise Model

Answer: B) Evaluate Algorithms

Explanation: The "Evaluate Algorithms" step allows you to test and compare the performance of different algorithms on your dataset, helping you select the best-performing ones.

4. **What is the primary benefit of using ensemble techniques in machine learning?**

A) They combine multiple models to improve accuracy and reduce variance

B) They simplify the algorithm

C) They reduce the need for data preparation

D) They make the model faster

Answer: A) They combine multiple models to improve accuracy and reduce

variance

Explanation: Ensemble methods combine the predictions of multiple models, which helps to improve overall accuracy and reduce the risk of overfitting.

5. **Which of the following tasks is part of the "Finalise Model" step in the machine learning project?**

- A) Feature selection
- B) Model deployment and saving for later use
- C) Algorithm evaluation
- D) Data summarization

Answer: B) Model deployment and saving for later use

Explanation: Finalising the model involves making predictions on new data, creating a standalone version of the model, and saving it for future use.

Tutor-Marked Assessments (TMAs)

1. Explain the six-step process of a machine learning project and discuss how each step contributes to building a robust machine learning model.
2. Describe how the "Prepare Data" step fits into the overall machine learning workflow. What tasks should be performed in this step to ensure the data is suitable for modelling?
3. Discuss the importance of evaluating algorithms in a machine learning project. What techniques can be used to evaluate algorithms, and how do you select the best-performing models?

4. Describe the role of ensemble techniques in improving machine learning model accuracy. Provide examples of situations where ensemble methods would be beneficial.

References/Further Reading

Brown, O. (2021). *Principles for evaluation of AI/ML model performance and evaluation gaps. Massachusetts Institute of Technology*. Retrieved from <https://www.ll.mit.edu/sites/default/files/publication/doc/principles-evaluation-ai-ml-model-performance-brown-md-62.pdf>

Franceschi, L. (2024). *Hyperparameter optimisation in machine learning*. arXiv. <https://arxiv.org/abs/2410.22854>

González, S. (2020). *A practical tutorial on bagging- and boosting-based ensemble methods*. ScienceDirect. <https://www.sciencedirect.com/science/article/pii/S1566253520303195>

Kruschel, S. (2025). *An evaluation of interpretable machine learning models. Journal of Information Systems*, 34(2), 123-145. <https://link.springer.com/article/10.1007/s12599-024-00922-2>

Miller, C. (2024). *A review of model evaluation metrics for machine learning applications*. PMC. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11420621/>

Naderalvojud, B. (2024). *Improving machine learning with ensemble learning on unbalanced datasets*. PMC. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10785929/>

Rainio, O. (2024). *Evaluation metrics and statistical tests for machine learning*. Nature. <https://www.nature.com/articles/s41598-024-56706-x>

Suggala, A. S., Liu, B., & Ravikumar, P. (2020). *Generalised boosting*. NeurIPS. <https://papers.neurips.cc/paper/2020/file/63f44623dd8686aba388944c8>

810087f-Paper.pdf

Zhang, Y. (2022). *A review of ensemble learning algorithms used in remote sensing applications*. MDPI. <https://www.mdpi.com/2076-3417/12/17/8654>

Zhou, Z. H. (2018). *Ensemble methods: Foundations and algorithms*. CRC Press.

Glossary of Terms

Activation Function: A mathematical function applied to the output of a neurone in a neural network. It introduces non-linearity, allowing the network to learn complex relationships in data.

Adversarial Networks: A type of neural network architecture consisting of a generator and a discriminator, used in Generative Adversarial Networks (GANs) to create realistic data.

Backpropagation: An algorithm used in training neural networks by calculating and adjusting the weights based on the gradient of the loss function with respect to the network's output.

Batch Normalisation: A technique used to improve the training speed and stability of neural networks by normalising the outputs of neurones within a mini-batch.

Bias-Variance Tradeoff: The balance between a model's ability to fit the training data closely (low bias) and its ability to generalise to new, unseen data (low variance).

Clustering: A type of unsupervised learning where data points are grouped into clusters based on similarities, often used for data exploration and pattern discovery.

Convolutional Neural Network (CNN): A specialised neural network architecture designed for processing grid-like data, such as images, by using convolutional layers to automatically learn hierarchical features.

Cross-Entropy Loss: A commonly used loss function in classification tasks, measuring the dissimilarity between predicted probabilities and true class labels.

Decision Tree: A tree-like model where each internal node represents a feature, each branch a decision rule, and each leaf a prediction; used for both classification and regression tasks.

Dimensionality Reduction: The process of reducing the number of input features in a dataset while retaining its essential information, often achieved through techniques like PCA.

Dropout: A regularisation technique used in neural networks where randomly selected neurones are ignored during training to prevent overfitting.

Embedding: A mapping of categorical variables into a continuous vector space, commonly used in natural language processing to represent words or entities.

Ensemble Learning: A method where multiple models are combined to improve overall performance, commonly seen in techniques like Random Forest and Gradient Boosting.

Feature Engineering: The process of selecting, transforming, and creating features from raw data to enhance a machine learning model's performance.

Fine-Tuning: The process of adjusting hyperparameters or further training a pre-trained model on a specific task or dataset.

Generative Adversarial Network (GAN): A type of neural network framework consisting of a generator and a discriminator, used to generate realistic data.

Gradient Descent: An optimisation algorithm used to minimise a loss function by iteratively adjusting the model's parameters in the direction of the steepest descent.

Hyperparameter Tuning: The process of selecting optimal hyperparameters for a machine learning model to achieve better performance.

Inference: The process of applying a trained machine learning model to make predictions on new, unseen data.

K-Means Clustering: A popular clustering algorithm that partitions data points into 'K' clusters based on their similarity to the mean of each cluster.

Kernel Methods: Techniques that transform data into a higher-dimensional space to make it easier to separate and classify.

Learning Rate: A hyperparameter that determines the step size in gradient descent, influencing how quickly a model learns.

Loss Function: A mathematical function that quantifies the difference between predicted and actual values, used to guide model training.

Markov Chain Monte Carlo (MCMC): A family of algorithms used for sampling from complex probability distributions, often applied in Bayesian inference.

Maximum Likelihood Estimation (MLE): A method for estimating the parameters of a statistical model by finding values that maximise the likelihood of the observed data.

Neural Architecture Search: An approach that automates the process of designing optimal neural network architectures.

Overfitting: When a model learns the training data too well, capturing noise and leading to poor generalisation on unseen data.

PCA (Principal Component Analysis): A dimensionality reduction technique that transforms data into a new coordinate system to maximise variance along principal components.

Precision and Recall: Evaluation metrics for classification tasks, where precision measures the ratio of correctly predicted positive observations to the total predicted positives, and recall measures the ratio of correctly predicted positives to the total actual positives.

Random Forest: An ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and control overfitting.

Regularisation: Techniques applied to prevent overfitting by adding penalties to the loss function based on the complexity of the model.

Reinforcement Learning: A paradigm of machine learning where an agent learns how to take actions in an environment to maximise a reward signal.

Residual Networks (ResNets): A type of neural network architecture designed to address the vanishing gradient problem by using skip connections to jump over some layers.

RNN (Recurrent Neural Network): A type of neural network designed for sequences and time-series data, with connections that loop back on themselves to capture temporal dependencies.

Self-Organising Maps (SOM): Unsupervised learning algorithms that map high-dimensional data onto a lower-dimensional grid, used for visualisation and clustering.

Semi-Supervised Learning: A learning paradigm that uses both labelled and unlabelled data to train models, often useful when obtaining labelled data is expensive.

Sentiment Analysis: The process of determining the sentiment or emotion expressed in a piece of text, often used in natural language processing applications.

Sequence-to-Sequence (Seq2Seq): A model architecture designed to transform input sequences into output sequences, often used in tasks like machine translation.

Singular Value Decomposition (SVD): A matrix factorisation technique that breaks down a matrix into three other matrices to extract latent features.

Stochastic Gradient Descent (SGD): An optimisation algorithm that estimates the gradient of the loss function using a random subset of the training data.

Support Vector Machine (SVM): A supervised learning algorithm used for classification and regression tasks, aiming to find a hyperplane that best separates classes.

Tensor: A mathematical object representing multi-dimensional arrays, fundamental to the representation of data in neural networks.

Transfer Learning: A technique where a pre-trained model is used as a starting point for a new task, often resulting in improved performance and faster convergence.

Underfitting: When a model is too simple to capture the underlying patterns in the data, leading to poor performance on both training and test data.

Unsupervised Learning: Machine learning tasks where the model learns patterns from unlabelled data without explicit target labels.

Variational Autoencoder (VAE): A type of generative model that combines autoencoders with variational inference, used for generating new data samples.

Weight Initialisation: The process of setting initial values for the weights of a neural network, which can significantly affect the training process.

Word Embedding: A technique in natural language processing that represents words as dense vectors in a continuous space, capturing semantic relationships.

XGBoost: An optimised gradient boosting library used for classification and regression tasks, known for its high performance and flexibility.

Zero-Shot Learning: A learning paradigm where a model can make predictions on classes it has never seen during training.